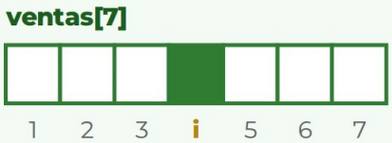
BACHILLERATO TÉCNICO EN SERVICIOS · ESPECIALIDAD INFORMÁTICA

Algoritmos, archivos y consultas

ALGORÍTMICA

2.° Curso






```
Para i <- 1 Hasta 7
    total <- total + ventas[i]
```

Productos · filtro: stock < 10

MIX-01	Sándwich mixto	**8**
SOP-01	Sopa paraguaya	**6**

los demás registros siguen en la tabla: el filtro no borra

Libro del estudiante con prácticas de laboratorio

Enfoque «Aprender Haciendo» · PSeInt y Microsoft Access

Caso integrador: Copetín Karumbé

Programa de estudio del Ministerio de Educación y Ciencias · Diseño Curricular de Informática

República del Paraguay · 2026

Presentación

Este libro acompaña el segundo año de Algorítmica del Bachillerato Técnico en Servicios, Especialidad Informática, conforme al programa oficial del Ministerio de Educación y Ciencias. Reúne en un solo volumen las 21 clases del año y sus 21 prácticas: ya no hace falta un cuaderno aparte.

Vas a aprender a resolver problemas con algoritmos y a escribirlos en pseudocódigo para ejecutarlos en PSeInt: decisiones, ciclos, vectores, matrices, ordenamiento, módulos y cadenas. Después vas a conocer cómo se guardan los datos en archivos y cómo los organiza el sistema operativo, y vas a terminar consultando una base de datos en Microsoft Access.

Cada clase abre con una ficha de capacidad e indicadores, desarrolla el tema con ejemplos resueltos, tablas y figuras, y cierra con actividades para ejercitar, resolver y decidir. Enseguida viene la Práctica con el mismo número: se hace en PSeInt, en el Explorador de archivos o en Access, con datos nuevos, pasos guiados y puntos de control para que verifiques tu trabajo.

El caso que nos acompaña en las clases es el Copetín Karumbé, un emprendimiento gastronómico paraguayo. En las prácticas trabajás con la Librería escolar Arandu, y en las evaluaciones aparecen una heladería y una fotocopiadora: los mismos algoritmos sirven para negocios distintos. El año termina con el Proyecto Final Integrador para la Feria de Informática.

Cómo está organizado este libro

Pieza	Qué encontrás
Clase N	Ficha con la capacidad del programa oficial e indicadores · desarrollo con ejemplos resueltos · recuadros Concepto clave, Ejemplo, Errores frecuentes y, cuando corresponde, En Paraguay, Ejemplo cotidiano o Aplicación profesional · actividades de aplicación.
Práctica N	Empieza en página propia y se puede fotocopiar suelta: competencia, lo que necesitás saber, actividades con objetivo, modelo, pasos y punto de control, y un desafío final. Algunas suman una transferencia con revisión entre pares.
Punto de control	Te dice cómo comprobar tu trabajo, sin darte la respuesta: si no se cumple, revisá tus pasos antes de seguir.
Evaluaciones	Al cierre de cada unidad (banda ámbar) y de cada etapa (banda azul), en páginas propias.

Índice

Prueba diagnóstica — ¿Qué sabés ya? **4**

UNIDAD 1 — Del problema al programa: estructuras de control **5**

Clase 1 — Del problema al algoritmo: diagrama de flujo, pseudocódigo y tipos de datos 5

Práctica 1 — La compra de útiles: del algoritmo en papel al programa en PSeInt 9

Clase 2 — La decisión simple: la estructura Si… Entonces 10

Práctica 2 — Descuentos y avisos con Si simple 15

Clase 3 — Dos caminos: la estructura Si… Entonces… Sino 16

Práctica 3 — Envíos y pagos con Si… Sino 19

Clase 4 — Condiciones combinadas: los operadores lógicos Y, O, NO 20

Práctica 4 — Condiciones compuestas: descuento estudiantil y medios de pago 23

Clase 5 — Decisiones dentro de decisiones: anidamiento y selección múltiple (Segun) 24

Práctica 5 — Clasificar pedidos y armar el menú de la librería 27

Clase 6 — Repetir con condición: Mientras, Repetir… Hasta Que y el contador 28

Práctica 6 — Validar datos y llegar a la meta con ciclos 32

Clase 7 — Repetir una cantidad conocida: Para, acumulador, bandera y centinela 33

Práctica 7 — La semana de la librería: Para, acumulador, centinela y bandera 36

Evaluación de la Unidad 1 38

UNIDAD 2 — Estructuras de datos y modularización **40**

Clase 8 — Vectores: concepto, declaración, carga y recorrido 40

Práctica 8 — El vector de precios y la consulta por número 43

Clase 9 — Operaciones con vectores: suma, promedio, máximo, mínimo y búsqueda 44

Práctica 9 — Recorrer el vector de unidades vendidas 47

Clase 10 — Matrices: arreglos bidimensionales, recorrido y totales por fila y columna 48

Práctica 10 — La matriz de ventas de la librería 51

Clase 11 — Ordenación de vectores: método burbuja optimizada y método de selección 53

Práctica 11 — El ranking de la librería: burbuja y selección 56

Clase 12 — Generación de números aleatorios y su uso 57

Práctica 12 — Sorteos y simulaciones con Azar 60

Clase 13 — Modularización: subprogramas, procedimientos y funciones; variables locales y globales 61

Práctica 13 — La caja de la librería en módulos 64

Clase 14 — Parámetros por valor y por referencia; datos de tipo cadena 65

Práctica 14 — Parámetros por referencia y códigos de artículos 68

Evaluación de la Unidad 2 70

Evaluación integradora de la 1.ª etapa 72

UNIDAD 3 — Sistema de archivos **74**

Clase 15 — Archivos: concepto, características y tipos; memoria principal y secundaria 74

Práctica 15 — Los archivos de la librería: tipos, propiedades y formato 77

Clase 16 — Administración de archivos: operaciones básicas y relación con el sistema operativo 79

Práctica 16 — Escribir, leer y agregar: operaciones con archivos 82

Clase 17 — Seguridad y protección de archivos; organización, acceso y buffers 84

Práctica 17 — Proteger y respaldar los archivos de la librería 87

Clase 18 — El sistema de archivos en la estructura visible del sistema operativo 88

Práctica 18 — El árbol de carpetas de la librería 91

Evaluación de la Unidad 3 92

UNIDAD 4 — Consultas a bases de datos **94**

Clase 19 — Bases de datos y filtros: mostrar solo lo que cumple una condición 94

Práctica 19 — La tabla Artículos en Access: diseño, carga y filtros 97

Clase 20 — Consultas: selección, paramétrica, cálculo de totales y campos calculados 99

Práctica 20 — Consultas de la librería: selección, parámetro, totales y campo calculado 102

Clase 21 — Consultas de referencias cruzadas y buenas prácticas 103

Práctica 21 — La referencia cruzada de la librería y el informe de reposición 105

Evaluación de la Unidad 4 107

Evaluación integradora de la 2.ª etapa 109

Proyecto Final Integrador — El sistema de gestión de un emprendimiento **111**

Prueba diagnóstica — ¿Qué sabés ya?

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Esta prueba no lleva nota: sirve para que tu docente conozca tu punto de partida antes de comenzar el año. Respondé con lo que sabés, sin ayuda.

1. Ordená los pasos para cobrar una venta en un copetín, numerándolos del 1 al 4: (a) entregar el vuelto · (b) recibir el pago · (c) calcular el total · (d) anotar los productos pedidos.

2. Calculá el importe de una venta de 3 empanadas a G. 5.000 cada una.

3. Sobre una compra de G. 60.000 se aplica un descuento del 10 %. Calculá el descuento y el monto final a pagar.

4. Marcá cuáles de estos datos son números y cuáles son texto: 42 · «chipa» · 12,5 · «K».

5. V o F: una computadora ejecuta las instrucciones exactamente en el orden en que se le dan.

6. Escribí, en tus palabras y en orden, los pasos que seguís para preparar tu mochila antes de ir al colegio (mínimo 4 pasos).

UNIDAD 1 — Del problema al programa: estructuras de control

En esta unidad vas a pasar del problema al programa: planificar antes de escribir, elegir los tipos de datos y usar las tres estructuras de control —secuencia, decisión y repetición— hasta escribir y ejecutar algoritmos completos en PSeInt.

Clase 1 — Del problema al algoritmo: diagrama de flujo, pseudocódigo y tipos de datos

Capacidad	Identifica estrategias para el planeamiento y solución de problemas con algoritmos.
Tema	Del problema al algoritmo: diagrama de flujo, pseudocódigo y tipos de datos
Indicadores de logro	• Distingue las tres formas de expresar un algoritmo: prosa, diagrama de flujo y pseudocódigo. • Identifica el tipo de dato (entero, real, carácter, cadena, lógico) que corresponde a cada dato de un problema. • Reconoce las tres estructuras básicas de control en un algoritmo dado.

Resolver un problema es planificar antes de escribir

Programar no empieza en la computadora: empieza en el papel. Antes de escribir una sola línea, conviene entender el problema, separar los datos que entran de los resultados que salen y ordenar los pasos. Ese plan de pasos ordenados y finitos que resuelve un problema es un algoritmo.

Dos palabras más completan el vocabulario de partida. Un programa es un algoritmo escrito en un lenguaje que la computadora puede ejecutar: el mismo plan, ahora con reglas estrictas de escritura. Un sistema es un conjunto de programas y datos que trabajan juntos para resolver las necesidades de una organización: el sistema de gestión del copetín tendrá un programa de caja, uno de stock y uno de informes, que comparten los mismos datos.

El Copetín Karumbé creció. Ya no alcanza con anotar las ventas en un cuaderno: hace falta un sistema que calcule totales, controle el stock y arme informes. Todo ese sistema se construye a partir de algoritmos bien pensados.

Tres formas de escribir el mismo algoritmo

Un mismo algoritmo se puede expresar de tres maneras. En prosa, con palabras corrientes. Con un diagrama de flujo, usando símbolos (óvalo para inicio y fin, romboide para entrada y salida, rectángulo para procesos, rombo para decisiones). Y en pseudocódigo, un lenguaje intermedio, ordenado y con sangría, muy parecido a un programa pero sin las reglas estrictas de un lenguaje real.

En este curso vas a trabajar sobre todo con pseudocódigo, porque se traduce casi palabra por palabra a un lenguaje de programación como el que se usa en PSeInt.

Ejemplo — un algoritmo secuencial en pseudocódigo

Calcular el importe de una venta de gaseosas:

```
Algoritmo VentaGaseosa
    Definir precio, cantidad, importe Como Entero
    precio <- 8000
    Escribir "Cantidad de gaseosas:"
    Leer cantidad
    importe <- precio * cantidad
    Escribir "Total a pagar: ", importe
FinAlgoritmo
```

Si el cliente pide 3 gaseosas: importe <- 8000 * 3 = 24.000.

Los tipos de datos

Cada dato que maneja un algoritmo pertenece a un tipo. Los enteros son números sin decimales (cantidad de empanadas: 15). Los reales llevan decimales (un promedio: 600.000,0). Los caracteres son un solo signo ("A", "7", "$"). Las cadenas son texto ("Sándwich mixto"). Los lógicos solo valen Verdadero o Falso.

Elegir bien el tipo evita errores: la cantidad de coquitos vendidos es un entero, nunca puede ser 12,5.

Dato del copetín	Ejemplo	Tipo de dato
Cantidad de empanadas	15	Entero
Promedio de ventas	600000.0	Real
Nombre del producto	"Coquito"	Cadena
Inicial de la categoría	"S"	Carácter
¿Aplica descuento?	Verdadero	Lógico

Las tres estructuras de control

Todo algoritmo, por más largo que sea, se arma con solo tres estructuras. La secuencial: instrucciones una detrás de otra. La condicional: el algoritmo elige entre caminos según una condición. La repetitiva: un bloque se repite varias veces. Esta unidad recorre las tres.

> **Concepto clave**
> Un algoritmo es una secuencia ordenada y finita de pasos que resuelve un problema. Se puede expresar en prosa, en diagrama de flujo o en pseudocódigo.

En el laboratorio vas a usar PSeInt, un programa gratuito que ejecuta pseudocódigo en español y muestra el resultado paso a paso, incluso instrucción por instrucción.

Observá en la figura la simbología normalizada del diagrama de flujo y cómo cada una de las tres estructuras se dibuja con ella, siempre con un ejemplo de la caja del copetín.

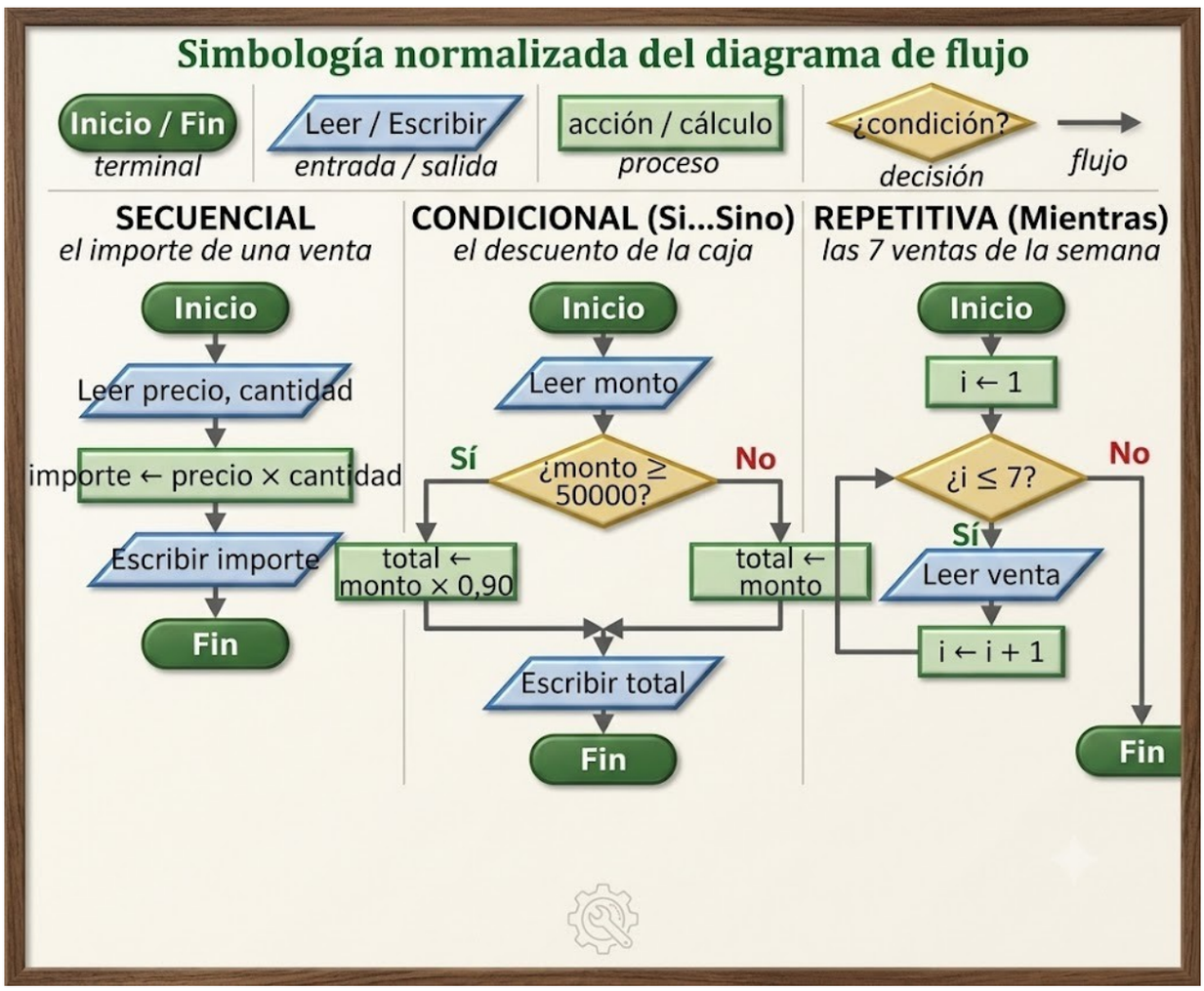


Figura 1.1 — Simbología normalizada del diagrama de flujo y las tres estructuras de control, con ejemplos de la caja del Copetín Karumbé.

El método de trabajo: cuatro pasos antes de programar

Un algoritmo que sale bien casi nunca sale de un tirón. Los programadores siguen un método de cuatro pasos: entender el problema, planificar la solución, escribir el algoritmo y probarlo con datos reales. Saltearse el primer paso es la causa más común de programas que «andan» pero calculan mal.

Paso	Qué se hace	En el copetín
1. Entender	Identificar qué se pide, qué datos entran y qué debe salir	«Calcular cuánto paga un cliente por su pedido»
2. Planificar	Elegir las estructuras: ¿hay decisión?, ¿hay repetición?	Solo cálculos: entrada → proceso → salida
3. Escribir	Redactar el pseudocódigo o dibujar el diagrama de flujo	Algoritmo PedidoCompleto en pseudocódigo
4. Probar	Ejecutar a mano con números reales (prueba de escritorio)	Verificar con un pedido verdadero del mostrador

La prueba de escritorio

Probar un algoritmo a mano se llama prueba de escritorio: ejecutás cada instrucción en orden, anotando en una tabla el valor de cada variable después de cada paso. Es la herramienta más barata para encontrar errores: no hace falta ni computadora, solo lápiz, papel y atención.

La regla es simple: una fila por instrucción ejecutada, una columna por variable. Si al final los valores no son los esperados, el error está en alguna fila de la tabla — y la tabla te dice exactamente en cuál.

Ejemplo resuelto paso a paso — el pedido completo

Un cliente pide 4 chipas (G. 3.000 cada una), 2 empanadas (G. 5.000) y 1 gaseosa (G. 8.000). El algoritmo calcula el importe de cada producto y el total del pedido.

Ejemplo — el algoritmo del pedido completo

```
Algoritmo PedidoCompleto
    impChipas <- 4 * 3000
    impEmpanadas <- 2 * 5000
    impGaseosa <- 1 * 8000
    total <- impChipas + impEmpanadas + impGaseosa
    Escribir "Total del pedido: ", total
FinAlgoritmo
```

La prueba de escritorio, instrucción por instrucción:

Paso	Instrucción	impChipas	impEmpanadas	impGaseosa	total
1	impChipas <- 4 * 3000	12.000	—	—	—
2	impEmpanadas <- 2 * 5000	12.000	10.000	—	—
3	impGaseosa <- 1 * 8000	12.000	10.000	8.000	—
4	total <- suma de los tres	12.000	10.000	8.000	30.000

El total del pedido es G. 30.000: 12.000 + 10.000 + 8.000. Cada variable guarda un dato intermedio con nombre propio, y eso hace al algoritmo fácil de leer y de corregir.

Concepto clave

La prueba de escritorio es la ejecución manual de un algoritmo anotando el valor de cada variable paso a paso. Todo algoritmo de este libro puede (y debe) verificarse así antes de pasarlo a PSeInt.

Variables y constantes: los nombres importan

Una variable es un casillero con nombre; elegir bien ese nombre es la primera decisión de calidad de un algoritmo. impChipas cuenta su propia historia; x no cuenta nada. La convención del libro: nombres en minúscula que digan qué guardan, sin espacios ni acentos, con mayúscula interna para separar palabras (impChipas, totalPedido).

Nombre	¿Sirve?	Por qué
total	sí	corto y claro: el total del pedido
impGaseosa	sí	dice el dato y el producto
x2	no	no dice nada: dentro de diez líneas nadie lo entiende
precio unitario	no	los nombres no llevan espacios

Los valores que no cambian nunca durante la ejecución — el precio de lista de la chipa, el umbral del descuento — se comportan como constantes: se definen una vez al principio y se usan por nombre. Si mañana la chipa sube de precio, se corrige UNA línea y no veinte.

Actividades de aplicación

Ejercitá:

1. Clasificá el tipo de dato de: 42 ; 3,14 ; "chipa" ; Verdadero ; "K".

2. Traducí a pseudocódigo: "guardar 8000 en la variable precio".

3. Escribí la instrucción de salida que muestre el texto Total a pagar: seguido del valor de importe.

4. Indicá qué estructura de control (secuencial, condicional o repetitiva) se necesita para "repetir el cobro hasta que no queden clientes".

5. Ordená estos pasos para calcular un importe: (a) Leer cantidad, (b) importe <- precio*cantidad, (c) Escribir importe, (d) precio <- 5000.

Resolvé (caso Copetín Karumbé):

6. Escribí en pseudocódigo un algoritmo que lea la cantidad de empanadas vendidas (precio 5.000) y muestre el importe total.

7. Para la venta de 4 sándwiches mixtos a 12.000 cada uno, calculá el importe y decidí qué tipo de dato conviene para la variable importe.

8. Dibujá con símbolos (en palabras) el diagrama de flujo del algoritmo de la venta de empanadas: indicá qué va en cada símbolo (óvalo, romboide, rectángulo).

Pensá y decidí (justificá tu respuesta):

9. V o F: un algoritmo puede tener pasos infinitos. Justificá.

10. Elegí el tipo correcto: la cantidad de coquitos vendidos se guarda como (entero / real) porque…

11. V o F: el diagrama de flujo y el pseudocódigo pueden expresar el mismo algoritmo. Justificá.

Desafío:

12. Redactá en pseudocódigo un algoritmo que lea el precio y la cantidad de un producto cualquiera del copetín y muestre el importe. Probalo mentalmente con 6 cocidos a 6.000 y anotá el resultado.

Práctica 1 — La compra de útiles: del algoritmo en papel al programa en PSeInt

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Escribir un algoritmo secuencial con sus tipos de datos, ejecutarlo en PSeInt y verificar el resultado con un cálculo hecho a mano.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

Un algoritmo secuencial ejecuta sus pasos en orden: leer los datos, calcular y mostrar. Antes de ejecutar, conviene saber qué resultado esperar.

En PSeInt, Definir declara el tipo de cada variable (Entero, Real, Caracter, Logico), Leer guarda lo que escribe el usuario, <- asigna y Escribir muestra.

Abrí PSeInt y, en Configurar → Opciones del Lenguaje (perfiles), seleccioná el perfil Flexible: es el perfil de referencia de este libro. En la Librería escolar Arandu el cuaderno cuesta 8.000 y el bolígrafo 3.000.

Actividad 1 — El algoritmo en papel

Objetivo: *Escribir en pseudocódigo el algoritmo que lee cuántos cuadernos y cuántos bolígrafos compra un cliente, calcula el total y el vuelto de su pago.*

Modelo — Estructura que debe tener tu algoritmo

1) Definir las variables y su tipo.
2) Leer las dos cantidades.
3) Calcular el total con los precios de la librería.
4) Leer el pago y calcular el vuelto.
5) Mostrar el total y el vuelto.

Pasos:

1. Elegí nombres claros para las variables (cantCuad, cantBol, total, pago, vuelto) y decidí el tipo de cada una.
2. Escribí el algoritmo completo en tu carpeta, con sangría.
3. Calculá a mano el total y el vuelto para un cliente que compra 3 cuadernos y 4 bolígrafos y paga con 50.000. Anotalos al costado.

Punto de control: Cada variable tiene un tipo justificado y tu cálculo a mano está anotado antes de pasar a la computadora.

Actividad 2 — Ejecutarlo en PSeInt

Objetivo: *Pasar el algoritmo a PSeInt, ejecutarlo con dos clientes y comparar con el cálculo a mano.*

Modelo — Esqueleto para completar (las líneas «…» las escribís vos)

```
Algoritmo CompraUtiles
    Definir cantCuad, cantBol, total, pago, vuelto Como Entero
    Escribir "Cantidad de cuadernos:"
    Leer cantCuad
    …
    total <- …
    Escribir "Total a pagar: ", total
    …
    vuelto <- …
    Escribir "Vuelto: ", vuelto
FinAlgoritmo
```

Pasos:

1. Completá el esqueleto con lo que escribiste en papel.
2. Ejecutalo con el cliente de la Actividad 1 (3 cuadernos, 4 bolígrafos, pago de 50.000).
3. Ejecutalo con un segundo cliente que elijas vos; calculá su total a mano antes de ejecutar.
4. Guardá el archivo como CompraUtiles.psc.

Punto de control: En los dos clientes, el total y el vuelto de la pantalla coinciden con los que calculaste antes de ejecutar.

Desafío final (para quienes terminan antes)

Agregá una variable Real que calcule el precio medio por artículo (total dividido por la cantidad de artículos). Probalo con 3 cuadernos y 5 bolígrafos y explicá por qué esa variable no puede ser Entero en general.

Clase 2 — La decisión simple: la estructura Si… Entonces

Capacidad	Utiliza un lenguaje de programación en la solución de problemas aplicando estructura de control
Tema	La decisión simple: la estructura Si… Entonces
Indicadores de logro	• Escribe una condición usando operadores relacionales (>, <, >=, <=, =, <>). • Construye una estructura Si… Entonces que ejecuta acciones solo cuando la condición es verdadera. • Predice la salida de un algoritmo con una condición simple para distintos datos de entrada.

Cuando el algoritmo tiene que decidir

Hasta ahora los pasos se ejecutaban siempre, uno detrás de otro. Pero muchos problemas exigen decidir. En el copetín, el descuento del 10 % se aplica solo si la venta llega a G. 50.000. Ese "solo si" es una decisión, y en pseudocódigo se escribe con la estructura Si… Entonces.

La condición y los operadores relacionales

Una condición es una pregunta que solo puede responderse Verdadero o Falso. Se arma con operadores relacionales: mayor que (>), menor que (<), mayor o igual (>=), menor o igual (<=), igual (=) y distinto (<>).

Por ejemplo, monto >= 50000 es Verdadero cuando la venta alcanza el umbral, y Falso cuando no.

Ejemplo — aplicar el descuento con un Si simple

```
Si monto >= 50000 Entonces
    monto <- monto - monto * 0.10
FinSi
Escribir "Total: ", monto
```

Venta de 5 sándwiches mixtos: monto = 60.000. Como 60.000 >= 50.000 es Verdadero, se descuenta 6.000 y el total pasa a 54.000.

La sintaxis del Si simple

La estructura tiene tres partes: la palabra Si, la condición, y el bloque de acciones que se ejecuta solo si la condición es verdadera. Si es falsa, el bloque se saltea y el algoritmo sigue.

La sangría (el bloque corrido hacia adentro) no es decorativa: muestra de un vistazo qué acciones dependen de la condición.

El flujograma muestra la forma del Si simple: el rombo pregunta, el camino del Sí pasa por el bloque y el camino del No lo saltea; los dos se juntan antes de seguir.

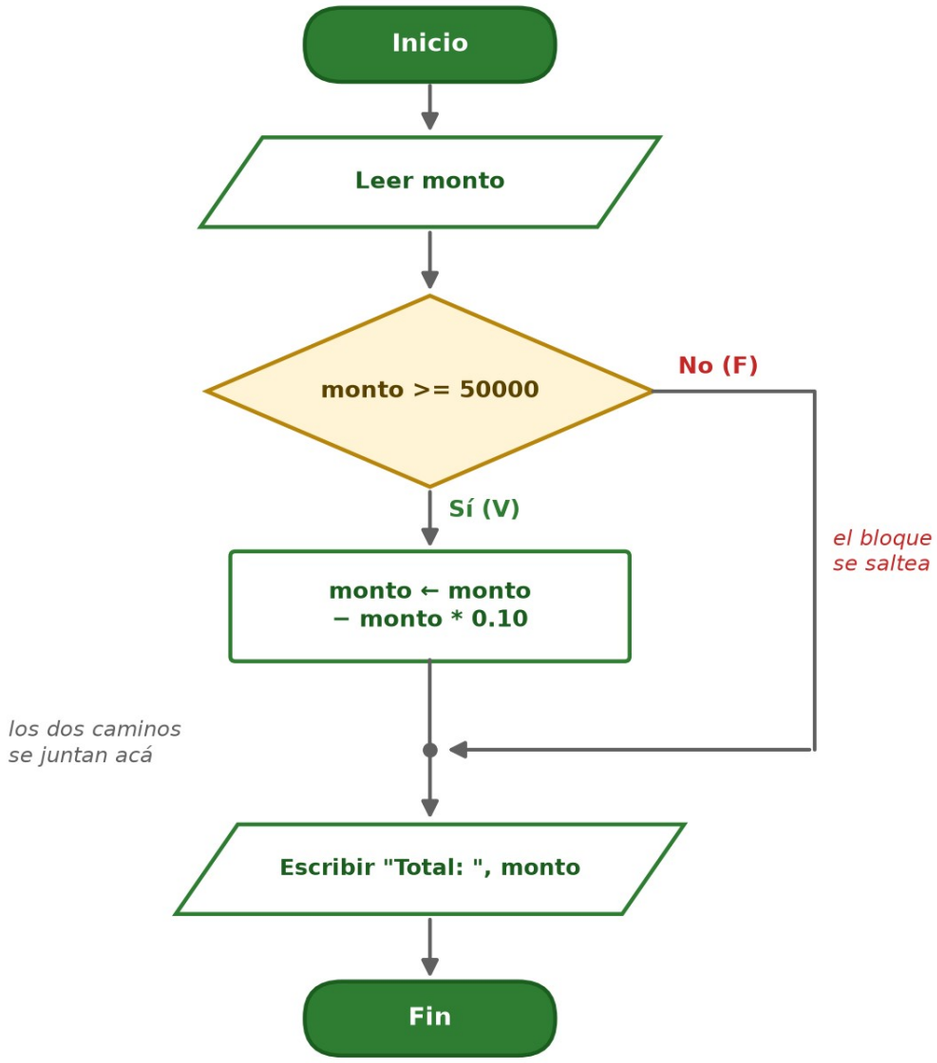


Figura 2.1 — Flujograma del Si simple: el descuento solo se aplica por el camino del Sí.

Operador	Se lee	Ejemplo	Resultado con monto=36000
>	mayor que	monto > 50000	Falso
>=	mayor o igual	monto >= 50000	Falso
<	menor que	monto < 50000	Verdadero
=	igual a	monto = 36000	Verdadero
<>	distinto de	monto <> 0	Verdadero

Leer el resultado según los datos

Un mismo algoritmo produce salidas distintas según lo que entra. Conviene probarlo con varios valores: uno que cumpla la condición y otro que no. A esa prueba mental se la llama traza o seguimiento.

> **Concepto clave**
> La estructura Si… Entonces ejecuta un bloque de acciones únicamente cuando la condición es verdadera. Si la condición es falsa, el bloque se omite.

Leer la condición como una pregunta

Toda condición es una pregunta que solo admite Verdadero o Falso. Antes de escribir el Si, conviene formularla en voz alta: «¿el monto llega a 50.000?». Si la pregunta tiene una respuesta clara con los datos disponibles, la condición está bien planteada; si dudás, falta un dato o sobra ambigüedad.

Pregunta del mostrador	Condición	Con monto = 60.000	Con monto = 24.000
¿Llega al descuento?	monto >= 50000	Verdadero	Falso
¿Es una venta chica?	monto < 50000	Falso	Verdadero
¿Es exactamente 50.000?	monto = 50000	Falso	Falso
¿Es distinta de cero?	monto <> 0	Verdadero	Verdadero

Una condición, varias acciones

El bloque del Si puede tener una sola instrucción o varias: todas las que estén entre Entonces y FinSi se ejecutan juntas cuando la condición es verdadera. Es un error común creer que el Si solo «cubre» la primera línea: en pseudocódigo, el bloque llega hasta el FinSi.

Ejemplo resuelto paso a paso — el descuento por monto

El copetín descuenta el 10 % cuando la venta llega a G. 50.000. Un cliente compra 12 empanadas a G. 5.000; otro compra 8 chipas a G. 3.000. Seguí la ejecución con cada uno.

Ejemplo — el descuento con bloque de varias acciones

```
Algoritmo DescuentoPorMonto
    Leer cantidad, precio
    monto <- cantidad * precio
    Si monto >= 50000 Entonces
        descuento <- monto * 0.10
        monto <- monto - descuento
        Escribir "Descuento aplicado: ", descuento
    FinSi
    Escribir "Total a pagar: ", monto
FinAlgoritmo
```

Cliente	Cálculo del monto	¿monto >= 50000?	Descuento	Total a pagar
12 empanadas	12 × 5.000 = 60.000	Verdadero	6.000	54.000
8 chipas	8 × 3.000 = 24.000	Falso	no se calcula	24.000

Fijate el detalle: para el cliente de las chipas, las tres instrucciones del bloque no se ejecutan — ni siquiera la que escribe el aviso. El algoritmo salta directo del Si al FinSi y muestra el total sin cambios.

Errores frecuentes con el Si simple

Los primeros programas con decisiones suelen fallar siempre por los mismos motivos. Reconocerlos de antemano ahorra horas de depuración:

Error	Síntoma	Corrección
Olvidar el FinSi	PSeInt marca error de sintaxis	todo Si abre y cierra su bloque
Usar = para asignar	el perfil Flexible lo acepta, pero en otros perfiles y lenguajes es un error	asigná con <- y compará con =
Texto sin comillas	categoria = salado no funciona	categoria = "salado"
Condición al revés	descuenta a las ventas chicas	prueba de escritorio con dos casos

En Paraguay — guaraníes sin céntimos

En Paraguay los montos se manejan en guaraníes enteros: en la práctica diaria no circulan céntimos. Por eso los precios y las cantidades de este libro son enteros.

Cuidado al aplicar un porcentaje: el 10 % de 55.555 es 5.555,5. Si la variable se definió Como Entero, PSeInt se detiene con un error de tipos al asignar 49.999,5. La solución es redondear antes de asignar: trunc(monto - monto * 0.10) da 49.999 y redon(...) da 50.000. Los ejemplos de este libro usan montos donde el 10 % es exacto.

Del pseudocódigo a PSeInt: probar el Si en la computadora

El descuento por monto se prueba en PSeInt en tres minutos: escribís el algoritmo tal como está en la caja, ejecutás con el botón de correr y cargás los datos cuando el programa los pida. La consola muestra el mismo

resultado que tu prueba de escritorio — y si no lo muestra, ganaste: encontraste un error antes de que llegue al mostrador.

Un hábito profesional desde la primera clase con decisiones: probá SIEMPRE los dos caminos. Una ejecución con 60.000 (entra al descuento) y otra con 24.000 (no entra). Un Si que solo se probó por un camino está probado a medias — el error favorito de los sistemas reales vive en el camino que nadie ensayó.

Del pseudocódigo al lenguaje de programación

PSeInt es un intérprete de pseudocódigo: ideal para aprender, pero no es un lenguaje de programación de los que se usan en la industria. El paso a un lenguaje real es más corto de lo que parece, porque las ideas son las mismas y cambia solo la escritura. PSeInt incluso traduce tu algoritmo: con Archivo → Exportar podés obtenerlo en Python, C, Java u otros lenguajes.

Ejemplo — el mismo algoritmo en PSeInt y en Python

En PSeInt (perfil Flexible):

```
Algoritmo VentaGrande
    // Calcula el importe de una venta de gaseosas
    Definir precio, cantidad, importe Como Entero
    precio <- 8000
    Escribir "Cantidad de gaseosas:"
    Leer cantidad
    importe <- precio * cantidad
    Si importe >= 50000 Entonces
        Escribir "Venta grande"
    FinSi
    Escribir "Total a pagar: ", importe
FinAlgoritmo
```

Exportado a Python 3 por PSeInt (con el comentario agregado a mano):

```
# Calcula el importe de una venta de gaseosas
precio = 8000
print("Cantidad de gaseosas:")
cantidad = int(input())
importe = precio * cantidad
if importe >= 50000:
    print("Venta grande")
print("Total a pagar: ", importe)
```

Con 7 gaseosas, los dos programas muestran «Venta grande» y «Total a pagar: 56000».

Elemento del lenguaje	En PSeInt	En Python	Qué tener en cuenta
Comentario	// texto	# texto	el programa lo ignora: es una nota para quien lee
Sangría	recomendada (legibilidad)	obligatoria: define los bloques	en Python, un espacio de más cambia el programa
Entrada	Leer cantidad	cantidad = int(input())	Python lee texto: hay que convertirlo a número
Salida	Escribir "Total: ", t	print("Total: ", t)	misma idea, otra palabra
Asignación y comparación	x <- 5 · Si x = 5	x = 5 · if x == 5	Python distingue = (asigna) de == (compara)
Fin de bloque	FinSi, FinPara…	no hay: lo marca la sangría	olvidar un FinSi es error en PSeInt

Todo lenguaje impone restricciones. Los nombres de variables empiezan con una letra, no llevan espacios y no pueden ser palabras reservadas (Si, Para, Escribir; en PSeInt también Mostrar, que es sinónimo de Escribir). Cada variable tiene un tipo y no se mezclan sin conversión: «5» entre comillas es un texto, no un número. Algunos lenguajes incluyen además sentencias para la pantalla, como la que cambia el color del texto (COLOR en QBasic, textcolor en Pascal); PSeInt no tiene esa instrucción, porque se concentra en la lógica del algoritmo.

Concepto clave — pseudocódigo, programa y lenguaje

El pseudocódigo expresa la lógica; el lenguaje de programación la escribe con reglas estrictas para que la computadora la ejecute. Comentarios, sangría, entrada, salida y estructuras de control existen en todos los lenguajes: cambia la sintaxis, no la idea.

Actividades de aplicación

Ejercitá:

1. Escribí la condición: "la venta es de al menos 50.000".
2. Evaluá monto >= 50000 para monto = 50.000 (¿V o F?).
3. Completá: Si stock ___ 5 Entonces Escribir "Reponer" (para avisar cuando queden menos de 5).
4. Indicá qué imprime el ejemplo si monto = 36.000.
5. Escribí un Si que muestre "Venta grande" cuando importe sea mayor que 100.000.

Resolvé (caso Copetín Karumbé):

6. Escribí un algoritmo que lea el monto de una venta y, si llega a 30.000, muestre "Llevás un coquito de regalo"; en todos los casos, que muestre el monto.
7. El copetín avisa "Reponer coquitos" cuando el stock baja de 20. Escribí el Si correspondiente y probalo con stock = 12.
8. Para una venta de 8 empanadas a 5.000, calculá el monto, indicá si cumple la condición del descuento y mostrá el total final.

Pensá y decidí (justificá tu respuesta):

9. V o F: si la condición de un Si simple es falsa, el algoritmo se detiene. Justificá.
10. Elegí: para "reponer cuando queden menos de 5" la condición correcta es (stock < 5 / stock <= 5). Justificá.
11. V o F: monto >= 50000 y monto > 49999 dan el mismo resultado para montos enteros. Justificá.

Desafío:

12. Escribí un algoritmo que lea el importe de una venta y muestre "Con descuento" si llega a 50.000. Probalo con ventas de 45.000 y de 75.000 y anotá qué imprime en cada caso.

Práctica 2 — Descuentos y avisos con Si simple

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Programar decisiones con Si simple y comprobar los dos caminos (condición verdadera y falsa), incluido el caso límite.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

El bloque de un Si simple se ejecuta solo cuando la condición es Verdadera; si es Falsa, se saltea y el algoritmo sigue.

Para probar una decisión hacen falta al menos tres datos: uno que cumpla, uno que no cumpla y el valor exacto del límite.

La Librería Arandu descuenta el 10 % en las compras de 100.000 o más, y avisa cuando quedan menos de 10 carpetas.

Actividad 1 — El descuento de la librería

Objetivo: *Programar el descuento del 10 % con un Si simple y probarlo con tres totales.*

Modelo — Esqueleto para completar

```
Algoritmo DescuentoLibreria
    Definir total Como Entero
    Escribir "Total de la compra:"
    Leer total
    Si … Entonces
        total <- …
        Escribir "Se aplicó el 10 % de descuento"
    FinSi
    Escribir "Total final: ", total
FinAlgoritmo
```

Pasos:

1. Antes de ejecutar, armá una tabla con tres totales: 120.000, 95.000 y 100.000, y anotá qué debería mostrar cada uno.
2. Completá la condición y el cálculo, y ejecutá con los tres totales.
3. Marcá en la tabla cuál de los tres es el caso límite y por qué importa probarlo.

Punto de control: Los tres resultados coinciden con tu tabla y el caso límite tomó el camino que anticipaste.

Actividad 2 — Dos avisos independientes

Objetivo: *Escribir dos Si simples seguidos para controlar el stock de carpetas.*

Modelo — Reglas del control de stock

Si el stock es menor que 10, mostrar «Reponer carpetas».

Si el stock es 0, mostrar además «No vender: sin stock».

Siempre, al final, mostrar «Control terminado».

Pasos:

1. Escribí el algoritmo AvisoStock con dos Si simples (no anidados).
2. Anticipá qué mensajes salen con stock 7, 0 y 25.
3. Ejecutá con los tres valores.

Punto de control: Para cada stock salen exactamente los mensajes anticipados, y «Control terminado» aparece en las tres ejecuciones.

Desafío final (para quienes terminan antes)

Ejecutá el descuento con un total de 100.005 y anotá qué pasa. Corregilo con trunc(...) en la asignación y explicá en una línea qué hace trunc.

Clase 3 — Dos caminos: la estructura Si… Entonces… Sino

Capacidad	Aplica estructuras selectivas anidadas en la resolución de problemas.
Tema	Dos caminos: la estructura Si… Entonces… Sino
Indicadores de logro	• Construye una estructura Si… Sino que elige entre dos bloques de acciones mutuamente excluyentes. • Determina cuál de los dos bloques se ejecuta según el valor de la condición. • Resuelve un problema del copetín que requiere elegir entre dos alternativas.

Cuando hay dos caminos posibles

El Si simple hace algo o no hace nada. Pero a veces hay que elegir entre dos acciones distintas: una si la condición es verdadera y otra si es falsa. Para eso está el Si… Entonces… Sino.

En la caja del copetín: si la venta llega al umbral se aplica descuento; si no, se cobra el precio pleno. Son dos caminos, y siempre se toma exactamente uno.

La sintaxis del Si… Sino

Después del bloque del Entonces se agrega la palabra Sino con su propio bloque. Se ejecuta uno u otro, nunca los dos, nunca ninguno.

La sangría vuelve a ser clave: cada bloque queda corrido bajo su palabra clave para que se lea sin ambigüedad.

Ejemplo — cobrar con o sin descuento

```
Si monto >= 50000 Entonces
    total <- monto - monto * 0.10
    Escribir "Aplicó descuento"
Sino
    total <- monto
    Escribir "Sin descuento"
FinSi
Escribir "Total: ", total
```

Con monto = 36.000 entra por el Sino: total = 36.000, imprime "Sin descuento".

El flujograma muestra los dos caminos del Si… Sino de la caja. Seguí con el dedo los dos datos del panel: uno entra por el Entonces y el otro por el Sino.

La Figura 3.1 lo muestra en forma gráfica: flujograma del Si… Entonces… Sino del descuento: con monto 60000 se paga 54000; con monto 48000 no hay descuento.

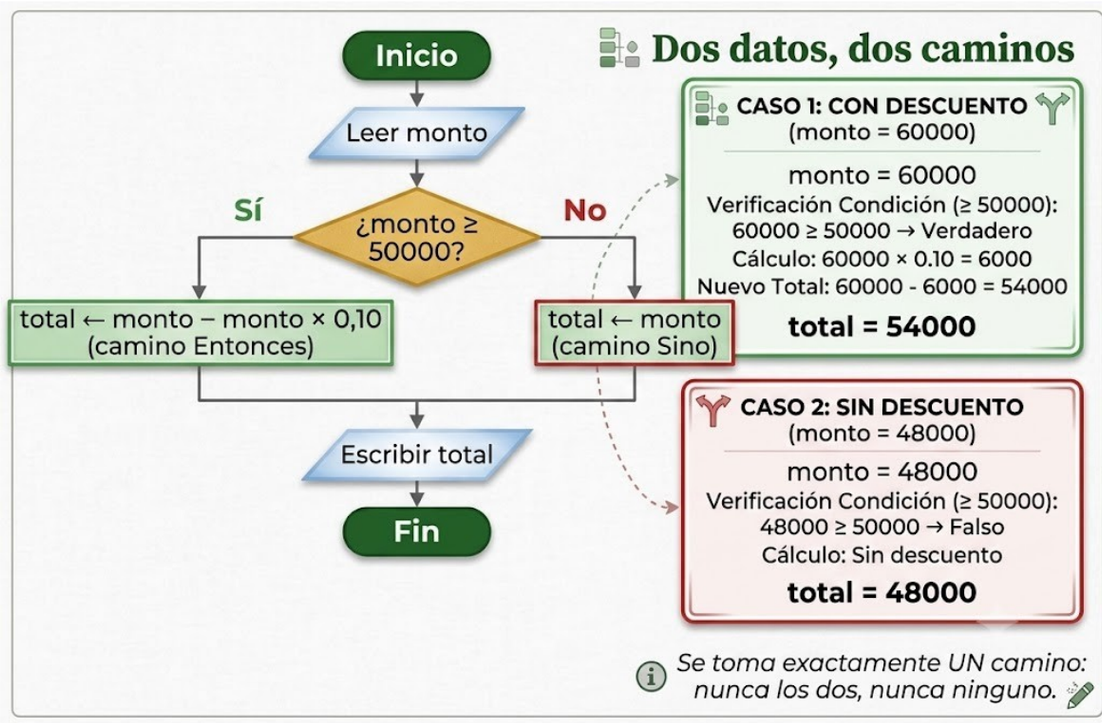


Figura 3.1 — Flujograma del Si… Entonces… Sino del descuento: con monto 60000 se paga 54000; con monto 48000 no hay descuento.

Condiciones mutuamente excluyentes

El Si… Sino sirve cuando las dos alternativas cubren todos los casos y no se superponen: aprobado o reprobado, mayor de edad o menor, con descuento o sin descuento. Si hubiera más de dos caminos, se necesitan estructuras anidadas (clase 5).

monto	Condición monto>=50000	Camino	total
36000	Falso	Sino	36000
50000	Verdadero	Entonces	45000
60000	Verdadero	Entonces	54000
48000	Falso	Sino	48000

Trazar los dos caminos

Para verificar un Si… Sino conviene probar un dato que entre por el Entonces y otro que entre por el Sino. Así se comprueba que ambos caminos funcionan.

Concepto clave

La estructura Si… Sino elige entre dos bloques mutuamente excluyentes: se ejecuta el del Entonces cuando la condición es verdadera y el del Sino cuando es falsa.

Ejemplo cotidiano — el cajero automático

Un cajero automático de un banco paraguayo usa esta misma lógica: si el saldo alcanza, entrega el dinero; si no, muestra "Saldo insuficiente". Dos caminos, una sola decisión.

Dos caminos que siempre se resuelven

La diferencia práctica entre el Si simple y el Si… Sino es que el segundo nunca deja al cliente sin respuesta: pase lo que pase, se ejecuta uno de los dos bloques. Por eso el Si… Sino es la estructura natural para tarifas, cargos y clasificaciones donde «no hacer nada» no es una opción.

Ejemplo resuelto paso a paso — el delivery del copetín

El copetín cobra G. 10.000 de envío, pero lo regala cuando el pedido llega a G. 80.000. Un cliente pide 6 sándwiches mixtos; otro pide 10. Ambos casos entran por el mismo algoritmo y salen por caminos distintos.

Ejemplo — el cargo de envío con Si… Sino

```
Algoritmo Delivery
    Leer monto
    Si monto >= 80000 Entonces
        Escribir "Envío gratis"
    Sino
        monto <- monto + 10000
        Escribir "Se suma envío de 10.000"
    FinSi
    Escribir "Total final: ", monto
FinAlgoritmo
```

Pedido	Monto	¿monto >= 80000?	Camino	Total final
6 mixtos × 12.000	72.000	Falso	Sino: suma 10.000	82.000
10 mixtos × 12.000	120.000	Verdadero	Entonces: gratis	120.000

Con 72.000 la condición es falsa y el pedido paga envío: 72.000 + 10.000 = 82.000. Con 120.000 la condición es verdadera y el total no cambia. Un solo algoritmo, dos comportamientos — sin escribir dos programas.

El error clásico: ponerle condición al Sino

El Sino no lleva condición: es «todo lo demás». Escribir «Sino monto < 80000» es redundante y, en la mayoría de los lenguajes, un error de sintaxis. Si necesitás una segunda condición distinta de la negación de la primera, lo que corresponde es otra estructura (un Si anidado o un Segun, que vas a ver en la Clase 5).

Concepto clave

El bloque del Sino se ejecuta exactamente cuando la condición del Si es falsa: no necesita (ni admite) condición propia. Si te descubrís queriendo escribirle una, el problema pide otra estructura.

Otro Si… Sino del mostrador: el vuelto

El cobro en efectivo siempre termina en la misma decisión: si el cliente entregó más de lo que debe, hay vuelto; si entregó justo, no. Dos caminos que cubren todos los casos válidos — otro trabajo natural para el Si… Sino.

Ejemplo — calcular el vuelto

```
Algoritmo Vuelto
    // Precondición de esta clase: entregado >= total
    Leer total, entregado
    Si entregado > total Entonces
        vuelto <- entregado - total
        Escribir "Vuelto: ", vuelto
    Sino
        Escribir "Pago exacto, gracias"
    FinSi
FinAlgoritmo
```

Prueba rápida con el pedido del delivery: total G. 82.000, el cliente entrega G. 100.000. La condición 100.000 > 82.000 es verdadera y el vuelto sale de la resta: G. 18.000. Con un billete justo de 82.000, el camino del Sino agradece el pago exacto. ¿Y si entregara menos? Ese caso pide una tercera rama — un anidamiento como los de la Clase 5.

Actividades de aplicación

Ejercitá:

1. Escribí un Si… Sino que muestre "Mayoría" si edad >= 18 y "Minoría" en caso contrario.
2. Indicá qué camino toma el primer ejemplo de la clase con monto = 49.999 y cuál es el total.
3. Completá el Sino que falta: Si nota >= 60 Entonces Escribir "Aprobado" Sino Escribir ___.
4. Calculá total para monto = 80.000 con la lógica del ejemplo.
5. ¿Cuántos bloques se ejecutan en un Si… Sino? ¿Cuántos como máximo?

Resolvé (caso Copetín Karumbé):

6. Escribí un algoritmo que lea el monto de una venta y muestre el total con descuento si llega a 50.000, o el total pleno si no.
7. El copetín marca "Pedido grande" cuando se venden 10 o más unidades y "Pedido chico" en caso contrario. Escribí el Si… Sino y probalo con 7 unidades.
8. Para una venta de 9 sopas paraguayas a 10.000, indicá el monto, el camino que toma y el total final.

Pensá y decidí (justificá tu respuesta):

9. V o F: en un Si… Sino siempre se ejecuta uno de los dos bloques. Justificá.
10. Elegí: si la condición es Verdadera se ejecuta el bloque del (Entonces / Sino).
11. V o F: el Si… Sino sirve para elegir entre tres o más caminos. Justificá.

Desafío:

12. Escribí un algoritmo de caja que lea el monto y, con un Si… Sino, muestre el total final y el mensaje correspondiente. Probalo con 30.000, 49.999, 50.000 y 90.000 y armá una tabla con monto, camino y total.

Práctica 3 — Envíos y pagos con Si… Sino

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Programar una decisión de dos caminos con Si… Sino y diseñar los casos de prueba de cada camino.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

El Si… Sino ejecuta siempre exactamente uno de sus dos bloques.

Un buen caso de prueba se elige antes de ejecutar y se anota con el resultado esperado.

La librería envía sin cargo los pedidos de 150.000 o más; a los demás les suma 15.000 de envío.

Actividad 1 — El cargo de envío

Objetivo: *Programar el cargo de envío con un Si… Sino y probar los dos caminos.*

Modelo — Esqueleto para completar

```
Algoritmo EnvioLibreria
    Definir total Como Entero
    Leer total
    Si … Entonces
        Escribir "Envío sin cargo"
    Sino
        total <- …
        Escribir "Se suma el envío: 15000"
    FinSi
    Escribir "Total final: ", total
FinAlgoritmo
```

Pasos:

1. Elegí tres totales de prueba: uno por debajo, uno por encima y el valor exacto del límite. Anotá el resultado esperado de cada uno.
2. Completá el esqueleto y ejecutá con tus tres totales.
3. Probá también con 98.000.

Punto de control: Los cuatro totales finales coinciden con lo que anotaste antes de ejecutar.

Actividad 2 — ¿Alcanza el pago?

Objetivo: *Programar un control de pago que muestre el vuelto o cuánto falta.*

Modelo — Comportamiento esperado

Si el pago alcanza: mostrar «Vuelto: » y el vuelto.

Si no alcanza: mostrar «Faltan: » y la diferencia.

Nunca debe mostrarse un vuelto negativo.

Pasos:

1. Escribí el algoritmo ControlPago con un Si… Sino.
2. Probalo con un total de 64.000 y pagos de 70.000 y de 50.000.
3. Agregá un tercer caso de pago exacto y decidí qué camino debería tomar.

Punto de control: En ninguna de las tres ejecuciones aparece un número negativo, y el pago exacto tomó el camino que habías decidido.

Desafío final (para quienes terminan antes)

Combiná las dos actividades en un solo algoritmo: primero calcula el total con o sin envío y después controla el pago. Probalo con un pedido de 98.000 y un pago de 120.000.

Clase 4 — Condiciones combinadas: los operadores lógicos Y, O, NO

Capacidad	Aplica estructuras selectivas anidadas en la resolución de problemas.
Tema	Condiciones combinadas: los operadores lógicos Y, O, NO
Indicadores de logro	• Combina dos o más condiciones con los operadores Y, O y NO. • Evalúa una condición compuesta aplicando la tabla de verdad correspondiente. • Elige el operador lógico adecuado para una regla del copetín.

Cuando una sola pregunta no alcanza

A veces una decisión depende de dos condiciones a la vez. El copetín hace envío gratis solo si la venta llega a 50.000 Y el cliente está en el barrio. Ahí no alcanza un operador relacional: hacen falta operadores lógicos que combinen condiciones.

Los tres operadores lógicos

Y (conjunción) es Verdadero solo si ambas condiciones son verdaderas. O (disyunción) es Verdadero si al menos una lo es. NO (negación) invierte el valor: convierte Verdadero en Falso y viceversa.

Ejemplo — envío gratis con dos condiciones

```
Si (monto >= 50000) Y (enBarrio = Verdadero) Entonces
    Escribir "Envío gratis"
Sino
    Escribir "Envío con costo"
FinSi
```

Con monto = 60.000 y enBarrio = Falso: la primera es V, la segunda F; V Y F = F, así que imprime "Envío con costo".

Las tablas de verdad

Cada operador se resume en una tabla de verdad, que lista el resultado para todas las combinaciones posibles de las condiciones. Es la herramienta para no equivocarse al evaluar una condición compuesta.

La figura evalúa la condición del envío gratis como lo hace la computadora: primero cada comparación por separado y después el operador que las une.

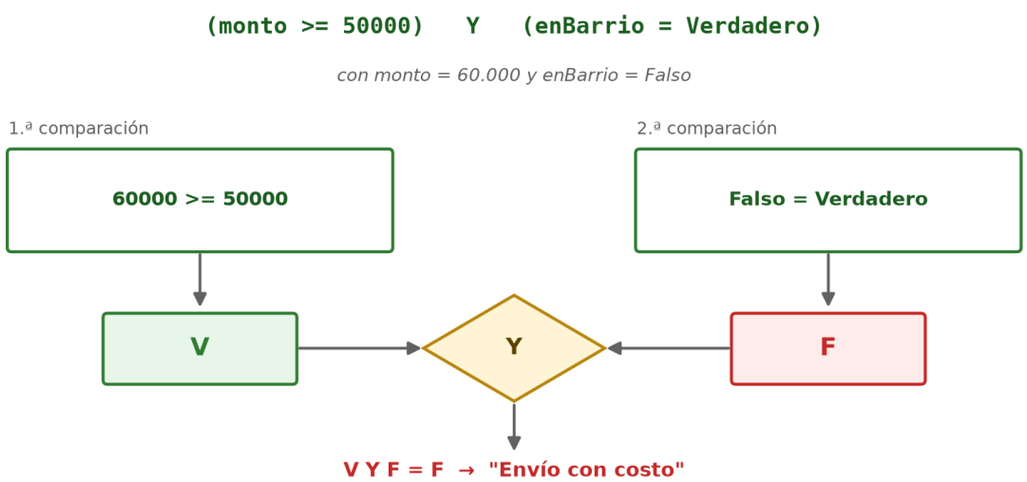


Figura 4.1 — Cómo se evalúa una condición compuesta: cada comparación da V o F y el operador Y decide.

A	B	A Y B	A O B	NO A
V	V	V	V	F
V	F	F	V	F
F	V	F	V	V
F	F	F	F	V

Elegir el operador correcto

La palabra clave del enunciado guía la elección. "Ambas cosas" o "y además" piden Y. "Una u otra" o "al menos una" piden O. "No", "que no sea" piden NO. Confundirlos cambia por completo la regla.

Concepto clave

Y es verdadero solo si ambas condiciones lo son; O es verdadero si al menos una lo es; NO invierte el valor de la condición.

La promo combo: una condición compuesta real

El copetín lanza una promo: G. 5.000 de descuento para quien lleve al menos 3 empanadas y al menos 1 gaseosa. Son dos exigencias que deben cumplirse a la vez — trabajo para el operador Y.

Ejemplo — el combo con el operador Y

```
Algoritmo PromoCombo
    Leer empanadas, gaseosas
    monto <- empanadas * 5000 + gaseosas * 8000
    Si empanadas >= 3 Y gaseosas >= 1 Entonces
        monto <- monto - 5000
        Escribir "Combo aplicado"
    FinSi
    Escribir "Total: ", monto
FinAlgoritmo
```

Ejemplo resuelto paso a paso — dos pedidos frente al combo

Evaluá la condición compuesta parte por parte, como hace la computadora: primero cada comparación por separado, después el operador que las une.

Pedido	Monto	empanadas >= 3	gaseosas >= 1	Y	Total
4 empanadas y 2 gaseosas	36.000	V	V	V → −5.000	31.000
2 empanadas y 2 gaseosas	26.000	F	V	F → sin combo	26.000

En el segundo pedido alcanza con que una de las dos partes sea falsa para que todo el Y sea falso: el cliente paga los 26.000 completos. Si la promo dijera «empanadas O gaseosas», en cambio, ese mismo pedido sí la ganaría.

La negación y la ley del contrario

El operador NO invierte el resultado: NO(stock < 10) es verdadero exactamente cuando stock >= 10. Fijate que el contrario de < es >= (no >): el caso del borde — stock igual a 10 — tiene que quedar de un solo lado. Confundir el borde es uno de los errores lógicos más frecuentes en los sistemas de gestión.

Condición original	Su negación correcta	Error típico
stock < 10	stock >= 10	stock > 10 (pierde el 10)
monto >= 50000	monto < 50000	monto <= 50000 (repite el 50.000)
categoria = "salado"	categoria <> "salado"	olvidar las comillas del texto

Concepto clave

Cuando se combinan operadores, el NO se evalúa primero, después el Y, y por último el O. Ante la duda, usá paréntesis: (a Y b) O c no es lo mismo que a Y (b O c).

El O en el mostrador: pago aceptado

El copetín acepta efectivo o pago con QR: alcanza con que una de las dos formas esté disponible para cerrar la venta. Es la situación típica del O — exigente con el rechazo (solo rechaza si TODO falla), generoso con la aceptación.

tieneEfectivo	tieneQR	tieneEfectivo O tieneQR	¿Se concreta la venta?
V	V	V	sí (elige el cliente)
V	F	V	sí, en efectivo
F	V	V	sí, con QR
F	F	F	no: venta perdida

Ejemplo cotidiano — efectivo o QR

El pago con código QR desde la billetera del celular convive hoy con el efectivo en muchos comercios. Por eso los sistemas de venta modelan el medio de pago como una condición compuesta: la venta se concreta si hay efectivo O QR.

Tres condiciones encadenadas

Los operadores lógicos no se limitan a dos condiciones: la promo puede exigir tres cosas a la vez. Supongamos que el combo solo vale si además el monto llega a G. 30.000: empanadas >= 3 Y gaseosas >= 1 Y monto >= 30000. El pedido de 4 empanadas y 2 gaseosas (36.000) evalúa V Y V Y V — verdadero — y gana el descuento.

Con tres o más condiciones, los paréntesis dejan de ser opcionales para la lectura humana: (empanadas >= 3 Y gaseosas >= 1) Y monto >= 30000 dice claramente que primero se verifica el combo y después el piso de monto. La computadora quizás no los necesite; el compañero que lea tu algoritmo el año que viene, sí.

Actividades de aplicación

Ejercitá:

1. Evaluá: (Verdadero Y Falso).
2. Evaluá: (Falso O Verdadero).
3. Evaluá: NO (Verdadero).
4. Evaluá para monto = 75000 y enBarrio = Verdadero: (monto >= 50000) Y (enBarrio = Verdadero).
5. Escribí la condición: "la venta llega a 50.000 O el cliente es socio".

Resolvé (caso Copetín Karumbé):

6. El copetín cambia la regla: hay envío gratis si el monto llega a 80.000 O si el cliente es socio. Escribí el algoritmo con un Si… Sino y probalo con monto = 60.000 y socio = Verdadero.
7. El copetín aplica precio especial si el producto es "salado" O "bebida". Escribí la condición compuesta y evaluala para categoria = "dulce".
8. Un combo se arma si hay chipa Y cocido en stock. Con stockChipa=10 y stockCocido=0, indicá si el combo se puede armar y por qué.

Pensá y decidí (justificá tu respuesta):

9. V o F: (V O F) es Verdadero. Justificá con la tabla.
10. Elegí el operador para "aprobás si tenés más de 60 en teoría Y más de 60 en práctica": (Y / O).
11. V o F: NO invierte el valor de una condición. Justificá.

Desafío:

12. Construí la tabla de verdad completa de la condición (monto >= 80000) O (socio = Verdadero) para las cuatro combinaciones posibles, e indicá en cuántos casos hay envío gratis. Compará con la regla del ejemplo de la clase, que usaba Y.

Práctica 4 — Condiciones compuestas: descuento estudiantil y medios de pago

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Escribir condiciones compuestas con Y, O y NO, usar variables lógicas y verificar el resultado con la tabla de verdad.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

Y es verdadera solo si las dos condiciones lo son; O, si al menos una lo es; NO invierte el valor.

Una variable Logico guarda Verdadero o Falso. En PSeInt se puede leer escribiendo Verdadero o Falso.

La librería hace un 10 % de descuento a quien presenta carnet de estudiante y compra 50.000 o más. Acepta efectivo, QR y tarjeta.

Actividad 1 — El descuento estudiantil

Objetivo: *Programar una condición con Y sobre una variable lógica y un monto.*

Modelo — Esqueleto para completar

```
Algoritmo DescuentoEstudiante
    Definir total Como Entero
    Definir esEstudiante Como Logico
    Leer total
    Leer esEstudiante
    Si (…) Y (…) Entonces
        total <- total - total * 0.10
        Escribir "Descuento estudiantil aplicado"
    Sino
        Escribir "Sin descuento"
    FinSi
    Escribir "Total final: ", total
FinAlgoritmo
```

Pasos:

1. Armá la tabla de verdad de la condición con sus cuatro combinaciones y marcá en cuál hay descuento.
2. Completá el esqueleto y ejecutá tres casos: 65.000 con Verdadero, 65.000 con Falso y 40.000 con Verdadero.
3. Ubicá cada ejecución en una fila de tu tabla.

Punto de control: Cada ejecución cae en la fila de la tabla que anticipaste, y solo la combinación V-V aplica el descuento.

Actividad 2 — Medio de pago y stock

Objetivo: *Combinar O y NO en una decisión anidada.*

Modelo — Reglas

Si NO hay stock (NO (stock > 0)): «Venta rechazada: sin stock».

Si hay stock y el medio es efectivo O QR O tarjeta: «Venta aceptada».

Si hay stock y el medio es otro: «Medio de pago no aceptado».

Pasos:

1. Definí medio como Caracter y stock como Entero.
2. Escribí el algoritmo MedioDePago con un Si… Sino que tiene otro Si… Sino adentro.
3. Probalo con: QR y stock 5; cheque y stock 5; efectivo y stock 0.

Punto de control: Los tres casos producen tres mensajes distintos, y el caso sin stock no llega a preguntar por el medio de pago.

Desafío final (para quienes terminan antes)

Cambiá el Y del descuento estudiantil por O y volvé a ejecutar los tres casos de la Actividad 1. ¿Qué casos cambian? Respondé con la tabla de verdad.

Clase 5 — Decisiones dentro de decisiones: anidamiento y selección múltiple (Segun)

Capacidad	Aplica estructuras selectivas anidadas en la resolución de problemas.
Tema	Decisiones dentro de decisiones: anidamiento y selección múltiple (Segun)
Indicadores de logro	• Construye estructuras condicionales anidadas para elegir entre más de dos caminos. • Reemplaza una cadena de Si anidados por una estructura Segun cuando corresponde. • Sigue la traza de una estructura anidada hasta el camino correcto.

Más de dos caminos

El Si… Sino elige entre dos. Pero clasificar una venta en "chica", "mediana" o "grande" necesita tres caminos. La solución es anidar: poner un Si dentro del Sino de otro Si.

Estructuras anidadas

En un anidamiento, cada Sino abre una nueva pregunta. El algoritmo evalúa las condiciones en orden y toma el primer camino cuya condición sea verdadera; los demás se descartan. La sangría creciente muestra la profundidad del anidamiento.

Ejemplo — clasificar la venta en tres categorías

```
Si monto < 20000 Entonces
    Escribir "Venta chica"
Sino
    Si monto < 50000 Entonces
        Escribir "Venta mediana"
    Sino
        Escribir "Venta grande"
    FinSi
FinSi
```

Con monto = 36.000: la primera es F, la segunda (36.000<50.000) es V → "Venta mediana".

La estructura Segun

Cuando la decisión depende del valor exacto de una variable (por ejemplo, un número de opción del menú), conviene la estructura Segun. Lista los casos posibles y ejecuta el que coincide, con una opción De Otro Modo para lo no previsto. Es más legible que muchos Si anidados.

Compará en la figura las dos formas de dibujar la misma decisión: a la izquierda, la cadena de Si anidados que pregunta una vez por cada valor; a la derecha, el Segun, que reparte en un solo paso según el valor de la opción.

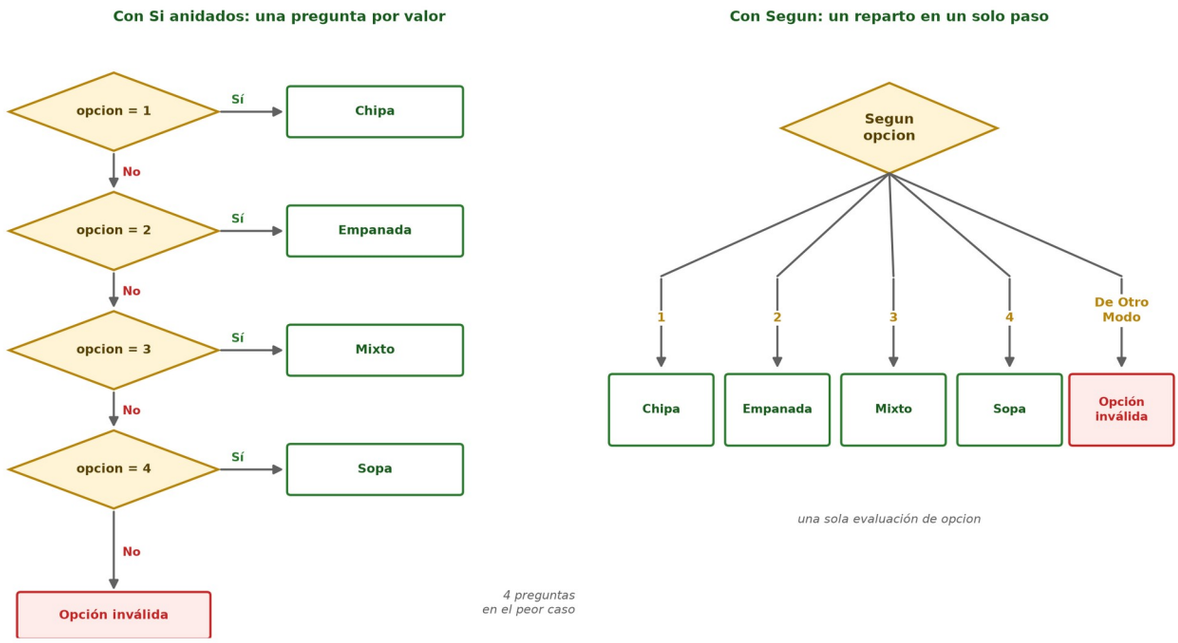


Figura 5.1 — El menú del mostrador con Si anidados y con Segun: la misma decisión, dos dibujos.

monto	1.ª cond. <20000	2.ª cond. <50000	Resultado
15000	V	—	Venta chica
36000	F	V	Venta mediana
60000	F	F	Venta grande
20000	F	V	Venta mediana

Elegir entre anidar y Segun

Se usa Segun cuando se compara una variable con valores fijos (1, 2, 3…). Se usan Si anidados cuando las condiciones son rangos (monto < 20000, monto < 50000…) o combinan varias variables.

Concepto clave

El anidamiento coloca una estructura condicional dentro de otra para elegir entre más de dos caminos. La estructura Segun compara una variable con valores fijos de forma más legible.

El menú del mostrador con Segun

El mostrador del copetín trabaja con un menú numerado: el cajero teclea la opción y el sistema responde el precio. Con cuatro opciones y un caso de error, el Segun resuelve en pocas líneas lo que exigiría tres Si anidados.

Ejemplo — el menú numerado del copetín

```
Algoritmo MenuMostrador
    Leer opcion
    Segun opcion Hacer
        1: Escribir "Chipa - 3.000"
        2: Escribir "Empanada - 5.000"
        3: Escribir "Sándwich mixto - 12.000"
        4: Escribir "Sopa paraguaya - 10.000"
        De Otro Modo:
            Escribir "Opción inválida"
    FinSegun
FinAlgoritmo
```

Ejemplo resuelto paso a paso — clasificar la venta con Si anidado

El dueño clasifica cada venta en chica (menos de 50.000), mediana (de 50.000 a menos de 100.000) o grande (100.000 o más). Como los rangos son numéricos y encadenados, acá conviene el Si anidado: el Segun compara contra valores exactos, no contra rangos.

Ejemplo — clasificar por rangos con Si anidado

```
Si monto < 50000 Entonces
    Escribir "Venta chica"
Sino
    Si monto < 100000 Entonces
        Escribir "Venta mediana"
    Sino
        Escribir "Venta grande"
    FinSi
FinSi
```

Venta	¿monto < 50000?	¿monto < 100000?	Clasificación
2 mixtos = 24.000	Verdadero	no se evalúa	chica
12 empanadas = 60.000	Falso	Verdadero	mediana
10 mixtos = 120.000	Falso	Falso	grande

Observá la economía del anidamiento: la segunda pregunta solo se hace cuando la primera falló. La venta de 24.000 se clasifica con una sola comparación; la de 120.000 necesita las dos.

Segun o Si anidado: cuál conviene

Situación	Estructura recomendada	Por qué
Opciones exactas (menú 1, 2, 3, 4)	Segun	una rama por valor, más legible
Rangos numéricos (chica/mediana/grande)	Si anidado	el Segun no compara rangos
Dos caminos nada más	Si… Sino	no hace falta más estructura
Condiciones sobre variables distintas	Si anidados	cada nivel pregunta otra cosa

Anidar dentro del Segun: menú más descuento

Las estructuras se combinan: dentro de una rama del Segun puede vivir un Si completo. El mostrador lo necesita apenas el menú se cruza con la promo del descuento por monto:

Ejemplo — una rama del menú con decisión adentro

```
Segun opcion Hacer
    2: // Empanada
        Leer cantidad
        monto <- cantidad * 5000
        Si monto >= 50000 Entonces
            monto <- monto - monto * 0.10
        FinSi
        Escribir "A pagar: ", monto
    // ... demás opciones ...
FinSegun
```

Traza con opción 2 y cantidad 12: monto arranca en 60.000, la condición del Si interior es verdadera y el total baja a G. 54.000 — el mismo resultado de la Clase 2, ahora dentro de un menú. Anidar no cambia las reglas de cada estructura: cada una abre y cierra su propio bloque, y la sangría muestra quién vive dentro de quién.

De Otro Modo: la red de seguridad

La rama De Otro Modo del Segun atrapa todo valor que no coincida con ninguna opción: el 7 tecleado por error, el 0, el número negativo. Sin ella, el Segun simplemente no hace nada ante un valor inesperado — y un sistema de mostrador que responde con silencio es un sistema que desconcierta al cajero.

La regla del libro: todo Segun de un sistema real lleva su De Otro Modo, aunque sea solo para avisar «Opción inválida». Es la misma filosofía del Si… Sino frente al Si simple: que ningún caso quede sin respuesta. Los programadores lo llaman programación defensiva — anticipar el error del usuario en lugar de suponer que nunca va a ocurrir.

Actividades de aplicación

Ejercitá:

1. Con el primer ejemplo de la clase (umbrales 20.000 y 50.000), indicá el resultado para monto = 19.999.
2. Con el mismo ejemplo, indicá el resultado para monto = 50.000.
3. Completá: para clasificar la opción 1, 2 o 3 del menú conviene la estructura ___.
4. Escribí el caso De Otro Modo de un Segun que reciba opciones 1, 2 y 3.
5. ¿Cuántos mensajes imprime el ejemplo para un mismo monto?

Resolvé (caso Copetín Karumbé):

6. Escribí con Si anidados un algoritmo que clasifique el monto de una venta en "chica" (<20.000), "mediana" (<50.000) o "grande" (>=50.000).
7. Escribí con Segun un menú que reciba 1=Chipa, 2=Empanada, 3=Mixto y muestre el precio (3.000 / 5.000 / 12.000), con De Otro Modo "Opción inválida".
8. Para monto = 49.999, seguí la traza del primer ejemplo paso a paso e indicá el resultado.

Pensá y decidí (justificá tu respuesta):

9. V o F: en un anidamiento se pueden ejecutar dos caminos a la vez. Justificá.
10. Elegí: para comparar una variable con los valores 1, 2 y 3 conviene (Segun / Si anidado múltiple).
11. V o F: el orden de las condiciones en un anidamiento no afecta el resultado. Justificá.

Desafío:

12. Escribí un algoritmo con Segun para el número de mesa (1 a 4) que muestre a qué mozo le toca atender. Agregá De Otro Modo para mesas inexistentes y probalo con la mesa 3 y la mesa 9.

Práctica 5 — Clasificar pedidos y armar el menú de la librería

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Elegir entre Si anidado y Segun según el tipo de decisión, y programar ambos con sus casos límite.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

Para rangos (menos de, entre, más de) se usa Si anidado; para valores exactos (1, 2, 3…) conviene Segun.

En un Si anidado se toma el primer camino cuya condición es verdadera; el orden de las preguntas importa.

La librería clasifica los pedidos de cuadernos: menos de 12, minorista; de 12 a 49, por docena; 50 o más, mayorista.

Actividad 1 — Tipo de pedido con Si anidado

Objetivo: *Programar la clasificación en tres categorías y probar los dos límites.*

Modelo — Esqueleto para completar

```
Algoritmo TipoPedido
    Definir cantidad Como Entero
    Leer cantidad
    Si cantidad < … Entonces
        Escribir "Pedido minorista"
    Sino
        Si … Entonces
            Escribir "Pedido por docena"
        Sino
            Escribir "Pedido mayorista"
        FinSi
    FinSi
FinAlgoritmo
```

Pasos:

1. Antes de ejecutar, armá una tabla con las cantidades 11, 12, 49 y 50 y la categoría que les corresponde según la regla.
2. Completá las dos condiciones y ejecutá con las cuatro cantidades.
3. Explicá en una línea por qué la segunda condición no necesita preguntar «cantidad >= 12».

Punto de control: Las cuatro cantidades límite caen en la categoría de tu tabla.

Actividad 2 — El menú con Segun

Objetivo: *Programar un menú de cinco artículos con Segun y De Otro Modo.*

Modelo — Menú de la caja

1 Cuaderno 8.000 · 2 Bolígrafo 3.000 · 3 Lápiz 2.000 · 4 Regla 4.000 · 5 Carpeta 15.000

Otra opción: «Opción inválida».

Escribí cada opción del Segun en su propia línea: 1: y abajo, con sangría, la asignación del precio.

Pasos:

1. Escribí el algoritmo MenuLibreria: guardá el precio en una variable dentro de cada opción y mostralo después del FinSegun solo si se eligió una opción válida.
2. Probalo con las opciones 4 y 7.
3. Probá también la opción 0.

Punto de control: Las opciones válidas muestran el precio del menú y las inválidas muestran un solo mensaje, sin un precio debajo.

Desafío final (para quienes terminan antes)

Reescribí el menú con Si anidados y contá cuántas líneas tiene cada versión. ¿Cuál es más fácil de leer y por qué?

Clase 6 — Repetir con condición: Mientras, Repetir… Hasta Que y el contador

Capacidad	Resuelve problemas utilizando ciclos repetitivos, contadores, acumuladores y banderas.
Tema	Repetir con condición: Mientras, Repetir… Hasta Que y el contador
Indicadores de logro	• Distingue el ciclo Mientras (0 o más repeticiones) del Repetir… Hasta Que (1 o más). • Implementa un contador que registra cuántas veces ocurre algo dentro de un ciclo. • Evita el ciclo infinito garantizando que la condición cambie dentro del ciclo.

Cuando hay que repetir

Cobrar a un cliente es una secuencia. Cobrar a todos los clientes del día es repetir esa secuencia muchas veces. Las estructuras repetitivas (o ciclos) hacen justamente eso: ejecutan un bloque varias veces sin escribirlo una y otra vez.

El ciclo Mientras

El Mientras evalúa la condición antes de cada repetición. Si es verdadera, ejecuta el bloque y vuelve a preguntar; cuando es falsa, sale. Como pregunta primero, puede ejecutarse cero veces si la condición arranca falsa.

Ejemplo — contar cuántos días superaron el promedio

```
promedio <- 600000
contador <- 0
i <- 1
Mientras i <= 7 Hacer
    Leer ventaDia
    Si ventaDia >= promedio Entonces
        contador <- contador + 1
    FinSi
    i <- i + 1
FinMientras
Escribir "Días que alcanzaron el promedio: ", contador
```

Con las ventas de la semana (420k, 500k, 450k, 580k, 850k, 1.100k, 300k) el contador termina en 2 (viernes y sábado).

El ciclo Repetir… Hasta Que

El Repetir ejecuta el bloque primero y evalúa la condición al final. Por eso se ejecuta al menos una vez. Sale cuando la condición de corte se vuelve verdadera. Sirve cuando la acción debe ocurrir por lo menos una vez, como pedir un dato hasta que sea válido.

Aspecto	Mientras	Repetir… Hasta Que
Evalúa la condición	al inicio	al final
Repeticiones mínimas	0	1
Sale cuando	la condición es falsa	la condición es verdadera
Uso típico	recorrer mientras haya datos	pedir un dato válido

El contador y el ciclo infinito

Un contador es una variable que aumenta de a uno para llevar la cuenta de algo (clientes atendidos, productos agotados). Se inicia en cero y crece con contador <- contador + 1. Cuidado: si dentro del ciclo nada modifica la condición, el ciclo nunca termina; es un ciclo infinito, y hay que evitarlo.

Concepto clave

El Mientras evalúa la condición antes de repetir (puede no ejecutarse nunca); el Repetir… Hasta Que la evalúa al final (se ejecuta al menos una vez). Un contador lleva la cuenta de las veces que ocurre algo.

La figura reúne el flujograma del Mientras con su traza de ejecución: mirá cómo el contador solo cambia en las vueltas en que la condición interna es verdadera.

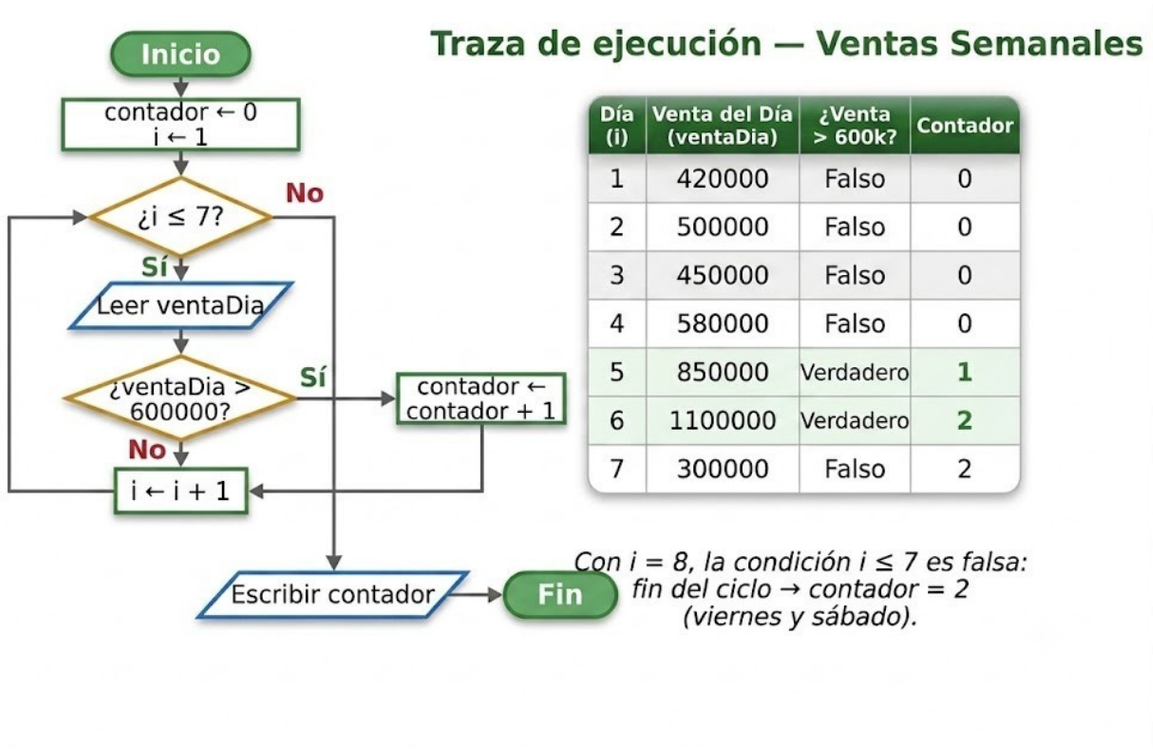


Figura 6.1 — Flujograma del ciclo Mientras y traza de ejecución iteración por iteración: el contador termina en 2 (viernes y sábado superan el promedio de 600000).

La meta del día: repetir hasta alcanzarla

El dueño fija una meta de caja de G. 100.000 y quiere saber cuántas ventas hacen falta para alcanzarla. Nadie sabe de antemano cuántas serán: puede bastar una venta grande o hacer falta veinte chicas. Cantidad desconocida de repeticiones — territorio del Mientras.

Ejemplo — acumular hasta la meta

```
Algoritmo MetaDelDia
    acumulado <- 0
    cantidad <- 0
    Mientras acumulado < 100000 Hacer
        Leer venta
        acumulado <- acumulado + venta
        cantidad <- cantidad + 1
    FinMientras
    Escribir "Meta alcanzada con ", cantidad, " ventas"
FinAlgoritmo
```

Ejemplo resuelto paso a paso — tres ventas para llegar

Una mañana entran ventas de G. 30.000, G. 24.000 y G. 60.000. Trazá el ciclo vuelta por vuelta: la condición se pregunta antes de cada entrada, y el ciclo corta apenas deja de cumplirse.

Vuelta	¿acumulado < 100000?	venta leída	acumulado	cantidad
inicio	—	—	0	0
1	0 < 100000: Verdadero	30.000	30.000	1
2	30.000 < 100000: Verdadero	24.000	54.000	2
3	54.000 < 100000: Verdadero	60.000	114.000	3
4	114.000 < 100000: Falso	corta el ciclo	114.000	3

La Figura 6.2 lo muestra en forma gráfica: el acumulador vuelta por vuelta frente a la meta de 100.000: con la tercera venta el acumulado llega a 114.000 y el ciclo corta.



Figura 6.2 — El acumulador vuelta por vuelta frente a la meta de 100.000: con la tercera venta el acumulado llega a 114.000 y el ciclo corta.

Resultado: la meta se alcanza con 3 ventas y la caja queda en G. 114.000. La cuarta pasada por la condición ya no entra al cuerpo del ciclo — el Mientras pregunta primero y ejecuta después.

Mientras o Repetir: cuál conviene

Criterio	Mientras	Repetir… Hasta Que
¿Cuándo evalúa la condición?	antes de cada vuelta	después de cada vuelta
¿Puede ejecutarse cero veces?	sí	no: al menos una vez
La condición…	de permanencia (sigue si es V)	de salida (corta cuando es V)
Uso típico en el copetín	acumular mientras falte para la meta	pedir la clave hasta que sea correcta

El ciclo infinito: cómo se produce y cómo se corta

Un Mientras cuya condición nunca se vuelve falsa no termina jamás: es el temido ciclo infinito. En la meta del día, bastaría olvidar la línea que acumula para que acumulado quede clavado en 0 — y 0 < 100000 sea verdadero para siempre.

Causa típica	Qué pasa	La vacuna
Olvidar actualizar la variable de control	la condición nunca cambia	verificá que el cuerpo modifique lo que la condición mira
Condición imposible de alcanzar	acumulado nunca llega a la meta	prueba de escritorio con datos extremos
Actualizar en el sentido contrario	el contador baja en vez de subir	leer el ciclo en voz alta antes de ejecutar

En PSeInt, un ciclo infinito se nota porque la ejecución no termina: cortala con el botón de detener y revisá el cuerpo del ciclo con la tabla de la prueba de escritorio. La pregunta mágica es siempre la misma: ¿qué instrucción de acá adentro acerca la condición a su final?

Repetir para validar: la carga que no acepta cualquier cosa

El otro trabajo clásico del Repetir… Hasta Que es la validación: volver a pedir un dato hasta que sea aceptable. Una cantidad de productos tiene que ser mayor que cero — y el ciclo insiste hasta lograrlo:

Ejemplo — validar la cantidad

```
Repetir
    Escribir "Cantidad (mayor que 0):"
    Leer cantidad
Hasta Que cantidad > 0
```

Fijate por qué acá el Repetir le gana al Mientras: pedir el dato al menos una vez es obligatorio — no se puede validar lo que todavía no se cargó. La condición de salida dice cómo es un dato bueno; el ciclo se encarga de rebotar los malos sin que el resto del algoritmo se entere.

Actividades de aplicación

Semana 2 del copetín (ventas diarias en guaraníes)

Día	Lun	Mar	Mié	Jue	Vie	Sáb	Dom
Venta	350.000	520.000	660.000	470.000	1.250.000	900.000	330.000

Ejercitá:

1. Indicá cuántas veces como mínimo se ejecuta un ciclo Repetir… Hasta Que.
2. Indicá cuántas veces como mínimo se ejecuta un Mientras.
3. Escribí la instrucción que incrementa un contador en uno.
4. Completá el arranque de un contador antes de un ciclo: contador <- ___.
5. Si el jueves de la primera semana hubiera vendido 650.000 en lugar de 580.000, ¿qué imprimiría el ejemplo de la clase?

Resolvé (caso Copetín Karumbé):

6. Escribí un algoritmo con Mientras que lea las 7 ventas de la semana 2 (tabla de datos) y cuente cuántas alcanzaron su promedio de 640.000.
7. Escribí un Repetir… Hasta Que que pida un monto y no salga hasta que el monto ingresado sea mayor que cero.
8. Con la semana 2, contá a mano cuántos días superan 500.000 y escribí la condición que usaría el contador.

Pensá y decidí (justificá tu respuesta):

9. V o F: un Mientras con condición inicial falsa no ejecuta su bloque ninguna vez. Justificá.
10. Elegí: para pedir un dato hasta que sea válido conviene (Mientras / Repetir… Hasta Que). Justificá.
11. V o F: si dentro del ciclo no cambia nada que afecte la condición, el ciclo puede volverse infinito. Justificá.

Desafío:

12. Escribí un algoritmo que recorra las 7 ventas de la semana 2 con un Mientras, cuente los días con venta menor a 500.000 y muestre el resultado. Verificá el conteo a mano.

Práctica 6 — Validar datos y llegar a la meta con ciclos

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Usar Repetir… Hasta Que para validar datos y Mientras para acumular hasta una meta, con contadores y acumuladores.
Entorno de trabajo: PSeInt.

Lo que necesitás saber

Repetir… Hasta Que se ejecuta al menos una vez y corta cuando la condición es Verdadera; Mientras puede no ejecutarse nunca y sigue mientras la condición es Verdadera.

Un contador suma 1 en cada vuelta; un acumulador suma un valor.

La librería tiene 12 carpetas en stock y se fija una meta de venta de 200.000 por turno.

Actividad 1 — Validar la cantidad

Objetivo: *Pedir una cantidad hasta que esté entre 1 y el stock disponible, contando los intentos.*

Modelo — Esqueleto para completar

```
Algoritmo ValidarCantidad
    Definir stock, cantidad, intentos Como Entero
    stock <- 12
    intentos <- 0
    Repetir
        Escribir "Cantidad de carpetas (1 a ", stock, "):"
        Leer cantidad
        intentos <- …
    Hasta Que (…) Y (…)
    Escribir "Cantidad aceptada: ", cantidad, " en ", intentos, " intentos"
FinAlgoritmo
```

Pasos:

1. Completá la condición de salida: el ciclo corta cuando la cantidad es válida.
2. Ejecutá ingresando, en orden, 0, 15 y 5.
3. Anticipá cuántos intentos va a informar antes de ingresar el último dato.

Punto de control: El programa rechaza los datos inválidos sin mensaje de error de PSeInt y la cantidad de intentos coincide con tu anticipación.

Actividad 2 — La meta del turno

Objetivo: *Acumular importes de tickets con Mientras hasta alcanzar la meta y contar los tickets.*

Modelo — Datos del turno (en este orden)

45.000 · 60.000 · 38.000 · 72.000 · 20.000

Pasos:

1. Escribí el algoritmo MetaLibreria con un acumulador, un contador y un Mientras acumulado < 200000.
2. Hacé primero la prueba de escritorio en una tabla (vuelta, ticket, acumulado, tickets).
3. Ejecutá ingresando los importes en orden; si el programa deja de pedir datos, no ingreses más.

Punto de control: El programa deja de pedir importes en la misma vuelta en que tu prueba de escritorio supera la meta, y muestra el mismo acumulado.

Desafío final (para quienes terminan antes)

¿Qué pasaría con la Actividad 2 si la meta fuera 0? Cambiala, ejecutá y explicá el resultado con la diferencia entre Mientras y Repetir.

Clase 7 — Repetir una cantidad conocida: Para, acumulador, bandera y centinela

Capacidad	Resuelve problemas utilizando ciclos repetitivos, contadores, acumuladores y banderas.
Tema	Repetir una cantidad conocida: Para, acumulador, bandera y centinela
Indicadores de logro	• Implementa un ciclo Para cuando la cantidad de repeticiones se conoce de antemano. • Usa un acumulador para sumar valores dentro de un ciclo y una bandera para marcar una condición. • Aplica el valor centinela para cortar un ciclo de lectura de datos.

El ciclo Para

Cuando se sabe cuántas veces hay que repetir (por ejemplo, las 7 ventas de la semana), el ciclo Para es el más cómodo. Lleva su propia variable de control, que arranca en un valor, avanza de a un paso y termina en un tope, sin que haga falta incrementarla a mano.

El acumulador

Un acumulador es una variable que va sumando valores dentro de un ciclo. Se inicia en cero y en cada vuelta hace acumulador <- acumulador + valor. Así se obtiene, por ejemplo, el total de ventas de la semana sumando las 7 ventas diarias.

Ejemplo — total de la semana con un acumulador

```
total <- 0
Para i <- 1 Hasta 7 Con Paso 1 Hacer
    Leer ventaDia
    total <- total + ventaDia
FinPara
promedio <- total / 7
Escribir "Total: ", total, " Promedio: ", promedio
```

Con las ventas de la semana: total = 4.200.000 y promedio = 600.000.

La bandera

Una bandera (o variable lógica) marca si algo ocurrió durante el ciclo. Arranca en Falso y pasa a Verdadero cuando se cumple la condición buscada; por ejemplo, "hubo al menos un día sin ventas". Al final del ciclo se consulta la bandera para decidir.

Herramienta	Se inicia en	Dentro del ciclo	Sirve para
Contador	0	cont <- cont + 1	contar ocurrencias
Acumulador	0	acum <- acum + valor	sumar valores
Bandera	Falso	bandera <- Verdadero	marcar que algo pasó
Centinela	—	cortar al leer el valor especial	fin de carga desconocida

El valor centinela

Cuando no se sabe cuántos datos vienen, se usa un centinela: un valor especial que indica el fin de la carga (por ejemplo, cargar ventas hasta que se ingrese 0). El ciclo se repite mientras el dato leído sea distinto del centinela.

Concepto clave

El ciclo Para se usa cuando se conoce la cantidad de repeticiones. Un acumulador suma valores, una bandera marca si ocurrió una condición y un centinela corta la carga cuando no se sabe cuántos datos vienen.

La caja del día con centinela

Al cierre, el cajero carga los montos de todas las ventas y teclea 0 para avisar que terminó. Ese 0 no es una venta: es el centinela, un valor imposible como dato real que marca el final de la carga. El Repetir… Hasta Que es su pareja natural, porque siempre hay al menos un intento de carga.

Ejemplo — el cierre de caja con centinela

```
Algoritmo CierreDeCaja
    total <- 0
    cantidad <- 0
    Repetir
        Leer monto
        Si monto <> 0 Entonces
            total <- total + monto
            cantidad <- cantidad + 1
        FinSi
    Hasta Que monto = 0
    Si cantidad > 0 Entonces
        promedio <- total / cantidad
        Escribir cantidad, " ventas - Total: ", total, " - Promedio: ", promedio
    Sino
        Escribir "No se cargaron ventas"
    FinSi
FinAlgoritmo
```

Ejemplo resuelto paso a paso — el cierre de un mediodía

El cajero carga G. 30.000, G. 54.000, G. 24.000, G. 12.000 y por último 0. Seguí el total y el contador vuelta por vuelta:

monto leído	¿es 0?	total	cantidad
30.000	no: acumula	30.000	1
54.000	no: acumula	84.000	2
24.000	no: acumula	108.000	3
12.000	no: acumula	120.000	4
0	sí: corta el ciclo	120.000	4

La Figura 7.1 lo muestra en forma gráfica: el centinela 0 corta la carga sin sumarse: el cierre queda en 4 ventas, G. 120.000 de total y G. 30.000 de promedio.

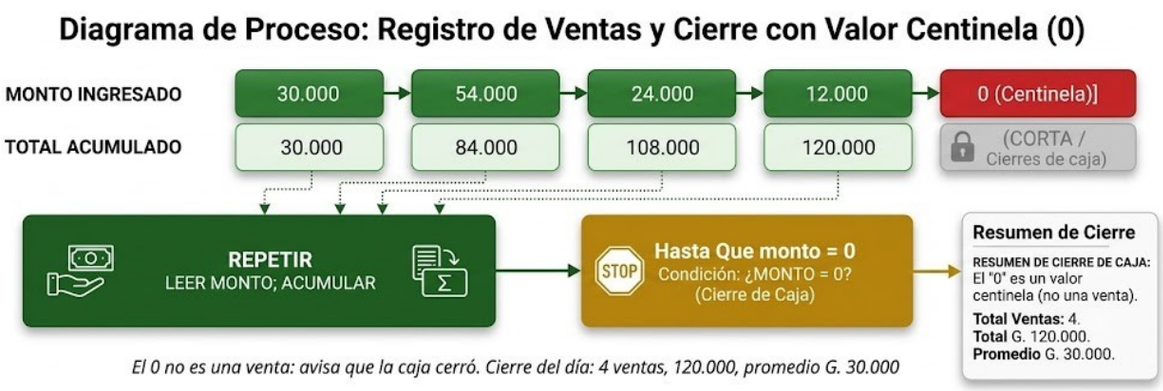


Figura 7.1 — El centinela 0 corta la carga sin sumarse: el cierre queda en 4 ventas, G. 120.000 de total y G. 30.000 de promedio.

El Si interior evita el error clásico: sin él, el 0 final «sumaría» una venta fantasma al contador y el promedio saldría mal. Con 4 ventas reales, el promedio es 120.000 / 4 = G. 30.000.

Contador, acumulador, bandera y centinela: el equipo completo

Herramienta	Qué guarda	Cómo cambia	En el cierre de caja
Contador	cuántas veces pasó algo	de a 1 (cantidad <- cantidad + 1)	cuenta las ventas: 4
Acumulador	una suma que crece	suma el dato (total <- total + monto)	junta el total: 120.000
Bandera	un estado lógico (V/F)	cambia cuando ocurre el evento	«¿hubo venta mayor a 50.000?»: Verdadero
Centinela	no guarda: es un dato especial	el usuario lo teclea para terminar	el 0 del final

Concepto clave

Las cuatro herramientas suelen convivir en el mismo ciclo: el centinela decide cuándo parar, el contador y el acumulador arman el informe, y la bandera registra si ocurrió algo digno de aviso.

Elegir el ciclo correcto: el mapa completo de la unidad

Con las tres estructuras repetitivas en la mochila, la pregunta ya no es «cómo se escribe cada una» sino «cuál corresponde acá». El mapa de decisión que vas a usar el resto del año:

¿Sabés cuántas veces repetir?	¿Debe ejecutarse al menos una vez?	Ciclo	Ejemplo del copetín
sí, de antemano	—	Para	las 7 ventas de la semana
no	no necesariamente	Mientras	acumular hasta la meta del día
no	sí, siempre	Repetir… Hasta Que	cargar montos hasta el centinela 0

Un mismo problema a veces admite dos ciclos — la caja con centinela puede escribirse con Mientras si se lee el primer monto antes de entrar —, pero siempre hay uno que queda más natural y más corto. La elegancia en programación no es un lujo: el código natural tiene menos lugares donde esconder errores.

Actividades de aplicación

Semana 2 del copetín (ventas diarias en guaraníes)

Día	Lun	Mar	Mié	Jue	Vie	Sáb	Dom
Venta	350.000	520.000	660.000	470.000	1.250.000	900.000	330.000

Ejercitá:

1. Escribí el encabezado de un Para que repita 7 veces (de 1 a 7).
2. Escribí la instrucción que acumula ventaDia en total.
3. Indicá el valor inicial de una bandera.
4. Completá: se corta la carga cuando el dato leído es igual al ___.
5. Con las ventas de la semana 2 (tabla de datos), calculá el total y el promedio.

Resolvé (caso Copetín Karumbé):

6. Escribí un algoritmo con Para que lea las 7 ventas de la semana 2, las acumule en total y muestre el promedio.
7. Escribí un algoritmo con centinela que cargue ventas hasta que se ingrese 0, y muestre cuántas ventas se cargaron (contador) y el total (acumulador).
8. Usá una bandera para detectar si en la semana hubo algún día con venta menor a 400.000. Con las ventas de la semana, indicá el valor final de la bandera.

Pensá y decidí (justificá tu respuesta):

9. V o F: el ciclo Para conviene cuando NO se sabe cuántas veces repetir. Justificá.
10. Elegí: para sumar las ventas de la semana se usa un (contador / acumulador). Justificá.
11. V o F: una bandera guarda un número. Justificá.

Desafío:

12. Escribí un algoritmo que recorra las 7 ventas con un Para, acumule el total, cuente con un contador los días que alcanzaron 640.000 y active una bandera si algún día vendió más de 1.000.000. Con la semana 2, anotá total, contador y bandera.

Práctica 7 — La semana de la librería: Para, acumulador, centinela y bandera

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Resolver problemas de la semana con ciclo Para, acumuladores, contadores, centinela y banderas, verificando con prueba de escritorio.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

El Para se usa cuando se conoce la cantidad de repeticiones; el centinela, cuando la carga termina con un valor especial.

Una bandera es una variable lógica que arranca en Falso y pasa a Verdadero cuando ocurre algo.

La librería abre de lunes a sábado. Ventas de la semana: 180.000 · 240.000 · 150.000 · 310.000 · 290.000 · 210.000.

Actividad 1 — Total y promedio con Para

Objetivo: *Acumular las seis ventas con un Para y calcular el promedio.*

Modelo — Esqueleto para completar

```
Algoritmo SemanaLibreria
    Definir venta, total, i Como Entero
    Definir promedio Como Real
    total <- 0
    Para i <- 1 Hasta 6 Con Paso 1 Hacer
        Leer venta
        total <- …
    FinPara
    promedio <- …
    Escribir "Total: ", total
    Escribir "Promedio: ", promedio
FinAlgoritmo
```

Pasos:

1. Calculá a mano el total y el promedio y anotalos.
2. Completá el esqueleto y ejecutá con las seis ventas.
3. Explicá por qué total arranca en 0 antes del ciclo y no dentro.

Punto de control: El total y el promedio de la pantalla coinciden con los anotados.

Actividad 2 — Los tickets del turno con centinela

Objetivo: *Cargar importes hasta que se ingrese 0 y mostrar cuántos tickets hubo y el total.*

Modelo — Datos del turno

12.000 · 8.500 · 30.000 · 4.500 · 0 (centinela)

Pasos:

1. Escribí el algoritmo TicketsDelTurno: una lectura antes del Mientras y otra al final del cuerpo.
2. Ejecutá con los datos del turno.
3. Ejecutá otra vez ingresando 0 de entrada.

Punto de control: El 0 no se cuenta como ticket, y la ejecución que empieza con 0 informa un turno vacío sin error.

Actividad 3 — Contador y bandera en el mismo recorrido

Objetivo: *Contar los días que alcanzaron el promedio y detectar con una bandera si hubo un día por debajo de 160.000.*

Modelo — Qué tiene que informar

Cuántos días alcanzaron 230.000 (el promedio de la Actividad 1).

Si hubo algún día por debajo de 160.000 (bandera) y cuál fue.

Pasos:

1. Escribí el algoritmo ControlSemana con un Para, un contador, una bandera hubo y una variable diaBajo.

2. Hacé la prueba de escritorio con las seis ventas antes de ejecutar.
3. Ejecutá y compará.

Punto de control: El contador y la bandera coinciden con tu prueba de escritorio. Si PSeInt marca «Identificador no válido», revisá que ninguna variable se llame igual que el algoritmo.

Transferencia y revisión entre pares (30 a 40 minutos)

Objetivo: *aplicar lo mismo a un segundo caso, con datos diferentes, y mejorar el trabajo con la revisión de otro equipo.*

Modelo — Mini-caso: la cantina del colegio

La cantina abre cinco días. Ventas: 95.000 · 120.000 · 80.000 · 140.000 · 115.000.

Programá en un solo algoritmo: total, promedio, cuántos días superaron 100.000 y una bandera que indique si algún día vendió menos de 85.000.

Pasos:

1. Adaptá el algoritmo de la Actividad 3 a cinco días y a los nuevos umbrales.
2. Intercambiá el programa con otro equipo: ejecutalo con los datos de la cantina y buscá al menos un error o una mejora.
3. Corregí tu versión con lo que te señalaron y justificá por escrito la versión final.

Punto de control: Las dos versiones del equipo (la original y la corregida) dan los mismos resultados que la prueba de escritorio de la cantina, y la justificación nombra el cambio hecho.

Desafío final (para quienes terminan antes)

Modificá la Actividad 3 para que también informe el mejor día (número de día y venta) sin usar vectores.

Evaluación de la Unidad 1

Del problema al programa: estructuras de control (Clases 1 a 7)

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Datos de la evaluación — la heladería Yvoty

Ventas de la semana (cinco días): 150.000 · 230.000 · 90.000 · 310.000 · 220.000.

Parte A — Diagnóstico rápido (marcá la opción correcta):

1. La estructura que se ejecuta al menos una vez es:
   a) Mientras
   b) Repetir… Hasta Que
   c) Si… Sino
   d) Segun

2. El operador lógico que es verdadero solo si ambas condiciones lo son es:
   a) O
   b) NO
   c) Y
   d) <>

3. Una variable que se inicia en cero y suma valores dentro de un ciclo es un:
   a) acumulador
   b) contador
   c) centinela
   d) bandera

Parte B — Resolución y aplicación:

4. Clasificá el tipo de dato de: 25 ; 12,5 ; "Frutilla" ; Falso ; "S".

5. La heladería regala un cucurucho extra si la compra llega a 40.000. Escribí la estructura Si… Sino que muestre «Con regalo» o «Sin regalo» y probala con 40.000 y con 39.500.

6. Escribí la condición compuesta «el pedido llega a 60.000 O el cliente tiene tarjeta de puntos» y evaluala para monto = 45.000 y tarjeta = Verdadero.

7. Escribí un algoritmo con Para que lea las cinco ventas de la semana, las acumule y muestre el total y el promedio.

8. Con las ventas de la heladería, calculá el total y el promedio y contá cuántos días alcanzaron el promedio.

9. Escribí un algoritmo con centinela que cargue montos hasta que se ingrese 0 y muestre cuántos montos se cargaron y su suma. Hacé la prueba de escritorio con 12.000, 18.000 y 0.

UNIDAD 2 — Estructuras de datos y modularización

En esta unidad vas a guardar muchos datos juntos en vectores y matrices, a recorrerlos, buscarlos y ordenarlos, y a dividir los programas en módulos con parámetros, incluido el trabajo con cadenas de texto.

Clase 8 — Vectores: concepto, declaración, carga y recorrido

Capacidad	Utiliza estructuras de datos estáticas (vectores y matrices) para el planeamiento y la solución de problemas de la vida diaria (lógico, matemático, comerciales, otros.)
Tema	Vectores: concepto, declaración, carga y recorrido
Indicadores de logro	• Declara un vector indicando su nombre, tipo y dimensión. • Carga los elementos de un vector recorriéndolo con un ciclo Para. • Accede a un elemento del vector mediante su índice.

Un problema: muchas variables para lo mismo

Guardar las 7 ventas de la semana en 7 variables sueltas (ventaLunes, ventaMartes…) es incómodo: cada operación habría que escribirla siete veces. La solución es un vector (o arreglo): una sola variable que guarda muchos valores del mismo tipo, numerados por un índice.

Configuración de PSeInt usada en este material — Desde esta clase trabajamos con el perfil flexible/estándar y con índices en base 1: un vector Dimension ventas[7] se recorre de 1 a 7 y las posiciones de Subcadena usan la misma base. Antes de las Clases 8–14, verificá Configurar → Opciones del Lenguaje (perfiles). Si el laboratorio usa base 0, hay que adaptar TODOS los límites e índices de manera coherente; no mezcles ambas convenciones.

Declaración y dimensión

Un vector se declara indicando su nombre, su tipo y cuántos elementos guarda (su dimensión). En pseudocódigo: Dimension ventas[7] reserva siete casilleros. Cada casillero se identifica con un índice entre 1 y 7: ventas[1], ventas[2], …, ventas[7].

El índice es la posición; el contenido es el valor guardado en esa posición. No hay que confundirlos: ventas[6] vale 1.100.000, pero 6 es la posición y 1.100.000 el valor.

Ejemplo — cargar las ventas de la semana

```
Dimension ventas[7]
Para i <- 1 Hasta 7 Con Paso 1 Hacer
    Escribir "Venta del día ", i, ":"
    Leer ventas[i]
FinPara
Escribir "La venta del sábado fue: ", ventas[6]
```

Si se cargan 420000, 500000, 450000, 580000, 850000, 1100000 y 300000, entonces ventas[6] muestra 1.100.000.

Un vector se dibuja como una fila de casilleros: observá la diferencia entre el índice (la posición, en ámbar) y el contenido (el valor guardado en cada casillero).

La Figura 8.1 lo muestra en forma gráfica: el vector ventas[7] como fila de casilleros: el índice es la posición (1 a 7) y el contenido, el valor guardado.

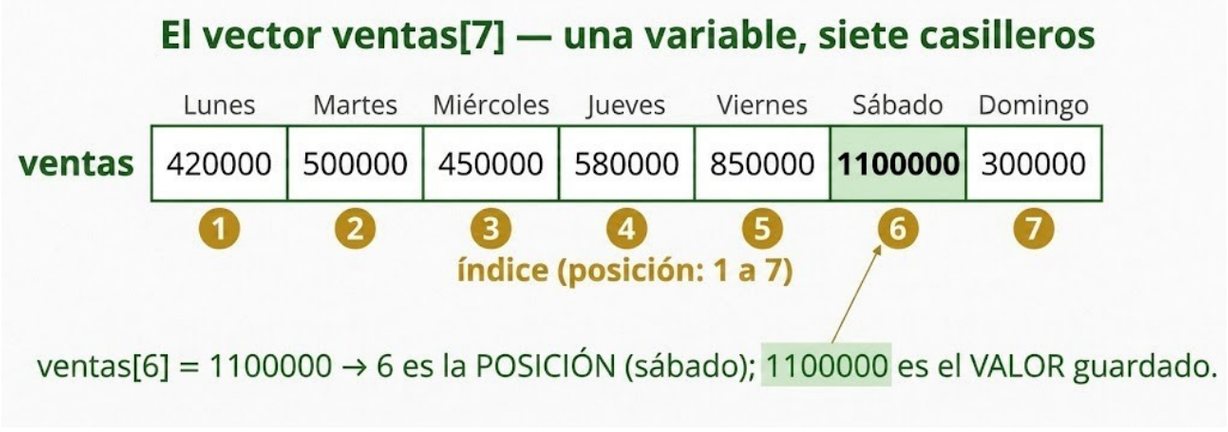


Figura 8.1 — El vector ventas[7] como fila de casilleros: el índice es la posición (1 a 7) y el contenido, el valor guardado.

Cargar el vector con un ciclo

Como las posiciones van de 1 a 7, un ciclo Para recorre el vector cargando cada casillero. La variable de control del Para hace de índice.

Índice	Día	Contenido ventas[i]
1	Lunes	420000
2	Martes	500000
3	Miércoles	450000
4	Jueves	580000
5	Viernes	850000
6	Sábado	1100000
7	Domingo	300000

Acceder y mostrar un elemento

Para usar un valor se lo llama por su índice. ventas[6] es la venta del sábado. Cambiar un valor es asignar: ventas[3] <- 460000 reemplaza la venta del miércoles.

Concepto clave

Un vector es una variable que guarda muchos valores del mismo tipo, identificados por un índice. Se declara con Dimension nombre[tamaño] y se recorre con un ciclo.

Ejemplo cotidiano — la planilla de la despensa

La planilla de ventas diarias que lleva cualquier despensa es, en la práctica, un vector: una fila de casilleros, uno por día, que después se suman para el cierre del mes.

Vectores paralelos: precios y stock del mismo producto

El copetín necesita dos datos por producto: su precio y su stock. La solución con vectores es tener dos, paralelos, donde el mismo índice apunta al mismo producto: precios[3] y stock[3] hablan ambos del Sándwich mixto. El orden de carga tiene que ser idéntico en los dos — ese es el contrato de los vectores paralelos.

índice j	1	2	3	4	5	6	7
producto	Chipa	Empanada	Mixto	Coquito	Gaseosa	Cocido	Sopa
precios[j]	3.000	5.000	12.000	500	8.000	6.000	10.000
stock[j]	40	30	8	200	25	12	6

Ejemplo resuelto paso a paso — consultar un producto por su índice

¿Cuánto vale el stock completo del Sándwich mixto? El producto ocupa la posición 3, así que la respuesta sale de combinar los dos vectores en esa misma posición.

Ejemplo — consulta con vectores paralelos

```
Dimension precios[7]
Dimension stock[7]
// ... carga en el orden de la tabla de productos ...
j <- 3
valor <- precios[j] * stock[j]
Escribir "Valor del stock: ", valor
```

Paso a paso: precios[3] vale 12.000 y stock[3] vale 8, así que valor <- 12.000 × 8 = G. 96.000. Con j <- 4, la misma línea calcularía el valor del stock de coquitos: 500 × 200 = G. 100.000. Una sola fórmula sirve para los siete productos: solo cambia el índice.

Concepto clave

Dos o más vectores son paralelos cuando comparten el índice: la posición j guarda datos distintos del MISMO elemento. Cargarlos en órdenes diferentes rompe el paralelismo y corrompe todos los cálculos.

Modificar el vector: la corrección del miércoles

Los vectores no son de solo lectura: asignar sobre una posición reemplaza su contenido. Supongamos que el cierre del miércoles estuvo mal cargado y el valor correcto era G. 460.000. Una sola instrucción corrige el dato:

Ejemplo — corregir una posición

```
// antes: ventas[3] = 450000, total = 4200000
ventas[3] <- 460000
// el total hay que recalcularlo recorriendo de nuevo
```

Ojo con el detalle final: corregir la posición no corrige los cálculos ya hechos. El total recalculado tras la corrección sería 4.200.000 − 450.000 + 460.000 = G. 4.210.000. En un sistema real, toda modificación de datos obliga a rehacer los informes que dependían de ellos — por eso los sistemas guardan los datos y recalculan los informes cada vez, en lugar de guardar los resultados.

El índice fuera de rango: el error que rompe programas

El vector ventas[7] tiene posiciones de la 1 a la 7 — ni la 0 ni la 8 existen. Pedir ventas[8] es el error de índice fuera de rango, y es de los que detienen la ejecución en seco. Sus disfraces más comunes:

Cómo se produce	Ejemplo	La vacuna
Ciclo que se pasa una posición	Para i <- 1 Hasta 8	el límite del Para = la dimensión
Empezar en 0 por costumbre	ventas[0]	en pseudocódigo los vectores arrancan en 1
Índice calculado sin control	ventas[dia + 3] con dia = 5	validar el índice antes de usarlo

La defensa profesional es validar el índice con lo que ya sabés de la Unidad 1: Si indice >= 1 Y indice <= 7 Entonces… El operador Y custodiando las dos fronteras del vector — las unidades del libro se dan la mano.

Actividades de aplicación

Ejercitá:

1. Declará un vector llamado precios que guarde 5 valores.
2. Indicá qué posición ocupa la venta del jueves en el vector ventas.
3. Escribí la instrucción que guarda 460000 en la posición 3 del vector ventas.
4. Indicá el valor de ventas[6] según la tabla.
5. Escribí el encabezado del ciclo Para que recorre un vector de 7 elementos.

Resolvé (caso Copetín Karumbé):

6. Escribí un algoritmo que declare el vector ventas[7], cargue las 7 ventas de la primera semana y muestre la venta del miércoles.
7. Escribí un algoritmo que cargue un vector precios[5] con los precios 3.000, 5.000, 12.000, 500 y 8.000 y muestre el tercer precio.
8. Con el vector de ventas cargado, escribí la instrucción que muestra juntas la venta del lunes y la del domingo.

Pensá y decidí (justificá tu respuesta):

9. V o F: en un vector, el índice y el valor guardado son lo mismo. Justificá.
10. Elegí: para recorrer un vector de tamaño conocido conviene un ciclo (Para / Repetir). Justificá.
11. V o F: Dimension ventas[7] reserva 7 casilleros. Justificá.

Desafío:

12. Escribí un algoritmo que cargue el vector ventas[7] y un vector dias[7] con los nombres, y muestre, uno por uno, cada día con su venta. Anotá qué línea aparece en sexto lugar.

Práctica 8 — El vector de precios y la consulta por número

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Declarar, cargar y consultar vectores (incluidos vectores paralelos) validando el índice antes de usarlo.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

Un vector se declara con Dimension y cada casillero se identifica por su índice: en el perfil Flexible, de 1 al tamaño.

Dos vectores son paralelos cuando la misma posición describe el mismo objeto: nombres[3] y precios[3] hablan del mismo artículo.

Precios de la librería, en este orden: Cuaderno 8.000 · Bolígrafo 3.000 · Lápiz 2.000 · Regla 4.000 · Carpeta 15.000 · Resaltador 6.000 · Goma 1.500.

Actividad 1 — Cargar y mostrar

Objetivo: *Cargar el vector precios[7] con un Para y mostrar elementos por su índice.*

Modelo — Esqueleto para completar

```
Algoritmo VectorPrecios
    Definir precios, i Como Entero
    Dimension precios[7]
    Para i <- 1 Hasta 7 Con Paso 1 Hacer
        Leer …
    FinPara
    Escribir "Artículo 5: ", …
    Escribir "Primero y último: ", …, " y ", …
FinAlgoritmo
```

Pasos:

1. Anotá en papel qué valor debería mostrar cada línea de salida.
2. Completá el esqueleto y ejecutá cargando los siete precios en orden.
3. Ejecutá otra vez cargando los precios en orden inverso y explicá qué cambió.

Punto de control: Las dos ejecuciones muestran los valores que anticipaste para cada orden de carga.

Actividad 2 — Consulta con vectores paralelos

Objetivo: *Consultar nombre y precio de un artículo por su número, validando que el número exista.*

Modelo — Comportamiento esperado

Se cargan nombres[7] y precios[7] con asignaciones (no con Leer).

Se lee un número de artículo.

Si está entre 1 y 7, se muestra «nombre: precio»; si no, «No existe ese artículo».

Pasos:

1. Escribí el algoritmo ConsultaArticulo con los dos vectores paralelos.
2. Probalo con los números 6 y 9.
3. Quitá la validación, ejecutá con 9 y copiá el mensaje que da PSeInt. Después volvé a poner la validación.

Punto de control: Con la validación, ningún número produce un error de PSeInt; sin ella, el número fuera de rango detiene el programa.

Desafío final (para quienes terminan antes)

Agregá un tercer vector paralelo con el stock de cada artículo y mostrá, para el número consultado, cuántas unidades quedan y cuánto dinero representan.

Clase 9 — Operaciones con vectores: suma, promedio, máximo, mínimo y búsqueda

Capacidad	Utiliza estructuras de datos estáticas (vectores y matrices) para el planeamiento y la solución de problemas de la vida diaria (lógico, matemático, comerciales, otros.)
Tema	Operaciones con vectores: suma, promedio, máximo, mínimo y búsqueda
Indicadores de logro	• Calcula la suma y el promedio de los elementos de un vector con un acumulador. • Encuentra el mayor y el menor elemento de un vector y su posición. • Busca un valor dentro de un vector recorriéndolo con un ciclo.

Recorrer para calcular

Una vez cargado el vector, la mayoría de los informes salen de recorrerlo. Sumar todas las ventas, sacar el promedio, encontrar el mejor día o buscar si hubo un día sin ventas son todas operaciones de recorrido.

Suma y promedio

La suma usa un acumulador: se inicia en cero y en cada posición se le agrega ventas[i]. El promedio es la suma dividida por la cantidad de elementos. Para las 7 ventas de la semana, la suma da 4.200.000 y el promedio 600.000.

Ejemplo — total, promedio y mejor día

```
suma <- 0
mayor <- ventas[1]
posMayor <- 1
Para i <- 1 Hasta 7 Con Paso 1 Hacer
    suma <- suma + ventas[i]
    Si ventas[i] > mayor Entonces
        mayor <- ventas[i]
        posMayor <- i
    FinSi
FinPara
Escribir "Total: ", suma, " Promedio: ", suma/7
Escribir "Mejor día: posición ", posMayor, " con ", mayor
```

Resultado: Total 4.200.000, Promedio 600.000, mejor día posición 6 con 1.100.000.

Máximo y mínimo

Para el máximo se supone que el primer elemento es el mayor y se recorre el resto: si aparece uno más grande, se lo guarda (y se guarda su posición). El mínimo funciona igual, al revés. En la semana, el máximo es 1.100.000 (posición 6, sábado) y el mínimo 300.000 (posición 7, domingo).

Seguí en la figura cómo cambia la variable mayor a medida que el recorrido avanza: solo se actualiza cuando aparece un valor más grande, y con él se guarda la posición.

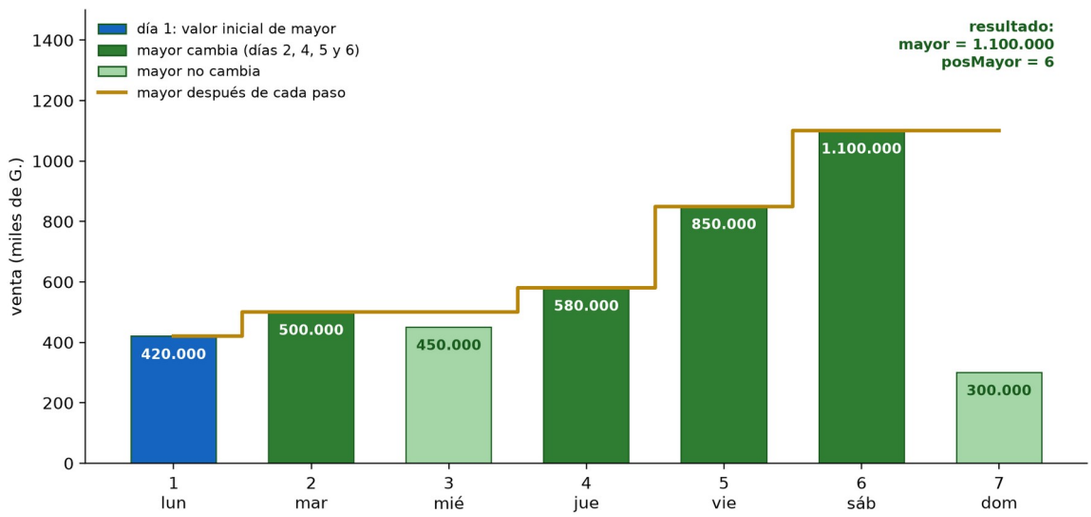


Figura 9.1 — El recorrido del máximo sobre la semana del copetín: mayor cambia en los días 2, 4, 5 y 6.

Operación	Se apoya en	Resultado (semana)
Suma	acumulador	4.200.000
Promedio	suma / 7	600.000
Máximo	comparación	1.100.000 (pos. 6)
Mínimo	comparación	300.000 (pos. 7)
Búsqueda de 850.000	bandera	Encontrado (pos. 5)

Búsqueda de un valor

Buscar es recorrer preguntando si el elemento coincide con lo buscado. Una bandera marca si se encontró. Por ejemplo, averiguar si algún día vendió exactamente 850.000.

Concepto clave

Las operaciones sobre un vector se resuelven recorriéndolo: un acumulador para sumar y promediar, comparaciones para el máximo y el mínimo, y una bandera para buscar.

Todo en un solo recorrido

Total, promedio, mejor día y peor día pueden calcularse con cuatro recorridos separados… o con uno solo que lo haga todo. En sistemas reales, donde los vectores tienen miles de posiciones, recorrer una sola vez es la diferencia entre un informe instantáneo y uno lento.

Ejemplo — el informe semanal en un recorrido

```
total <- 0
mayor <- ventas[1] ; posMayor <- 1
menor <- ventas[1] ; posMenor <- 1
Para i <- 1 Hasta 7 Con Paso 1 Hacer
    total <- total + ventas[i]
    Si ventas[i] > mayor Entonces
        mayor <- ventas[i] ; posMayor <- i
    FinSi
    Si ventas[i] < menor Entonces
        menor <- ventas[i] ; posMenor <- i
    FinSi
FinPara
promedio <- total / 7
```

Ejemplo resuelto paso a paso — el informe de la semana

Con las ventas de la semana del copetín, seguí las variables clave en los momentos donde cambian:

Día (i)	ventas[i]	total acumulado	mayor (posición)	menor (posición)
1 Lunes	420.000	420.000	420.000 (1)	420.000 (1)
2 Martes	500.000	920.000	500.000 (2)	420.000 (1)
3 Miércoles	450.000	1.370.000	500.000 (2)	420.000 (1)
4 Jueves	580.000	1.950.000	580.000 (4)	420.000 (1)
5 Viernes	850.000	2.800.000	850.000 (5)	420.000 (1)
6 Sábado	1.100.000	3.900.000	1.100.000 (6)	420.000 (1)
7 Domingo	300.000	4.200.000	1.100.000 (6)	300.000 (7)

El informe final: total G. 4.200.000, promedio 4.200.000 / 7 = G. 600.000, mejor día el sábado (posición 6, G. 1.100.000) y peor día el domingo (posición 7, G. 300.000). Solo dos días — viernes y sábado — alcanzaron o superaron el promedio: el promedio no es «la mitad de los días», es la suma repartida en partes iguales.

La búsqueda con bandera

¿Hubo algún día con venta de exactamente G. 850.000? La bandera arranca en Falso y solo cambia si el recorrido encuentra el valor. Guardar también la posición permite responder «sí, el viernes» en lugar de un simple «sí».

Ejemplo — buscar 850.000 en la semana

```
encontrado <- Falso
posicion <- 0
Para i <- 1 Hasta 7 Con Paso 1 Hacer
    Si ventas[i] = 850000 Entonces
        encontrado <- Verdadero
        posicion <- i
    FinSi
FinPara
// encontrado = Verdadero, posicion = 5 (viernes)
```

El segundo mejor día

Un informe pide a veces más que el máximo: el podio. Encontrar el segundo mayor exige mantener DOS variables durante el recorrido y actualizar con cuidado: cuando aparece un nuevo máximo, el viejo máximo pasa a segundo.

Ejemplo — máximo y segundo en un recorrido

```
mayor <- ventas[1] ; segundo <- 0
Para i <- 2 Hasta 7 Con Paso 1 Hacer
    Si ventas[i] > mayor Entonces
        segundo <- mayor
        mayor <- ventas[i]
    Sino
        Si ventas[i] > segundo Entonces
            segundo <- ventas[i]
        FinSi
    FinSi
FinPara
```

Con la semana del copetín, el recorrido termina con mayor = 1.100.000 (sábado) y segundo = 850.000 (viernes). El orden de las dos asignaciones del primer bloque es crítico: si asignaras mayor antes de salvar su valor en segundo, el viejo máximo se perdería. Probalo con la prueba de escritorio y vas a ver el error enseguida. Un detalle más: segundo arranca en 0 porque las ventas nunca son negativas; si el vector pudiera tener valores negativos, convendría arrancar con el menor de los dos primeros elementos.

Contar con condición: otro patrón del recorrido

Entre sumar todo y buscar un valor hay un patrón intermedio: contar cuántos elementos cumplen una condición. ¿Cuántos días superaron los G. 500.000? El contador solo avanza cuando la condición da verdadera:

Ejemplo — contar los días buenos

```
buenos <- 0
Para i <- 1 Hasta 7 Con Paso 1 Hacer
    Si ventas[i] > 500000 Entonces
        buenos <- buenos + 1
    FinSi
FinPara
// buenos = 3 (jueves 580.000, viernes 850.000, sábado 1.100.000)
```

Tres días de la semana superaron los 500.000. Fijate que el martes, con exactamente 500.000, no cuenta: la condición es estrictamente mayor. Cambiarla a >= lo incluiría y el resultado sería 4 — otra vez el borde de la condición decidiendo el informe, como en la Clase 4.

Actividades de aplicación

Semana 2 del copetín (ventas diarias en guaraníes)

Día	Lun	Mar	Mié	Jue	Vie	Sáb	Dom
Venta	350.000	520.000	660.000	470.000	1.250.000	900.000	330.000

Ejercitá:

1. Escribí la instrucción que acumula ventas[i] en suma.
2. Indicá el promedio de la semana 2 si la suma es 4.480.000 y hay 7 elementos.
3. Indicá la posición del máximo en el vector de la semana 2.
4. Indicá el valor del mínimo del vector de la semana 2 y su posición.
5. ¿Con qué valor conviene iniciar la variable mayor antes de recorrer?

Resolvé (caso Copetín Karumbé):

6. Escribí un algoritmo que recorra ventas[7] con los datos de la semana 2, acumule el total y muestre el promedio.

7. Escribí un algoritmo que encuentre el mayor elemento de ventas[7] y la posición donde está. Probalo con la semana 2.

8. Escribí un algoritmo con bandera que indique si en la semana 2 hubo algún día con venta de exactamente 900.000 y, si lo hubo, en qué posición.

Pensá y decidí (justificá tu respuesta):

9. V o F: para hallar el máximo conviene iniciar mayor en cero. Justificá.

10. Elegí: el promedio se calcula (suma/cantidad / suma*cantidad). Justificá.

11. V o F: buscar un valor en un vector siempre exige recorrerlo. Justificá.

Desafío:

12. Escribí un algoritmo que recorra ventas[7] y muestre en un solo recorrido: el total, el promedio, el mejor día (posición y valor) y el peor día. Verificá con los datos de la semana 2.

Práctica 9 — Recorrer el vector de unidades vendidas

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Calcular total, promedio, máximo, mínimo y búsqueda secuencial recorriendo un vector, verificando con prueba de escritorio.
Entorno de trabajo: PSeInt.

Lo que necesitás saber

Para el máximo y el mínimo se arranca con el primer elemento y se actualiza cuando aparece uno mayor (o menor), guardando la posición.

La búsqueda secuencial compara elemento por elemento y puede cortar en cuanto encuentra el buscado.

Unidades vendidas en la semana, por artículo (mismo orden que en la Práctica 8): 35 · 120 · 90 · 18 · 12 · 44 · 59.

Actividad 1 — Total, promedio, máximo y mínimo

Objetivo: *Recorrer el vector unid[7] y calcular total, promedio, cuántos superan el promedio, máximo y mínimo con sus posiciones.*

Modelo — Qué tiene que informar

Total y promedio de unidades.
Cuántos artículos superan el promedio.
Mayor y menor, con su posición.

Pasos:

1. Hacé la prueba de escritorio del máximo en una tabla (i, unid[i], mayor, posMayor).
2. Escribí el algoritmo UnidadesSemana: un Para de carga, uno para el total y otro para el resto.
3. Ejecutá con los siete valores.

Punto de control: Los seis datos de la pantalla coinciden con tu prueba de escritorio; el total dividido por 7 da exactamente el promedio informado.

Actividad 2 — Buscar un artículo por su nombre

Objetivo: *Programar una búsqueda secuencial que se detenga al encontrar y cuente las comparaciones.*

Modelo — Esqueleto para completar

```
pos <- 0
i <- 1
Mientras (i <= 7) Y (pos = 0) Hacer
    Si nombres[i] = buscado Entonces
        pos <- i
    FinSi
    i <- i + 1
FinMientras
```

Pasos:

1. Completá el algoritmo BuscarArticulo con el vector nombres[7] de la Práctica 8 y el mensaje final (encontrado en la posición… o no está).
2. Buscá «Carpeta», «Compás» y «carpeta» (en minúscula).
3. Anotá cuántas comparaciones hizo cada búsqueda.

Punto de control: Una búsqueda que encuentra hace menos comparaciones que una que no encuentra. Explicá con tus palabras el resultado de «carpeta».

Desafío final (para quienes terminan antes)

Ampliá la Actividad 1 para informar también el segundo artículo más vendido, en el mismo recorrido.

Clase 10 — Matrices: arreglos bidimensionales, recorrido y totales por fila y columna

Capacidad	Utiliza estructuras de datos estáticas (vectores y matrices) para el planeamiento y la solución de problemas de la vida diaria (lógico, matemático, comerciales, otros.)
Tema	Matrices: arreglos bidimensionales, recorrido y totales por fila y columna
Indicadores de logro	• Declara una matriz indicando sus filas y columnas. • Recorre una matriz con dos ciclos Para anidados. • Calcula totales por fila y por columna de una matriz.

De la fila a la tabla

Un vector es una fila de casilleros. Pero muchos datos son una tabla: las unidades vendidas de cada producto en cada día forman una grilla de filas (productos) y columnas (días). Esa grilla es una matriz o arreglo bidimensional.

Declaración y doble índice

Una matriz se declara con dos dimensiones: Dimension ventas[3,4] reserva 3 filas por 4 columnas. Cada casillero se identifica con dos índices: ventas[2,3] es la fila 2, columna 3. En el copetín: fila = producto, columna = día.

Ejemplo — total por producto (por fila)

```
// Filas: 1 Chipa, 2 Empanada, 3 Mixto. Columnas: Lu, Ma, Mi, Ju
Para f <- 1 Hasta 3 Con Paso 1 Hacer
    totalFila <- 0
    Para c <- 1 Hasta 4 Con Paso 1 Hacer
        totalFila <- totalFila + ventas[f,c]
    FinPara
    Escribir "Producto ", f, " vendió ", totalFila, " unidades"
FinPara
```

Con la matriz del copetín: Chipa 123, Empanada 75, Mixto 45. Total general 243.

La matriz del copetín se ve así: cada casillero se ubica con dos índices, primero la fila y después la columna. Ubicá en la figura el casillero ventas[2,3].

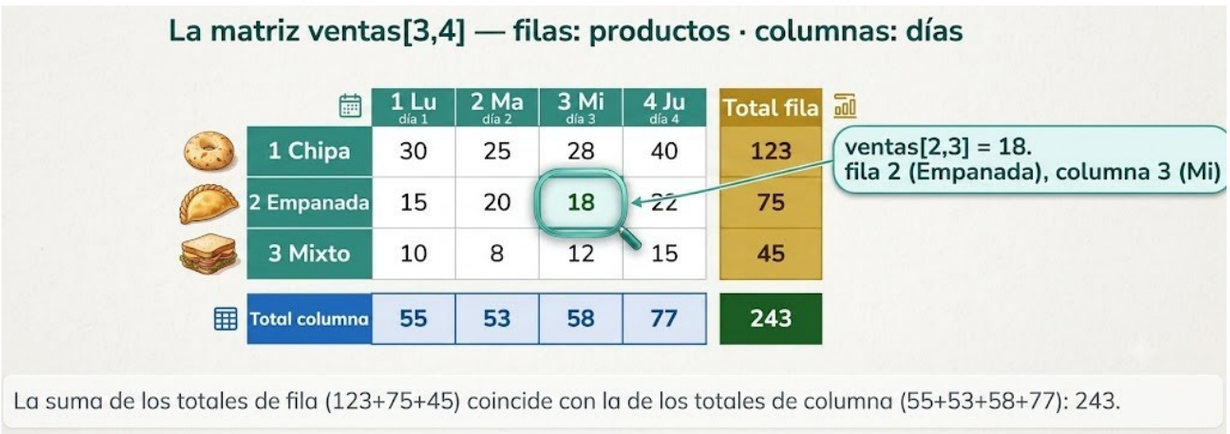


Figura 10.1 — La matriz ventas[3,4] con su doble índice y los totales por fila y por columna: ambas sumas coinciden en 243.

Recorrer con dos ciclos anidados

Para recorrer toda la matriz se usan dos Para anidados: el de afuera recorre las filas y el de adentro, las columnas. Así se visita cada casillero una sola vez.

Producto \ Día	Lu	Ma	Mi	Ju	Total fila
Chipa	30	25	28	40	123
Empanada	15	20	18	22	75
Mixto	10	8	12	15	45
Total columna	55	53	58	77	243

Totales por fila y por columna

El total por fila suma una fila completa (todo lo vendido de un producto en la semana). El total por columna suma una columna (todo lo vendido en un día). La suma de todos los totales de fila coincide con la de todos los totales de columna: es el total general.

Concepto clave

Una matriz guarda datos en filas y columnas; cada elemento se identifica con dos índices [fila, columna]. Se recorre con dos ciclos Para anidados.

Ejemplo cotidiano — la planilla de asistencia

Una planilla de asistencia (alumnos en las filas, días en las columnas) es una matriz: se suma por fila para las faltas de cada alumno y por columna para los presentes de cada día.

De unidades a guaraníes: la matriz y el vector de precios juntos

La matriz del copetín cuenta unidades vendidas, pero el dueño piensa en guaraníes. La conversión combina dos estructuras: el total por fila de la matriz (unidades de cada producto) multiplicado por el precio de ese producto en el vector paralelo de precios. El índice de fila y el índice del vector apuntan al mismo producto.

Ejemplo — facturación por producto

```
// precios paralelo a las filas: 1 Chipa 3000, 2 Empanada 5000, 3 Mixto 12000
facturacionTotal <- 0
Para f <- 1 Hasta 3 Con Paso 1 Hacer
    unidades <- 0
    Para c <- 1 Hasta 4 Con Paso 1 Hacer
        unidades <- unidades + ventas[f,c]
    FinPara
    importe <- unidades * precios[f]
    facturacionTotal <- facturacionTotal + importe
    Escribir "Producto ", f, ": ", importe
FinPara
```

Ejemplo resuelto paso a paso — la facturación de la semana corta

Con los totales por fila que ya conocés (Chipa 123, Empanada 75, Mixto 45), la facturación se arma producto por producto:

Producto (fila)	Unidades (total fila)	Precio unitario	Importe facturado
1 — Chipa	123	3.000	369.000
2 — Empanada	75	5.000	375.000
3 — Sándwich mixto	45	12.000	540.000
TOTAL	243	—	1.284.000

Fijate la sorpresa comercial que revela el cruce: la chipa vende casi el triple de unidades que el mixto (123 contra 45), pero factura menos (369.000 contra 540.000). Las unidades y los guaraníes cuentan historias distintas — por eso los sistemas de gestión calculan las dos.

Concepto clave

Una matriz de cantidades y un vector paralelo de precios se combinan por el índice de fila: importe de la fila f = (total de la fila f) × precios[f]. El doble control del total general (243 unidades; G. 1.284.000) valida todo el recorrido.

Recorrer por columnas primero: otro camino, el mismo total

Los dos ciclos anidados pueden invertirse: columna afuera, fila adentro. El recorrido visita los mismos 12 casilleros en otro orden — primero todo el lunes, después todo el martes — y el total general, por supuesto, sigue dando 243.

Orden de los ciclos	Cómo visita la matriz	Sirve para
fila afuera, columna adentro	producto por producto	totales por producto (por fila)
columna afuera, fila adentro	día por día	totales por día (por columna)

La elección del orden no cambia el resultado global, pero sí qué subtotal resulta cómodo calcular: el total de la variable interior se reinicia en cada vuelta del ciclo exterior. Regla práctica: el ciclo de afuera define «de qué» son los subtotales.

La matriz crece sin tocar el algoritmo

La matriz 3×4 de la clase es un modelo a escala: el mismo algoritmo de recorrido sirve para cualquier tamaño con solo cambiar los límites de los dos Para. La estructura no se reescribe — se redimensiona:

Matriz	Qué modela	Casilleros
ventas[3,4]	3 productos × 4 días (la semana corta)	12
ventas[7,7]	los 7 productos × la semana completa	49
ventas[7,30]	los 7 productos × un mes entero	210

Esa independencia entre el algoritmo y el tamaño de los datos es una idea central de la programación: los ciclos parametrizados escalan solos. El sistema que hoy procesa la semana corta del copetín procesará el mes completo cambiando dos números — no doscientas líneas.

Actividades de aplicación

Matriz bebidas[3,4]: unidades vendidas de lunes a jueves

Bebida	Lu	Ma	Mi	Ju
Cocido	14	9	11	16
Gaseosa	22	18	25	31
Jugo	7	10	6	12

Ejercitá:

1. Declará una matriz stock de 3 filas y 4 columnas.
2. Indicá el valor de bebidas[3,2] en la matriz de datos.
3. Indicá el total de la fila de la Gaseosa.
4. Indicá el total de la columna del jueves.
5. Escribí el encabezado de los dos ciclos anidados para recorrer una matriz de 3×4.

Resolvé (caso Copetín Karumbé):

6. Escribí un algoritmo que recorra la matriz bebidas[3,4] y muestre el total vendido por cada bebida (por fila).
7. Escribí un algoritmo que calcule el total de cada día (por columna) de la matriz bebidas[3,4].
8. Calculá a mano el total general de la matriz de bebidas sumando primero por filas y verificá que coincida con la suma por columnas.

Pensá y decidí (justificá tu respuesta):

9. V o F: en ventas[2,3] el primer índice es la columna. Justificá.
10. Elegí: para recorrer una matriz completa se usan (uno / dos) ciclos anidados. Justificá.
11. V o F: la suma de los totales de fila es igual a la suma de los totales de columna. Justificá.

Desafío:

12. Escribí un algoritmo que recorra la matriz bebidas[3,4] y muestre el total por bebida, el total por día y el total general. Con los precios Cocido 6.000, Gaseosa 8.000 y Jugo 7.000 en un vector paralelo, calculá además la facturación total.

Práctica 10 — La matriz de ventas de la librería

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Cargar y recorrer matrices con ciclos anidados, calcular totales por fila, por columna y general, y facturación con un vector paralelo.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

En v[f,c], f es la fila y c la columna. Para recorrer una matriz completa se anidan dos Para.

La suma de los totales de fila y la suma de los totales de columna tienen que dar el mismo total general: es un control automático.

Unidades vendidas de lunes a viernes (filas: Cuaderno, Bolígrafo, Carpeta). Cargá la matriz con los datos de la tabla al final de la práctica.

Actividad 1 — Cargar la matriz y sumar por fila

Objetivo: *Cargar v[3,5] fila por fila y mostrar el total de cada artículo.*

Modelo — Esqueleto para completar

```
Dimension v[3,5]
Para f <- 1 Hasta 3 Con Paso 1 Hacer
    Para c <- 1 Hasta 5 Con Paso 1 Hacer
        Leer v[f,c]
    FinPara
FinPara
Para f <- 1 Hasta 3 Con Paso 1 Hacer
    totalFila <- 0
    …
    Escribir "Fila ", f, ": ", totalFila
FinPara
```

Pasos:

1. Calculá a mano el total de cada fila con la tabla de datos.
2. Completá el esqueleto en el algoritmo MatrizVentas y ejecutá cargando los 15 datos fila por fila.
3. Explicá por qué totalFila vuelve a 0 en cada fila.

Punto de control: Los tres totales de fila coinciden con tu cálculo a mano.

Actividad 2 — Totales por día y el día con más unidades

Objetivo: *Recorrer la matriz por columnas, mostrar el total de cada día y detectar el de más unidades.*

Modelo — Qué tiene que informar

El total de cada uno de los cinco días.

El total general.

El día con más unidades y cuántas fueron.

Pasos:

1. Agregá al algoritmo un recorrido con el Para de columnas afuera y el de filas adentro.
2. Acumulá el total general mientras recorrés las filas.
3. Ejecutá y compará.

Punto de control: La suma de los cinco totales por día es igual a la suma de los tres totales por fila.

Actividad 3 — La facturación con un vector paralelo

Objetivo: *Calcular cuánto facturó cada artículo y la facturación total, con precio[3] paralelo a las filas.*

Modelo — Precios

precio[1] = 8.000 (Cuaderno) · precio[2] = 3.000 (Bolígrafo) · precio[3] = 15.000 (Carpeta)

Pasos:

1. Multiplicá cada total de fila por su precio.
2. Acumulá la facturación total.
3. Ejecutá y verificá cada producto con la calculadora.

Punto de control: La facturación total es igual a la suma de las tres facturaciones por artículo que calculaste.

Transferencia y revisión entre pares (30 a 40 minutos)

Objetivo: *aplicar lo mismo a un segundo caso, con datos diferentes, y mejorar el trabajo con la revisión de otro equipo.*

Modelo — Mini-caso: la cantina del colegio

Matriz m[2,3]: filas Empanada y Jugo; columnas Recreo 1, Recreo 2, Salida.

Empanada: 40 · 35 · 52 · Jugo: 18 · 26 · 21.

Pasos:

1. Adaptá MatrizVentas a 2 filas y 3 columnas: total por fila, por columna y general.
2. Intercambiá con otro equipo: revisá los límites de los Para y el reinicio de los totales.
3. Corregí y justificá la versión final.

Punto de control: En la versión final, la suma de los totales de fila es igual a la suma de los totales de columna.

Desafío final (para quienes terminan antes)

Agregá el promedio diario de cada artículo como Real y mostrá cuál artículo tiene el promedio más alto.

Datos para cargar

Matriz v[3,5] — unidades vendidas de lunes a viernes

Artículo	Lu	Ma	Mi	Ju	Vi
Cuaderno	8	5	7	10	6
Bolígrafo	25	30	18	22	27
Carpeta	2	4	1	3	5

Clase 11 — Ordenación de vectores: método burbuja optimizada y método de selección

Capacidad	Utiliza estructuras de datos estáticas (vectores y matrices) para el planeamiento y la solución de problemas de la vida diaria (lógico, matemático, comerciales, otros.)
Tema	Ordenación de vectores: método burbuja optimizada y método de selección
Indicadores de logro	• Explica en qué consiste ordenar un vector de forma ascendente o descendente. • Aplica el método de la burbuja optimizada para ordenar un vector. • Aplica el método de selección e interpreta el resultado (por ejemplo, un ranking).

Ordenar para informar

El ranking de productos más vendidos del copetín necesita las unidades ordenadas de mayor a menor. Ordenar es reacomodar los elementos de un vector siguiendo un criterio: ascendente (de menor a mayor) o descendente (de mayor a menor).

El método de la burbuja

La burbuja compara elementos vecinos y los intercambia si están en el orden equivocado. Repite pasadas hasta que el vector queda ordenado; los valores más grandes "suben" como burbujas. La versión optimizada corta antes si en una pasada no hubo ningún intercambio (ya estaba ordenado).

Ejemplo — la burbuja optimizada, ranking descendente

Unidades vendidas en el mes: unid = [123, 75, 45, 200, 90, 60]. La variable lógica hubo registra si la pasada intercambió algo; si una pasada termina sin intercambios, el vector ya está ordenado y el ciclo corta.

```
i <- 1
Repetir
    hubo <- Falso
    Para j <- 1 Hasta 6 - i Con Paso 1 Hacer
        Si unid[j] < unid[j+1] Entonces   // < para orden descendente
            aux <- unid[j]
            unid[j] <- unid[j+1]
            unid[j+1] <- aux
            hubo <- Verdadero
        FinSi
    FinPara
    i <- i + 1
Hasta Que NO hubo O i > 5
```

Resultado: [200, 123, 90, 75, 60, 45]. La cuarta pasada no intercambia nada y el ciclo corta: hizo 4 pasadas en lugar de 5.

Seguí en la figura las pasadas de la burbuja sobre las unidades vendidas del copetín: en cada pasada, un valor más llega a su lugar definitivo.



Figura 11.1 — Las pasadas de la burbuja optimizada hasta obtener el ranking descendente [200, 123, 90, 75, 60, 45] del copetín.

El intercambio con variable auxiliar

Para intercambiar dos elementos hace falta una variable auxiliar: aux <- a ; a <- b ; b <- aux. Sin la auxiliar, uno de los valores se perdería.

Ranking	Producto	Unidades
1.º	Coquito	200
2.º	Chipa	123
3.º	Gaseosa	90
4.º	Empanada	75
5.º	Cocido	60
6.º	Mixto	45

El método de selección

La selección busca el menor (o el mayor) del tramo no ordenado y lo coloca en su lugar definitivo, y así avanza. Hace menos intercambios que la burbuja. Sirve igual para armar el ranking del copetín.

Concepto clave

Ordenar es reacomodar un vector de menor a mayor o de mayor a menor. La burbuja compara vecinos e intercambia; la selección busca el extremo y lo ubica. El intercambio necesita una variable auxiliar.

El método de selección, pasada por pasada

La selección trabaja distinto que la burbuja: en cada pasada busca el mayor de lo que queda sin ordenar y lo fija adelante con un único intercambio. Hace la misma cantidad de comparaciones, pero muchos menos intercambios — por eso suele preferirse cuando mover los datos es costoso.

Ejemplo — el método de selección en pseudocódigo

```
Para i <- 1 Hasta 5 Con Paso 1 Hacer
    posMayor <- i
    Para j <- i + 1 Hasta 6 Con Paso 1 Hacer
        Si unid[j] > unid[posMayor] Entonces
            posMayor <- j
        FinSi
    FinPara
    Si posMayor <> i Entonces
        aux <- unid[i]
        unid[i] <- unid[posMayor]
        unid[posMayor] <- aux
    FinSi
FinPara
```

En cada vuelta del ciclo exterior, el ciclo interior solo busca la posición del mayor; el intercambio se hace una vez, al final, y solo si hace falta.

La Figura 11.2 lo muestra en forma gráfica: las cinco pasadas del método de selección sobre el ranking mensual del copetín: en ámbar, la zona ya ordenada que crece de a una posición por pasada.



Figura 11.2 — Las cinco pasadas del método de selección sobre el ranking mensual del copetín: en ámbar, la zona ya ordenada que crece de a una posición por pasada.

Ejemplo resuelto paso a paso — el ranking del mes por selección

Las unidades vendidas en el mes son [123, 75, 45, 200, 90, 60] (Chipa, Empanada, Mixto, Coquito, Gaseosa, Cocido). Ordenalas de mayor a menor con selección, anotando qué se intercambia en cada pasada:

Pasada	Mayor de lo que queda	Intercambio	Vector resultante
1	200 (posición 4)	200 ↔ 123	[200, 75, 45, 123, 90, 60]
2	123 (posición 4)	123 ↔ 75	[200, 123, 45, 75, 90, 60]
3	90 (posición 5)	90 ↔ 45	[200, 123, 90, 75, 45, 60]
4	75 (posición 4)	ya está en su lugar	[200, 123, 90, 75, 45, 60]
5	60 (posición 6)	60 ↔ 45	[200, 123, 90, 75, 60, 45]

El ranking final [200, 123, 90, 75, 60, 45] corona al coquito como campeón de unidades. Observá la pasada 4: cuando el mayor ya está en su posición, la selección no intercambia nada — la burbuja, en el mismo caso, igual habría hecho varias comparaciones con intercambios.

Ordenar la semana: otro vector, el mismo método

El mismo algoritmo, sin cambiar una línea, ordena las ventas de la semana de mayor a menor: [1.100.000, 850.000, 580.000, 500.000, 450.000, 420.000, 300.000]. El podio queda a la vista: sábado, viernes y jueves. Esa es la gracia de los métodos de ordenación: se escriben una vez y sirven para cualquier vector del mismo tipo.

Criterio	Burbuja optimizada	Selección
Estrategia	compara vecinos e intercambia	busca el mayor y lo fija
Intercambios	muchos (uno por cada par desordenado)	a lo sumo uno por pasada
Puede cortar antes	sí: si una pasada no intercambió	no: siempre completa sus pasadas
Cuándo conviene	vectores casi ordenados	cuando mover datos es caro

¿Cuánto trabajo hizo el método?

Los métodos de ordenación también se miden. Para un vector de 6 elementos, la selección hace 5 + 4 + 3 + 2 + 1 = 15 comparaciones — igual que la burbuja sin optimizar — pero a lo sumo 5 intercambios (uno por pasada), contra los hasta 15 de la burbuja. En el ranking del mes hizo solo 4: la pasada 4 no necesitó intercambiar.

Esta contabilidad parece menor con 6 productos, pero un sistema comercial ordena miles de registros: ahí la diferencia entre intercambiar poco o mucho se siente en segundos de espera. Medir el trabajo de un algoritmo — cuántas comparaciones, cuántos movimientos — es el primer paso de una disciplina completa que vas a cruzarte si seguís en informática: el análisis de algoritmos.

Ordenar ascendente: un signo de diferencia

Todo lo que hiciste descendente se vuelve ascendente cambiando UNA comparación: donde el método buscaba el mayor (>), que busque el menor (<). El ranking del mes ordenado de menor a mayor queda [45, 60, 75, 90, 123, 200] — el mixto abre la lista y el coquito la cierra.

¿Para qué querría el copetín el orden ascendente? Para la otra pregunta comercial: qué productos venden MENOS y merecen revisarse — ¿se promocionan, se reemplazan, se retiran? El algoritmo es indiferente al sentido; el negocio, no. Elegir el orden correcto para la decisión correcta es parte del oficio.

Actividades de aplicación

Vector del ranking: unid2 = unidades vendidas de seis productos en el mes

Posición	1	2	3	4	5	6
unid2	64	140	38	92	118	75

Ejercitá:

1. Ordená de mayor a menor: [45, 200, 90].
2. Escribí las tres instrucciones del intercambio de a y b con auxiliar.
3. En el vector unid2 de la tabla de datos, ¿qué valor queda primero después de ordenarlo de forma descendente y en qué posición estaba antes de ordenar?
4. Indicá el operador de comparación para ordenar de menor a mayor con la burbuja.
5. ¿Qué optimización corta la burbuja antes de tiempo?

Resolvé (caso Copetín Karumbé):

6. Escribí un algoritmo que ordene de mayor a menor el vector unid2 con la burbuja optimizada (bandera hubo) e indicá el resultado.
7. Escribí un algoritmo que ordene unid2 de menor a mayor con el método de selección e indicá el resultado y cuántos intercambios hace.
8. Con unid2 ordenado de forma descendente, escribí las instrucciones que muestran el ranking (puesto y unidades) de los tres primeros.

Pensá y decidí (justificá tu respuesta):

9. V o F: para intercambiar dos elementos no hace falta una variable auxiliar. Justificá.
10. Elegí: para un ranking de más vendido a menos vendido se ordena de forma (ascendente / descendente).

11. V o F: la burbuja optimizada puede terminar antes si el vector ya está ordenado. Justificá.

Desafío:

12. Ordená a mano unid2 de mayor a menor con la burbuja optimizada, mostrando el vector después de cada pasada e indicando en qué pasada corta el ciclo.

Práctica 11 — El ranking de la librería: burbuja y selección

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Ordenar vectores con la burbuja optimizada y con el método de selección, moviendo juntos los vectores paralelos.
Entorno de trabajo: PSeInt.

Lo que necesitás saber

La burbuja compara vecinos e intercambia; la versión optimizada corta cuando una pasada no hizo intercambios (bandera hubo).

Si el vector tiene un paralelo (los nombres), cada intercambio se hace en los dos vectores, o el ranking queda con nombres equivocados.

Usá las unidades de la Práctica 9 (35 · 120 · 90 · 18 · 12 · 44 · 59) y los nombres de la Práctica 8.

Actividad 1 — Ranking con la burbuja optimizada

Objetivo: *Ordenar unid[7] de mayor a menor con la burbuja optimizada, intercambiando también los nombres, y contar las pasadas.*

Modelo — Esqueleto para completar

```
i <- 1
pasadas <- 0
Repetir
    hubo <- Falso
    Para j <- 1 Hasta 7 - i Con Paso 1 Hacer
        Si u[j] < u[j+1] Entonces
            // intercambiar u[j] con u[j+1]
            // intercambiar nom[j] con nom[j+1]
            hubo <- Verdadero
        FinSi
    FinPara
    pasadas <- pasadas + 1
    i <- i + 1
Hasta Que NO hubo O i > 6
```

Pasos:

1. Hacé a mano la primera pasada y anotá cómo queda el vector.
2. Completá el algoritmo RankingBurbuja con los dos intercambios (usá aux para los números y auxN para los nombres).
3. Mostrá el ranking con su puesto y la cantidad de pasadas.

Punto de control: Tu primera pasada a mano coincide con la del programa (podés mostrar el vector después de cada pasada para comprobarlo), y cada nombre del ranking conserva sus unidades de la Práctica 9.

Actividad 2 — Precios de menor a mayor con selección

Objetivo: *Ordenar los siete precios de menor a mayor con el método de selección y contar los intercambios.*

Modelo — Idea del método

En la vuelta i se busca la posición del menor entre i y 7 (posMin).
Si posMin es distinta de i, se intercambian p[i] y p[posMin] y se cuenta el intercambio.

Pasos:

1. Escribí el algoritmo PreciosSeleccion.
2. Antes de ejecutar, anticipá cuál será el primer intercambio.
3. Ejecutá con los siete precios en el orden de la Práctica 8.

Punto de control: El vector final está en orden creciente, el primer intercambio fue el que anticipaste y la cantidad de intercambios es menor que 7.

Desafío final (para quienes terminan antes)

Contá cuántas comparaciones hizo cada método con el mismo vector (agregá un contador justo antes del Si que compara). ¿Cuál comparó menos?

Clase 12 — Generación de números aleatorios y su uso

Capacidad	Utiliza estructuras de datos estáticas (vectores y matrices) para el planeamiento y la solución de problemas de la vida diaria (lógico, matemático, comerciales, otros.)
Tema	Generación de números aleatorios y su uso
Indicadores de logro	• Genera un número aleatorio dentro de un rango dado. • Aplica números aleatorios para simular o sortear en un problema del caso. • Distingue una situación que requiere azar de una que no.

El azar en un programa

A veces el programa necesita un valor imprevisible: sortear un cliente ganador, simular la demanda de un día o repartir mesas al azar. Para eso existe la generación de números aleatorios.

La función de azar

En PSeInt, Azar(X) devuelve un entero entre 0 y X−1. Para obtener un rango inclusivo [mínimo, máximo] usamos Azar(máximo−mínimo+1)+mínimo. Por ejemplo, Azar(50)+1 genera valores entre 1 y 50. Esta forma evita depender de variantes de sintaxis que no están disponibles igual en todos los perfiles.

La figura muestra cómo se arma un rango con Azar: primero se generan tantos valores como resultados posibles y después se corre todo el rango sumando el mínimo.

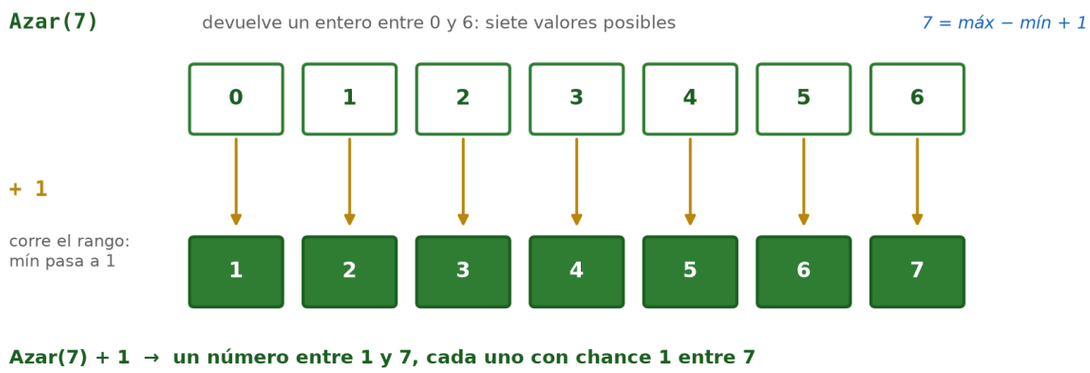


Figura 12.1 — Del Azar(7) a un número entre 1 y 7: la cantidad de valores posibles y el corrimiento del mínimo.

Ejemplo — sortear el cliente ganador del día

```
Escribir "¿Cuántos tickets se entregaron hoy?"
Leer cantidad
ganador <- Azar(cantidad) + 1
Escribir "El ticket ganador es el número ", ganador
```

Si se ingresaron 50 tickets, Azar(50) + 1 devuelve, por ejemplo, 37. Cada corrida puede dar un número distinto.

Sortear del copetín

El copetín entrega un número de ticket a cada cliente del día. Si hubo 50 tickets, el sorteo del ganador es ganador <- Azar(50) + 1. Cada ticket tiene la misma probabilidad de salir.

Situación	¿Necesita azar?	Instrucción
Sortear un ganador	Sí	Azar(cantidad) + 1
Simular una venta	Sí	Azar(900001) + 300000
Calcular un total	No	acumulador
Asignar mesa al azar	Sí	Azar(4) + 1
Ordenar un ranking	No	método de ordenación

Simular con azar dentro de un ciclo

Combinando azar y ciclos se pueden simular situaciones: por ejemplo, generar 7 ventas aleatorias entre 300.000 y 1.200.000 para probar el sistema antes de tener datos reales. Cada corrida da resultados distintos.

Concepto clave

La generación de números aleatorios devuelve un valor imprevisible dentro de un rango. Sirve para sortear y simular, no para cálculos que deben dar siempre el mismo resultado.

Simular una semana de ventas

Antes de decidir si contrata otro ayudante, el dueño quiere ensayar escenarios: ¿qué pasa si las ventas diarias se mueven entre G. 300.000 y G. 1.200.000? La función de azar genera esos escenarios sin esperar una semana real — eso es simular.

Ejemplo — simulación de cinco días

```
Algoritmo SimularSemana
    totalSim <- 0
    diasBuenos <- 0
    Para d <- 1 Hasta 5 Con Paso 1 Hacer
        venta <- Azar(900001) + 300000
        totalSim <- totalSim + venta
        Si venta > 600000 Entonces
            diasBuenos <- diasBuenos + 1
        FinSi
    FinPara
    Escribir "Total simulado: ", totalSim
    Escribir "Días sobre 600.000: ", diasBuenos
FinAlgoritmo
```

Ejemplo resuelto paso a paso — una corrida de la simulación

Supongamos que en una corrida la función Azar devuelve 350.000, 720.000, 610.000, 480.000 y 1.050.000. Trazá los dos acumuladores:

Día	venta (Azar)	¿venta > 600000?	diasBuenos	totalSim
1	350.000	Falso	0	350.000
2	720.000	Verdadero	1	1.070.000
3	610.000	Verdadero	2	1.680.000
4	480.000	Falso	2	2.160.000
5	1.050.000	Verdadero	3	3.210.000

Esta corrida cierra con G. 3.210.000 y 3 días buenos sobre 5. Si volvés a ejecutar, los números cambian — cada corrida es un escenario distinto dentro del mismo rango. Correr la simulación muchas veces y comparar resultados es la base de las proyecciones que usan los negocios reales.

Concepto clave

La computadora no genera azar verdadero: produce números pseudoaleatorios, calculados con fórmulas a partir de una semilla. Para simular ventas o sortear premios alcanzan de sobra; para seguridad informática se usan generadores especiales.

El sorteo del premio del día

El copetín sortea un producto de regalo entre los siete de su tabla. Con el vector de nombres cargado en el orden de siempre, un solo Azar(7) + 1 elige la posición ganadora:

Ejemplo — el sorteo con el vector de productos

```
Dimension nombres[7]
// carga: Chipa, Empanada, Mixto, Coquito, Gaseosa, Cocido, Sopa
ganador <- Azar(7) + 1
Escribir "Premio del día: ", nombres[ganador]
```

Si en una corrida Azar devuelve 4, el premio es el Coquito — nombres[4]. El patrón «Azar como índice de un vector» es muy frecuente: sortear un premio, elegir una pregunta al azar de un banco, repartir turnos. La única regla es que el rango del Azar coincida exactamente con las posiciones válidas del vector: de 1 a 7, ni 0 ni 8.

Repetir la simulación: la ley de los grandes números en chiquito

Una corrida de la simulación es una anécdota; muchas corridas son información. Si ejecutás la simulación de la semana diez veces y anotás los totales, vas a ver valores distintos que rondan un centro — porque el Azar(900001) + 300000 promedia 750.000 por día a la larga. Cuantas más corridas, más se estabiliza el promedio de los resultados.

Este es el uso profesional de la simulación: no predecir UN futuro, sino ver el abanico de futuros posibles y qué tan probables son. Un ciclo exterior que repita la simulación y acumule los resultados convierte tu algoritmo de la clase en una herramienta de proyección de verdad — es exactamente el desafío que te espera en las actividades.

Contar los valores posibles del Azar

¿Cuántos resultados posibles tiene un rango? La cuenta es máximo − mínimo + 1. Por eso la cantidad de resultados se usa como argumento de Azar y luego se suma el mínimo: Azar(7) + 1 elige entre 7 valores, y Azar(900001) + 300000 entre 900.001 montos posibles. En un sorteo parejo, cada valor tiene la misma chance: 1 entre 7 para cada producto del premio del día.

Llamada	Valores posibles (fin − inicio + 1)	Chance de cada valor
Azar(7) + 1	7	1 entre 7
Azar(100) + 1	100	1 entre 100
Azar(900001) + 300000	900.001	1 entre 900.001
Azar(2)	2 — funciona como moneda	1 entre 2

El último renglón esconde un truco muy usado: Azar(2) simula una moneda — 0 cruz, 1 cara — y sirve para decisiones al azar dentro de un algoritmo: ¿el cliente simulado compra o no compra? Con esa moneda y un ciclo, tu simulación de ventas puede volverse tan realista como quieras.

Actividades de aplicación

Ejercitá:

1. Escribí la instrucción que sortea un número entre 1 y 4.
2. Indicá el rango posible de Azar(50) + 1 y explicá por qué incluye 50.
3. Decidí si "calcular el total de ventas" necesita azar (sí/no).
4. Escribí la instrucción que simula una venta entre 300.000 y 1.200.000.
5. ¿Dos corridas de Azar(50) + 1 dan siempre el mismo número?

Resolvé (caso Copetín Karumbé):

6. Escribí un algoritmo que pida cuántas mesas están habilitadas y sortee la mesa que recibe un postre gratis.
7. Escribí un algoritmo que genere 7 ventas aleatorias entre 300.000 y 1.200.000, las guarde en un vector y muestre el total.
8. Escribí un algoritmo que asigne al azar una mesa (1 a 4) a cada uno de 3 clientes.

Pensá y decidí (justificá tu respuesta):

9. V o F: el azar sirve para calcular un total exacto de ventas. Justificá.
10. Elegí: Azar(4) + 1 puede devolver (0 / 3). Justificá indicando su rango completo.
11. V o F: dos corridas de un sorteo aleatorio pueden dar resultados distintos. Justificá.

Desafío:

12. Escribí un algoritmo que simule un día del copetín: genere ventas aleatorias por hora (de 8 a 12, cinco horas) entre 50.000 y 300.000, las acumule y muestre el total simulado del día.

Práctica 12 — Sorteos y simulaciones con Azar

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Generar valores aleatorios en un rango, usarlos en sorteos y simulaciones, y controlar los resultados con invariantes que no dependen del azar.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

Azar(n) devuelve un entero entre 0 y n − 1. Para un rango de mín a máx se usa Azar(máx − mín + 1) + mín.

Como el resultado cambia en cada ejecución, se controla con propiedades que siempre se cumplen: el rango, las sumas, las cantidades.

La librería sortea un cuaderno por curso y quiere simular cuántas unidades compra cada cliente (de 1 a 6).

Actividad 1 — El sorteo del cuaderno

Objetivo: *Sortear cinco veces un número de lista entre 1 y la cantidad de estudiantes del curso.*

Modelo — Esqueleto para completar

```
Leer estudiantes
Para k <- 1 Hasta 5 Con Paso 1 Hacer
    ganador <- …
    Escribir "Sorteo ", k, ": número de lista ", ganador
FinPara
```

Pasos:

1. Escribí la expresión con Azar para un rango de 1 a estudiantes.
2. Ejecutá tres veces con la cantidad de estudiantes de tu curso.
3. Anotá los 15 números obtenidos.

Punto de control: Ninguno de los 15 números es 0 ni supera la cantidad de estudiantes, y las tres ejecuciones no dieron la misma secuencia.

Actividad 2 — Simular 20 clientes

Objetivo: *Simular las unidades que compran 20 clientes y contar las frecuencias en un vector.*

Modelo — Idea

frec[6] arranca en 0.

Para cada cliente: u <- Azar(6) + 1 ; frec[u] <- frec[u] + 1.

Al final, mostrar frec y la suma de frec.

Pasos:

1. Escribí el algoritmo SimulacionCompras.
2. Ejecutalo dos veces y copiá las dos tablas de frecuencias.
3. Compará las dos tablas: ¿qué cambia y qué se mantiene?

Punto de control: En las dos ejecuciones la suma de las frecuencias es exactamente 20, aunque las frecuencias cambien.

Desafío final (para quienes terminan antes)

Llevá la simulación a 600 clientes y observá las frecuencias. ¿Alrededor de qué valor se ubican? Explicá por qué.

Clase 13 — Modularización: subprogramas, procedimientos y funciones; variables locales y globales

Capacidad	Aplica técnicas y procedimientos adecuados de modularización para simplificar problemas complejos.
Tema	Modularización: subprogramas, procedimientos y funciones; variables locales y globales
Indicadores de logro	• Distingue un procedimiento de una función según si devuelve o no un valor. • Define un subprograma y lo invoca desde el algoritmo principal. • Diferencia una variable local de una global según su alcance.

Dividir para simplificar

Un sistema grande se entiende mejor si se parte en piezas pequeñas, cada una con una tarea. Esas piezas son los subprogramas o módulos. En el copetín: un módulo calcula el total de una línea, otro aplica el descuento, otro imprime el ticket.

Procedimientos y funciones

Un procedimiento hace una tarea (por ejemplo, imprimir un ticket) pero no devuelve un valor. En PSeInt, una función declara una variable de retorno en la cabecera (por ejemplo, Funcion total <- CalcularTotal(...)) y asigna allí el resultado. La regla simple: si el resultado se usa en un cálculo posterior, es función; si solo produce una salida o un efecto, es procedimiento.

Ejemplo — función que calcula el total de una línea

```
Funcion tot <- CalcularTotal(precio, cantidad)
    tot <- precio * cantidad
FinFuncion
// En el algoritmo principal:
importe <- CalcularTotal(12000, 3)
Escribir "Total: ", importe
```

CalcularTotal(12000, 3) devuelve 36.000. La variable tot es local a la función.

Definir e invocar

El subprograma se define una vez y se invoca (se llama) cada vez que se necesita. Definir CalcularTotal en un lugar y llamarlo con distintos datos evita repetir código y concentra los cambios en un solo sitio.

Aspecto	Procedimiento	Función
¿Devuelve valor?	No	Sí (variable de retorno en la cabecera)
Se usa para	una acción o salida	un cálculo que se reutiliza
Ejemplo copetín	ImprimirTicket	CalcularTotal
Se invoca	como una instrucción	dentro de una expresión

Variables locales y globales

Una variable local vive solo dentro del subprograma donde se declara: afuera no existe. Una variable global se conoce en todo el algoritmo. Conviene usar locales siempre que se pueda, porque evitan que un módulo pise por error un dato de otro.

En PSeInt no se pueden declarar variables globales: cada SubProceso y cada Funcion tiene sus propias variables, y el algoritmo principal también. Si un módulo usa una variable con el mismo nombre que una del principal, son dos variables distintas. Por eso, en PSeInt, los datos entran siempre por parámetros y salen por el valor que devuelve una función (o por un parámetro por referencia, que vas a ver en la Clase 14). Lenguajes como Python, C o Pascal sí permiten variables globales, con los riesgos que describe esta clase.

Ejemplo — dos variables con el mismo nombre

```
SubProceso Cambiar
    total <- 99
    Escribir "dentro: ", total
FinSubProceso

Algoritmo Principal
    total <- 5
    Cambiar
    Escribir "fuera: ", total
FinAlgoritmo
```

En PSeInt se ve «dentro: 99» y después «fuera: 5»: el total del módulo es local y no toca el del principal.

Concepto clave

Modularizar es dividir un problema en subprogramas. Un procedimiento ejecuta una tarea sin devolver valor; una función devuelve un valor. Las variables locales viven solo dentro de su módulo.

El sistema de caja armado con módulos

El cobro completo del copetín junta tres tareas bien distintas: calcular el importe, aplicar el descuento y emitir el ticket. Un buen diseño le da a cada tarea su propio módulo, y deja al programa principal el papel de director de orquesta: invocar a cada uno en orden y pasarles los datos.

La Figura 13.1 lo muestra en forma gráfica: los tres módulos del sistema de caja y el programa principal que los invoca: cada uno resuelve una sola tarea y devuelve su resultado.



Figura 13.1 — Los tres módulos del sistema de caja y el programa principal que los invoca: cada uno resuelve una sola tarea y devuelve su resultado.

Ejemplo — el sistema de caja completo

```
Funcion total <- CalcularTotal(precio, cantidad)
    total <- precio * cantidad
FinFuncion
Funcion neto <- AplicarDescuento(monto)
    Si monto >= 50000 Entonces
        neto <- monto - monto * 0.10
    Sino
        neto <- monto
    FinSi
FinFuncion
SubProceso ImprimirTicket(total)
    Escribir "Copetín Karumbé - Total: G. ", total
FinSubProceso
Algoritmo Caja
    Leer precio, cantidad
    bruto <- CalcularTotal(precio, cantidad)
    neto <- AplicarDescuento(bruto)
    ImprimirTicket(neto)
FinAlgoritmo
```

Ejemplo resuelto paso a paso — cobrar 12 empanadas

Un cliente lleva 12 empanadas a G. 5.000. Seguí el dato de módulo en módulo:

Paso	Invocación	Entra	Sale
1	CalcularTotal(5000, 12)	precio 5.000, cantidad 12	bruto = 60.000
2	AplicarDescuento(60000)	monto 60.000 (≥ 50.000)	neto = 54.000
3	ImprimirTicket(54000)	total 54.000	muestra el ticket

El principal quedó en tres líneas legibles, y cada módulo se puede probar por separado: ¿CalcularTotal(3000, 4) devuelve 12.000? ¿AplicarDescuento(24000) devuelve 24.000 sin tocar nada? Probar de a partes es mucho más fácil que cazar un error en un algoritmo de cincuenta líneas.

Procedimiento o función: cómo elegir

Pregunta clave	Si la respuesta es sí…	Ejemplo en la caja
¿El módulo produce UN valor que otro va a usar?	Función	CalcularTotal devuelve el bruto
¿El módulo hace algo (mostrar, guardar) sin devolver?	Procedimiento (SubProceso)	ImprimirTicket muestra el ticket
¿Decide entre caminos pero entrega un resultado?	Función igual	AplicarDescuento devuelve el neto

Variables locales: cada módulo con su mundo

La variable total de CalcularTotal y la variable total que pudiera tener el principal son personas distintas con el mismo nombre: cada módulo crea sus variables locales al ser invocado y las destruye al terminar. Por eso los módulos no se pisan entre sí — y por eso los datos viajan por parámetros, no por telepatía.

La contracara es la variable global, visible desde todos los módulos. Tienta, porque evita pasar parámetros; y justamente por eso es peligrosa: cualquier módulo puede modificarla y el error aparece lejos de su causa. La regla profesional del libro: parámetros siempre, globales solo cuando el dato es de verdad de todo el programa (una constante como el umbral de descuento).

> **Aplicación profesional — un módulo por tarea**
> Los sistemas de venta se construyen así: un módulo de caja, uno de stock, uno de informes. Cuando el negocio pide un cambio (otra promo, otro medio de pago), el programador toca UN módulo sin desarmar el resto.

Actividades de aplicación

Ejercitá:

1. Indicá si "imprimir un ticket" es procedimiento o función.
2. Indicá si "calcular el importe de una venta" es procedimiento o función.
3. Escribí la invocación de CalcularTotal para 5 mixtos a 12.000.
4. Indicá el resultado de CalcularTotal(5000, 8).
5. Definí en una frase qué es una variable local.

Resolvé (caso Copetín Karumbé):

6. Usá la función CalcularTotal(precio, cantidad) de la clase para calcular el importe de 7 empanadas a 5.000.
7. Definí un procedimiento ImprimirTicket(producto, importe) que muestre el detalle de una venta e invocalo con "Mixto" y 36.000.
8. Definí una función que reciba un monto y devuelva el total con 10 % de descuento si llega a 50.000; probala con 90.000 y con 40.000.

Pensá y decidí (justificá tu respuesta):

9. V o F: una función siempre devuelve un valor. Justificá.
10. Elegí: una variable declarada dentro de un módulo es (local / global). Justificá.
11. V o F: modularizar obliga a repetir el mismo código varias veces. Justificá.

Desafío:

12. Escribí un mini-sistema con una función CalcularTotal, una función AplicarDescuento y un procedimiento ImprimirTicket, y usalos para cobrar 6 sándwiches mixtos a 12.000. Indicá el bruto y el total final.

Práctica 13 — La caja de la librería en módulos

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Dividir un programa en funciones y procedimientos con parámetros, reutilizarlos y comprobar el alcance local de las variables.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

Una Funcion devuelve un valor que se usa en una expresión; un SubProceso realiza una acción y no devuelve valor.

En PSeInt no hay variables globales: cada módulo tiene sus propias variables; los datos entran por parámetros.

Los módulos se escriben antes del Algoritmo principal, en el mismo archivo. Un SubProceso sin parámetros se escribe sin paréntesis.

Actividad 1 — Funciones y procedimiento de la caja

Objetivo: *Programar CalcularImporte, AplicarDescuento (10 % desde 100.000) y MostrarTicket, y usarlos desde el principal.*

Modelo — Esqueleto para completar

```
Funcion importe <- CalcularImporte(precio, cantidad)
    importe <- …
FinFuncion

Funcion neto <- AplicarDescuento(monto)
    …
FinFuncion

SubProceso MostrarTicket(articulo, total)
    Escribir "Librería Arandu | ", articulo, " | Total: ", total
FinSubProceso

Algoritmo CajaArandu
    …
FinAlgoritmo
```

Pasos:

1. Completá los módulos y el principal: leer artículo, precio y cantidad; calcular el bruto; aplicar el descuento; mostrar el ticket.
2. Anticipá el ticket para 8 carpetas a 15.000 y para 6 reglas a 4.000.
3. Ejecutá los dos casos.

Punto de control: Los dos tickets coinciden con lo anticipado y solo uno de los dos tuvo descuento.

Actividad 2 — Reutilizar y comprobar el alcance

Objetivo: *Invocar la misma función varias veces y comprobar que una variable local no modifica la del principal.*

Modelo — Dos experimentos

```
// 1) Un pedido de tres líneas con la misma función:
total <- CalcularImporte(8000, 5) + CalcularImporte(3000, 10) + CalcularImporte(1500, 4)
// 2) Alcance: el módulo y el principal usan una variable llamada tickets
SubProceso SumarTres
    Definir tickets Como Entero
    tickets <- 3
    Escribir "En el módulo: ", tickets
FinSubProceso
```

Pasos:

1. Calculá a mano el total del pedido de tres líneas y ejecutalo en un algoritmo TresLineas.
2. Escribí el algoritmo Alcance: tickets <- 10, llamá a SumarTres y después mostrá tickets.
3. Antes de ejecutar, escribí qué creés que va a mostrar el principal.

Punto de control: El total del pedido coincide con tu cálculo, y lo que muestra el principal confirma o corrige tu hipótesis escrita.

Desafío final (para quienes terminan antes)

Agregá una función CantidadParaMeta(precio, meta) que devuelva cuántas unidades hay que vender para llegar a una meta (usá trunc y sumá 1 si no es exacta). Probala con precio 6.000 y meta 100.000.

Clase 14 — Parámetros por valor y por referencia; datos de tipo cadena

Capacidades	• Aplica técnicas y procedimientos adecuados de modularización para simplificar problemas complejos. • Identifica las funciones utilizadas con datos de tipo cadena.
Tema	Parámetros por valor y por referencia; datos de tipo cadena
Indicadores de logro	• Distingue el pasaje de parámetros por valor del pasaje por referencia. • Usa funciones de cadena: longitud, mayúsculas y subcadena. • Manipula caracteres individuales de una cadena para extraer información.

Los parámetros: cómo entran los datos a un módulo

Los parámetros son los datos que un subprograma recibe para trabajar. Los parámetros formales están en la definición; los actuales son los valores concretos con que se lo invoca. En CalcularTotal(12000, 3), los formales son precio y cantidad, y los actuales son 12000 y 3.

Por valor y por referencia

En el pasaje por valor, el módulo recibe una copia: si la modifica, el original no cambia. En el pasaje por referencia, recibe el dato mismo: los cambios se reflejan afuera. Por valor se usa para datos de entrada; por referencia, cuando el módulo debe devolver un cambio en la variable original.

Ejemplo — paso por referencia, listo para ejecutar

En PSeInt, el parámetro que debe modificar la variable original se marca con Por Referencia:

```
SubProceso AplicarDescuento(monto Por Referencia)
    Si monto >= 50000 Entonces
        monto <- monto - monto * 0.10
    FinSi
FinSubProceso

Algoritmo ProbarReferencia
    Definir monto Como Real
    monto <- 60000
    AplicarDescuento(monto)
    Escribir monto
FinAlgoritmo
```

Al ejecutarlo, el programa muestra 54000. Si se borra «Por Referencia», el parámetro pasa por valor y el programa muestra 60000: el módulo trabajó sobre una copia.

Ejemplo — leer un código de producto

```
codigo <- "MIX-01"
categoria <- Subcadena(codigo, 1, 3)   // "MIX"
numero <- Subcadena(codigo, 5, 6)      // "01"
Escribir "Categoría: ", categoria
Escribir "Longitud del código: ", Longitud(codigo)   // 6
```

Para el cliente "María", Longitud("María") devuelve 5 y Mayusculas("María") devuelve "MARÍA".

Datos de tipo cadena

Una cadena es una secuencia de caracteres: un nombre de cliente, un código de producto. Las cadenas tienen funciones propias: Longitud (cuántos caracteres tiene), Mayusculas (convierte a mayúsculas), Subcadena (extrae un tramo) y la concatenación (unir dos cadenas).

Función de cadena	Qué hace	Ejemplo → resultado
Longitud(c)	cuenta los caracteres	Longitud("María") → 5
Mayusculas(c)	pasa a mayúsculas	Mayusculas("chipa") → "CHIPA"
Subcadena(c,i,f)	extrae de i a f	Subcadena("MIX-01",1,3) → "MIX"
Concatenar	une dos cadenas	"MIX" + "-01" → "MIX-01"

Manipular caracteres individuales

A cada carácter de una cadena se llega por su posición, como en un vector. En el código "MIX-01", los tres primeros caracteres ("MIX") indican la categoría y los dos últimos ("01"), el número. Con Subcadena se separan esas partes.

Concepto clave

Por valor el módulo recibe una copia (no cambia el original); por referencia recibe el dato mismo (sí lo cambia). Las cadenas se manejan con funciones como Longitud, Mayusculas y Subcadena.

En Paraguay — el RUC

El RUC se escribe como un número base, un guion y un dígito verificador (por ejemplo, 80012345-6). Separar esas dos partes es un trabajo típico de funciones de cadena: se busca la posición del guion y se extrae lo que está antes y lo que está después.

Ejemplo resuelto paso a paso — el mismo descuento, dos comportamientos

La función AplicarDescuento recibe un monto de G. 60.000. Si el parámetro va por valor, el módulo trabaja con una copia: adentro el neto queda en 54.000, pero la variable original del principal sigue valiendo 60.000. Si va por referencia, módulo y principal comparten la misma variable: al volver, el original ya vale 54.000.

Momento	Por valor	Por referencia
Antes de invocar	monto = 60.000	monto = 60.000
Dentro del módulo	trabaja con una copia: 54.000	trabaja con el original: 54.000
Después de volver	monto sigue en 60.000	monto quedó en 54.000

La Figura 14.1 lo muestra en forma gráfica: por valor, el módulo modifica una copia y el original no cambia; por referencia, módulo y principal comparten la variable y el cambio persiste.

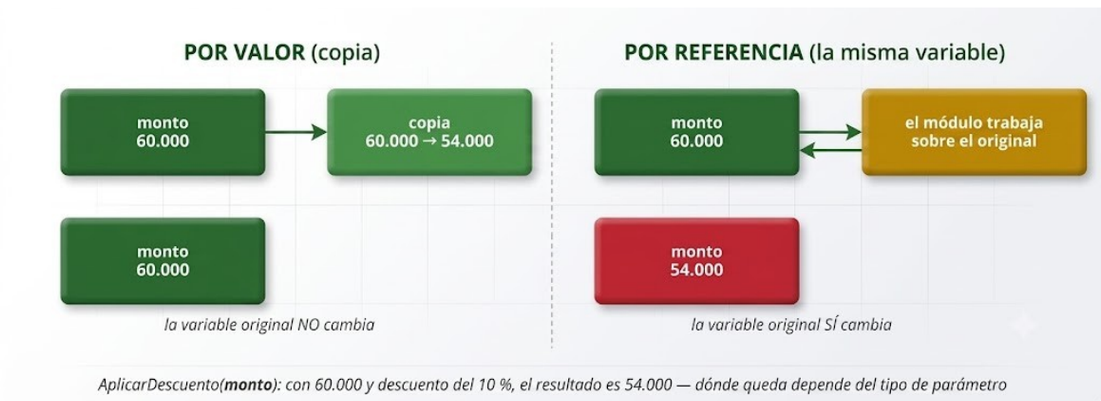


Figura 14.1 — Por valor, el módulo modifica una copia y el original no cambia; por referencia, módulo y principal comparten la variable y el cambio persiste.

¿Cuál conviene? Por valor cuando el módulo solo necesita leer el dato (más seguro: no puede romper nada afuera). Por referencia cuando el objetivo es justamente modificar la variable del que invoca. La regla profesional: por valor salvo necesidad demostrada.

Armar códigos de producto con cadenas

Los códigos del copetín — EMP-01, MIX-01, SOP-01 — son cadenas, y las funciones de cadena permiten armarlos, medirlos y desarmarlos. Fijate cada operación con datos reales del negocio:

Función	Qué hace	Con datos del copetín	Resultado
Concatenar(a, b)	une dos cadenas	Concatenar("EMP", "-01")	"EMP-01"
Longitud(c)	cuenta los caracteres	Longitud("Chipa")	5
Subcadena(c, i, f)	extrae un tramo	Subcadena("Sándwich mixto", 1, 8)	"Sándwich"
Mayusculas(c)	pasa a mayúsculas	Mayusculas("chipa")	"CHIPA"

Un uso típico: validar que todo código tenga 6 caracteres (Longitud) y extraer sus tres primeras letras para saber el producto (Subcadena). Los sistemas de venta hacen exactamente esto cada vez que el cajero escanea o teclea un código.

Concepto clave

La cadena es un dato compuesto: una secuencia de caracteres numerados desde 1 en el perfil usado en este material. Igual que en los vectores, se puede acceder a una posición — pero las funciones de cadena (Concatenar, Longitud, Subcadena, Mayusculas) resuelven las operaciones más comunes sin recorrerla a mano.

Recorrer una cadena carácter por carácter

Con el perfil base 1 usado en este material, la cadena se recorre igual que un vector: un ciclo Para desde 1 hasta su longitud, extrayendo de a un carácter con Subcadena. El patrón sirve para validar, contar o transformar texto:

Ejemplo — contar las letras a de un nombre

```
nombre <- "Sopa paraguaya"
cuenta <- 0
Para i <- 1 Hasta Longitud(nombre) Con Paso 1 Hacer
    Si Subcadena(nombre, i, i) = "a" Entonces
        cuenta <- cuenta + 1
    FinSi
FinPara
// cuenta = 5
```

Comparar cadenas: la relación entre textos

Las cadenas también se comparan con los operadores relacionales. El igual y el distinto preguntan si dos textos son idénticos, letra por letra. El menor y el mayor ordenan alfabéticamente, pero según el código de cada carácter: en PSeInt, como en casi todos los lenguajes, las mayúsculas van antes que las minúsculas. Por eso conviene comparar después de pasar los dos textos a mayúsculas.

Comparación	Resultado en PSeInt	Por qué
"chipa" < "empanada"	Verdadero	c va antes que e en el alfabeto
"Chipa" = "chipa"	Falso	la C mayúscula es otro carácter
Mayusculas("Chipa") = Mayusculas("chipa")	Verdadero	comparadas en mayúsculas, son iguales
"Zapallo" < "anana"	Verdadero	las mayúsculas van antes que todas las minúsculas
"10" < "9"	Verdadero	como texto se compara carácter por carácter: 1 va antes que 9

Errores frecuentes — al comparar cadenas

1. Comparar sin unificar mayúsculas: el cliente escribe «chipa» y el sistema busca «Chipa». La solución es comparar Mayusculas(a) = Mayusculas(b).
2. Guardar números como texto: «10» < «9» da Verdadero. Si con el dato se calcula o se ordena por valor, tiene que ser numérico.
3. Olvidar las comillas: categoria = salado compara con una variable llamada salado, no con el texto «salado».

La palabra «Sopa paraguaya» esconde cinco letras a: una en Sopa y cuatro en paraguaya. El mismo esqueleto — ciclo hasta la longitud, condición sobre el carácter — valida un código de producto (¿tiene exactamente un guion?), cuenta los espacios de una frase o detecta un carácter prohibido en un nombre de archivo.

Actividades de aplicación

Ejercitá:

1. Indicá el resultado de Longitud("Ña Rosa").
2. Indicá el resultado de Mayusculas("chipa").
3. Indicá el resultado de Subcadena("SOP-07", 5, 6).
4. Indicá los parámetros formales de CalcularTotal(precio, cantidad).
5. Indicá qué tipo de pasaje NO modifica la variable original.

Resolvé (caso Copetín Karumbé):

6. Escribí un algoritmo que lea el código de un producto (formato XXX-NN) y muestre por separado la categoría y el número usando Subcadena. Probalo con "SOP-07".
7. Escribí un algoritmo que lea el nombre de una clienta y muestre su longitud y su versión en mayúsculas. Probalo con "Ña Rosa".
8. Explicá, con el ejemplo de un módulo que aplica un descuento, cuándo conviene pasar el monto por referencia y cuándo por valor.

Pensá y decidí (justificá tu respuesta):

9. V o F: en el pasaje por valor los cambios del módulo se reflejan en la variable original. Justificá.
10. Elegí: para extraer un tramo de una cadena se usa (Longitud / Subcadena).
11. V o F: a cada carácter de una cadena se accede por su posición. Justificá.

Desafío:

12. Escribí un algoritmo que tome el código "COC-12", extraiga la categoría y el número con Subcadena, muestre la longitud del código y arme un mensaje concatenando "Producto de categoría " con la categoría.

Práctica 14 — Parámetros por referencia y códigos de artículos

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Distinguir el pasaje por valor y por referencia y procesar cadenas (Longitud, Subcadena, Mayusculas, Concatenar y comparación) en códigos y nombres.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

Por valor, el módulo recibe una copia; por referencia (Por Referencia), trabaja sobre la variable original.

Las cadenas se comparan carácter por carácter según su código: las mayúsculas van antes que las minúsculas y "12" es menor que "9" como texto.

Los códigos de la librería tienen el formato XXX-NN: tres letras de categoría, un guion y dos dígitos (por ejemplo, ESC-03).

Actividad 1 — Valor o referencia

Objetivo: *Comprobar con un experimento qué pasa con la variable original en cada tipo de pasaje.*

Modelo — Los dos módulos

```
SubProceso RecargoValor(monto)
    monto <- monto + 5000
    Escribir "Dentro (por valor): ", monto
FinSubProceso

SubProceso RecargoRef(monto Por Referencia)
    monto <- monto + 5000
    Escribir "Dentro (por referencia): ", monto
FinSubProceso
```

Pasos:

1. Escribí el algoritmo PasajeParametros: total <- 40000; llamá a RecargoValor(total) y mostrá total; llamá a RecargoRef(total) y mostrá total.
2. Antes de ejecutar, completá una tabla con lo que esperás ver en cada una de las cuatro líneas.
3. Ejecutá y compará.

Punto de control: Las cuatro líneas coinciden con tu tabla; si alguna no coincide, explicá por qué con el tipo de pasaje.

Actividad 2 — Desarmar un código de artículo

Objetivo: *Extraer la categoría y el número de un código y armar un mensaje con Concatenar.*

Modelo — Esqueleto para completar

```
Leer cod
cat <- Subcadena(cod, …, …)
num <- Subcadena(cod, …, …)
msj <- Concatenar("Categoría ", Mayusculas(cat))
msj <- Concatenar(msj, ", número ")
msj <- Concatenar(msj, num)
Escribir msj
Escribir "Longitud del código: ", Longitud(cod)
```

Pasos:

1. Completá las posiciones de Subcadena para el formato XXX-NN.
2. Ejecutá con "cua-08" (en minúsculas) y con "GEO-12".
3. Explicá qué hizo Mayusculas en el primer caso.

Punto de control: Los dos códigos dan categoría en mayúsculas y número de dos dígitos, y la longitud informada es la misma en ambos.

Actividad 3 — Comparar nombres sin que molesten las mayúsculas

Objetivo: *Comparar cadenas directamente y en mayúsculas, y sacar conclusiones.*

Modelo — Comparaciones a probar

```
Escribir "Directo: ", a < b
Escribir "En mayúsculas: ", Mayusculas(a) < Mayusculas(b)
Escribir "¿Iguales sin importar mayúsculas? ", Mayusculas(a) = Mayusculas(b)
```

Pasos:

1. Escribí el algoritmo CompararNombres que lee a y b y muestra las tres comparaciones.
2. Anticipá y después ejecutá con: "carpeta" y "Regla"; "Lápiz" y "lápiz"; "12" y "9".
3. Escribí una regla práctica para ordenar nombres escritos por personas distintas.

Punto de control: Para cada par, al menos una de las tres comparaciones contradice el orden alfabético que esperarías al leer: tu regla práctica explica por qué.

Transferencia y revisión entre pares (30 a 40 minutos)

Objetivo: *aplicar lo mismo a un segundo caso, con datos diferentes, y mejorar el trabajo con la revisión de otro equipo.*

Modelo — Mini-caso: validar códigos

La librería recibe códigos escritos a mano: ESC-03, ESC03, ES-003, GEO-1.

Hay que decidir cuáles tienen el formato XXX-NN.

Pasos:

1. Escribí una condición con Longitud y Subcadena que acepte solo el formato XXX-NN y probala con los cuatro códigos.
2. Intercambiá con otro equipo y buscá un código inválido que tu condición acepte por error.
3. Mejorá la condición o justificá por qué es suficiente para la librería.

Punto de control: Los cuatro códigos se clasifican igual en las versiones de los dos equipos, y la justificación indica qué casos quedan fuera del control.

Desafío final (para quienes terminan antes)

Escribí una función lógica CodigoValido(cod) que devuelva Verdadero si el código tiene 6 caracteres y un guion en la posición 4. Probala con ESC-03, ESC03, ES-003 y GEO-1.

Evaluación de la Unidad 2

Estructuras de datos y modularización (Clases 8 a 14)

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Datos de la evaluación — la heladería Yvoty

Litros vendidos por sabor: litros = [14, 22, 9, 31, 18] (Chocolate, Frutilla, Limón, Dulce de leche, Coco).

Unidades por formato y turno: Cucurucho 20 · 35 · 28 ; Vaso 12 · 15 · 22 (mañana, tarde, noche).

Parte A — Diagnóstico rápido (marcá la opción correcta):

1. Para recorrer una matriz completa se necesitan:
   a) un ciclo Repetir
   b) ningún ciclo
   c) un solo ciclo
   d) dos ciclos anidados

2. Un subprograma que devuelve un valor mediante una variable de retorno declarada en su cabecera es:
   a) un procedimiento
   b) una función
   c) un vector
   d) una variable global

3. Para ordenar un ranking de más vendido a menos vendido se ordena de forma:
   a) descendente
   b) ascendente
   c) aleatoria
   d) no se ordena

Parte B — Resolución y aplicación:

4. Declará el vector litros de 5 elementos y escribí el ciclo que lo carga.

5. Escribí un algoritmo que recorra litros[5] y muestre el total, el promedio y el sabor más vendido con su posición. Indicá los resultados con los datos de la evaluación.

6. Con la matriz de formatos y turnos, calculá el total por formato, el total por turno y el total general.

7. Ordená de mayor a menor el vector litros con la burbuja optimizada: mostrá el vector después de cada pasada e indicá qué sabor queda primero.

8. Definí una función Importe(precio, cantidad) que devuelva el importe y calculá el de 3 potes a 25.000.

9. Para el código "HEL-14", indicá qué devuelven Subcadena(codigo, 1, 3), Subcadena(codigo, 5, 6) y Longitud(codigo).

Evaluación integradora de la 1.ª etapa

Unidades 1 y 2 · Cierre de la 1.ª etapa (junio)

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Datos de la evaluación — la fotocopiadora Ñandutí

Precio por copia: 500 guaraníes; desde 100 copias en un mismo trabajo, 400 por copia.

Copias por día (lunes a sábado): copias = [320, 450, 280, 510, 390, 270].

Copias por tipo y turno: Blanco y negro 120 · 200 · 90 ; Color 30 · 45 · 15 (mañana, tarde, noche).

Parte A — Diagnóstico rápido (marcá la opción correcta):

1. La estructura Si… Sino elige entre:
   a) un solo camino
   b) dos caminos mutuamente excluyentes
   c) tres o más caminos
   d) ningún camino

2. Un acumulador se usa para:
   a) contar ocurrencias
   b) marcar una condición
   c) sumar valores en un ciclo
   d) ordenar un vector

3. El elemento de un vector se identifica por su:
   a) tipo
   b) tamaño
   c) color
   d) índice

Parte B — Resolución y aplicación:

4. Escribí un algoritmo con Si… Sino que calcule el importe de un trabajo según la cantidad de copias. Probalo con 100 y con 80 copias y explicá qué tienen de curioso los dos resultados.

5. Escribí un algoritmo con Para que cargue el vector copias[6], acumule el total y muestre el promedio. Indicá el resultado con los datos de la evaluación.

6. Encontrá el mayor elemento de copias[6] y su posición, y contá cuántos días alcanzaron el promedio.

7. Calculá el total por tipo de copia, el total por turno y el total general de la matriz de la fotocopiadora.

8. Ordená de mayor a menor el vector copias con la burbuja optimizada e indicá en qué pasada corta el ciclo.

9. Definí una función que reciba la cantidad de copias y devuelva el importe con la regla de la fotocopiadora; calculá el importe de un trabajo de 150 copias.

UNIDAD 3 — Sistema de archivos

En esta unidad vas a ver dónde viven los datos cuando el programa termina: qué es un archivo, qué operaciones se hacen con él, cómo se protege y cómo lo organiza el sistema operativo en carpetas.

Clase 15 — Archivos: concepto, características y tipos; memoria principal y secundaria

Capacidad	Analiza las utilidades de un sistema de archivos.
Tema	Archivos: concepto, características y tipos; memoria principal y secundaria
Indicadores de logro	• Define qué es un archivo y enuncia sus características (nombre, extensión, tamaño, ubicación). • Distingue los tipos de archivo por su contenido y por su organización. • Compara la memoria principal con la secundaria según velocidad y permanencia.

Guardar para no perder

Un programa que calcula el total de la semana pierde todo al cerrarse: los datos vivían solo en la memoria de trabajo. Para conservarlos hay que guardarlos en un archivo: un conjunto de datos relacionados que se almacena con un nombre en la memoria secundaria y sigue disponible después de apagar la máquina.

El Copetín Karumbé guarda el cierre de cada día en un archivo para poder consultarlo semanas después.

Características de un archivo

Todo archivo tiene un nombre y una extensión que indica su tipo (ventas_semana.txt es de texto; informe.docx, de Word). Además tiene un tamaño (en bytes) y una ubicación (la carpeta donde vive).

Ejemplo — el archivo de ventas del copetín

El copetín guarda las ventas de la semana en un archivo de texto:

Nombre: ventas_semana.txt

Contenido (una línea por día):

```
Lunes;420000
Martes;500000
...
Domingo;300000
```

Es un archivo de texto, de organización secuencial: para leer el domingo hay que recorrer los días anteriores.

Tipos de archivo

Por su contenido, los archivos son de texto (legibles, como un .txt) o binarios (una imagen, un programa). Por su organización interna, pueden ser secuenciales (los datos se leen en orden, del primero al último) o directos (se salta a un dato por su posición, sin leer los anteriores).

Aspecto	Memoria principal (RAM)	Memoria secundaria (disco)
Velocidad	muy rápida (nanosegundos)	más lenta (milisegundos)
Permanencia	volátil (se borra al apagar)	permanente
Uso típico	cálculos en curso	guardar archivos
Ejemplo	las variables del programa	ventas_semana.txt

Memoria principal y secundaria

La memoria principal (RAM) es rápida pero volátil: se borra al apagar. La memoria secundaria (disco, pendrive) es más lenta pero permanente. Por eso los cálculos ocurren en la RAM, pero los resultados que hay que conservar se escriben en un archivo de la memoria secundaria. La diferencia de velocidad es enorme: la RAM responde en nanosegundos y el disco, en milisegundos.

Concepto clave

Un archivo es un conjunto de datos relacionados guardado con un nombre en la memoria secundaria. La memoria principal (RAM) es rápida y volátil; la secundaria es más lenta pero permanente.

Un comercio necesita conservar sus comprobantes y registros durante años: por eso se guardan en la memoria secundaria (disco o nube) y no en la RAM, que se borra al apagar.

El cierre del día tiene que sobrevivir al apagado

Todo lo que el sistema de caja calcula — el total, el contador, el promedio — vive en la memoria principal, y la memoria principal se borra al apagar. Si el cierre del sábado importa el lunes, hay que escribirlo en un archivo: esa es la frontera entre un programa que calcula y un sistema que recuerda.

La Figura 15.1 lo muestra en forma gráfica: las dos memorias y el viaje de los datos: el cierre se calcula en la RAM y solo persiste si se lo escribe en un archivo del disco.

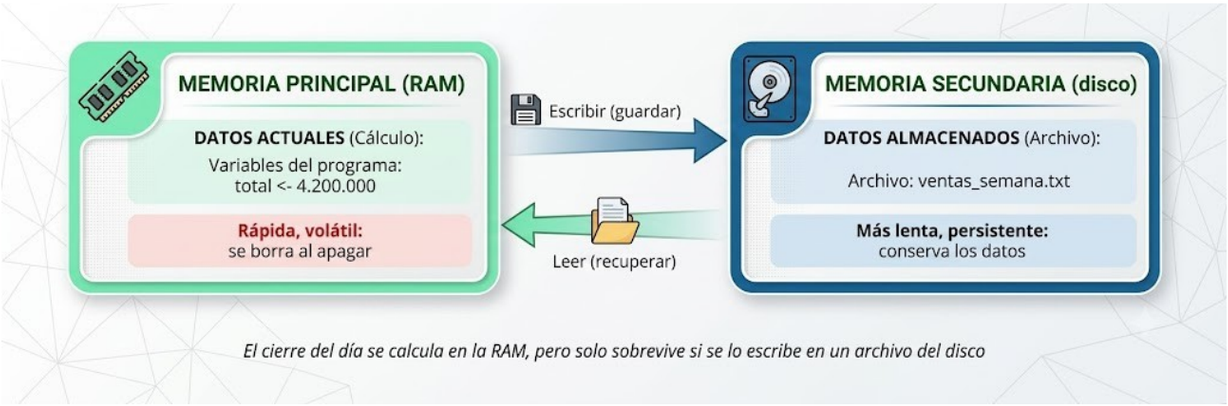


Figura 15.1 — Las dos memorias y el viaje de los datos: el cierre se calcula en la RAM y solo persiste si se lo escribe en un archivo del disco.

Ejemplo resuelto paso a paso — anatomía de ventas_semana.txt

El archivo del copetín guarda una línea por día, con el día y el monto separados por punto y coma. Es un formato de texto simple que cualquier programa (y cualquier persona) puede leer:

Ejemplo — el contenido del archivo

```
Lunes;420000
Martes;500000
Miércoles;450000
Jueves;580000
Viernes;850000
Sábado;1100000
Domingo;300000
```

Tres observaciones sobre esta anatomía. Primera: el archivo tiene organización secuencial — para llegar al domingo hay que pasar por las seis líneas anteriores, siete lecturas en total. Segunda: el separador (;) permite que un programa parta cada línea en dos campos: el día y el monto. Tercera: los montos suman G. 4.200.000, el mismo total que calculaste con el vector — el archivo es la versión persistente de esos mismos datos.

Texto o binario: qué guarda cada uno

Aspecto	Archivo de texto	Archivo binario
Contenido	caracteres legibles	bytes en el formato interno del programa
Se abre con el Bloc de notas	sí, y se entiende	se abre, pero se ve ilegible
Ejemplos	ventas_semana.txt, un .csv	una foto .jpg, una base .accdb
Ventaja	portable y fácil de revisar	compacto y rápido de procesar

Aplicación profesional — el CSV como idioma común

El intercambio de datos entre sistemas se hace muchísimo con archivos de texto separados por punto y coma o por coma (CSV): una planilla de cálculo los exporta y casi cualquier sistema los importa.

Extensiones que vas a cruzarte

La extensión — las letras después del punto — le dice al sistema operativo con qué programa abrir el archivo y a vos qué esperar adentro:

Extensión	Tipo	Qué guarda
.txt	texto	texto plano, como ventas_semana.txt
.csv	texto	tablas separadas por punto y coma o por coma (exportables a Excel)
.xlsx	binario	planillas de Excel con fórmulas y formato
.accdb	binario	bases de datos de Access (Unidad 4)
.jpg / .png	binario	imágenes
.exe	binario	programas ejecutables — ojo con los desconocidos

Cambiarle la extensión a un archivo no lo convierte: renombrar foto.jpg como foto.txt no lo vuelve texto, solo confunde al sistema operativo sobre cómo abrirlo. La conversión real la hace un programa que lea el formato de origen y escriba el de destino.

El archivo como contrato entre programas

Un archivo bien definido es un punto de encuentro: el programa que lo escribe y el que lo lee pueden ser distintos, estar en máquinas distintas y hasta escribirse en lenguajes distintos — mientras respeten el mismo formato. La línea «día;monto» de ventas_semana.txt es un contrato: quien escribe promete ese formato, quien lee confía en él.

Romper el contrato rompe la cadena: una línea sin punto y coma, un monto con puntos de miles, un día en mayúsculas donde se esperaba minúscula — y el programa lector falla o, peor, calcula mal en silencio. Por eso la definición del formato se documenta antes de escribir la primera línea, y por eso los formatos estándar (como el CSV) valen oro.

Actividades de aplicación

Ejercitá:

1. Indicá la extensión del archivo ventas_semana.txt y qué tipo de contenido tiene.
2. Clasificá por permanencia: RAM y disco.
3. Indicá qué tipo de organización (secuencial o directa) obliga a leer en orden.
4. Nombrá dos características de un archivo además del nombre.
5. Decidí dónde se guarda un dato que debe sobrevivir al apagado: (RAM / memoria secundaria).

Resolvé (caso Copetín Karumbé):

6. El copetín necesita conservar el total de cada día. Explicá por qué no alcanza con guardarlo en una variable y qué hay que usar en su lugar.
7. Describí el contenido que tendría el archivo ventas_semana2.txt con las 7 ventas de la semana 2 (una línea por día, formato Día;monto).
8. Indicá, para el archivo del copetín, su nombre, su extensión, su tipo de contenido y su tipo de organización.

Pensá y decidí (justificá tu respuesta):

9. V o F: los datos guardados en la RAM sobreviven al apagado. Justificá.
10. Elegí: para leer el último registro de un archivo secuencial hay que (recorrer los anteriores / saltar directo). Justificá.
11. V o F: la memoria secundaria es más rápida que la RAM. Justificá.

Desafío:

12. Diseñá en papel el archivo que usaría el copetín para guardar, además de las ventas, el nombre del producto más vendido de cada día. Indicá nombre, extensión, tipo y una línea de ejemplo.

Práctica 15 — Los archivos de la librería: tipos, propiedades y formato

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Crear archivos de texto, reconocer sus características (nombre, extensión, tamaño, ubicación) y diseñar el formato de un archivo de datos.

Entorno de trabajo: Explorador de archivos de Windows y Bloc de notas.

Lo que necesitás saber

Un archivo es un conjunto de datos con nombre guardado en memoria secundaria; la extensión indica qué tipo de contenido tiene.

Un archivo de texto con una línea por registro y los campos separados por punto y coma se puede abrir con el Bloc de notas y con una planilla de cálculo.

Trabajá en una carpeta tuya dentro de Documentos. Si la vista del Explorador no muestra las extensiones, activá Vista → Extensiones de nombre de archivo (o la opción equivalente de tu versión de Windows).

Actividad 1 — Crear el primer archivo de datos

Objetivo: *Crear con el Bloc de notas el archivo articulos.txt con los ocho artículos de la librería y observar sus propiedades.*

Modelo — Primeras líneas del archivo (formato código;nombre;precio;stock)

```
LIB-01;Cuaderno;8000;35
LIB-02;Bolígrafo;3000;120
…
```

Pasos:

1. Copiá las ocho líneas de la tabla de datos, una por artículo, sin líneas en blanco.
2. Guardá como articulos.txt en tu carpeta, con codificación UTF-8.
3. En el Explorador, abrí Propiedades del archivo y anotá: nombre, extensión, tipo, ubicación y tamaño.

Punto de control: El archivo tiene exactamente ocho líneas y anotaste las cinco propiedades leyéndolas del cuadro Propiedades, no de memoria.

Actividad 2 — Clasificar archivos

Objetivo: *Clasificar archivos de la computadora por extensión, tipo de contenido y forma de acceso.*

Modelo — Tabla para completar

Archivo | Extensión | Texto o binario | ¿Se puede leer con el Bloc de notas? | Acceso típico (secuencial o directo)

Pasos:

1. Buscá en tu carpeta o en Documentos cinco archivos de tipos distintos (por ejemplo .txt, .docx, .jpg, .pdf, .accdb).
2. Abrí cada uno con el Bloc de notas (Abrir con) solo para observar; cerralo sin guardar.
3. Completá la tabla.

Punto de control: Clasificaste como texto solo los archivos que el Bloc de notas muestra legibles, y no guardaste cambios en ningún archivo binario.

Actividad 3 — Diseñar el archivo de ventas

Objetivo: *Diseñar el formato del archivo donde la librería registra cada venta.*

Modelo — Decisiones de diseño

Nombre del archivo y extensión.

Campos de cada línea y su orden.

Separador.

Una línea de ejemplo.

Cuántas líneas tendría en un día con 25 ventas.

Pasos:

1. Elegí los campos mínimos: fecha, código de artículo, cantidad e importe.
2. Escribí tres líneas de ejemplo coherentes con los precios de la librería.
3. Creá el archivo con el Bloc de notas y abrilo con una planilla de cálculo para verificar que cada campo cae en su columna.

Punto de control: Al abrirlo con la planilla, cada campo de tus tres líneas cae en una columna distinta y el importe es precio × cantidad.

Desafío final (para quienes terminan antes)

Agregá un renglón de encabezado al archivo de ventas (con los nombres de los campos). ¿Qué ventaja tiene y qué tendría que tener en cuenta un programa que lo lea?

Datos para cargar

Artículos de la Librería Arandu

Código	Nombre	Precio	Categoría	Stock
LIB-01	Cuaderno	8000	papelería	35
LIB-02	Bolígrafo	3000	escritura	120
LIB-03	Lápiz	2000	escritura	90
LIB-04	Regla	4000	geometría	18
LIB-05	Carpeta	15000	papelería	12
LIB-06	Resaltador	6000	escritura	44
LIB-07	Goma	1500	escritura	59
LIB-08	Compás	12000	geometría	7

Clase 16 — Administración de archivos: operaciones básicas y relación con el sistema operativo

Capacidad	Aplica técnicas y procedimientos adecuados para el diseño de un sistema de archivos.
Tema	Administración de archivos: operaciones básicas y relación con el sistema operativo
Indicadores de logro	• Enumera las operaciones básicas sobre un archivo (crear, abrir, escribir, leer, cerrar). • Ordena las operaciones en la secuencia correcta para guardar y recuperar datos. • Explica el papel del sistema operativo en la administración de los archivos.

Las operaciones básicas

Trabajar con un archivo sigue siempre un ciclo: abrir (o crear si no existe), escribir o leer datos, y cerrar. Cerrar es tan importante como abrir: asegura que todo lo escrito quede guardado de verdad en el disco.

Escribir y leer

Al abrir un archivo se elige el modo: escritura (para grabar datos) o lectura (para recuperarlos). En escritura, cada instrucción agrega datos al archivo; en lectura, cada instrucción trae el siguiente dato. Para guardar las 7 ventas se abre en escritura, se recorre con un ciclo grabando cada línea y se cierra. Los bloques de esta unidad usan pseudocódigo conceptual para representar esas operaciones: no son instrucciones nativas de PSeInt y no deben copiarse allí como si fueran comandos ejecutables.

Concepto clave — la notación de archivos de este libro

PSeInt no trabaja con archivos, así que esta unidad usa una notación conceptual, siempre la misma:

```
Abrir "nombre.txt" Para Lectura      // o Para Escritura, o Para Agregar
Escribir Archivo dato1, ";", dato2   // graba una línea
Leer Archivo linea                   // trae la línea siguiente
FinDeArchivo                         // Verdadero cuando no quedan líneas
Cerrar Archivo
```

Es una forma de pensar el procedimiento, no un programa para copiar en PSeInt. Los lenguajes reales tienen instrucciones equivalentes con otros nombres.

Ejemplo — pseudocódigo conceptual: guardar las ventas de la semana en un archivo (no ejecutar directamente en PSeInt)

```
Abrir "ventas_semana.txt" Para Escritura
Para i <- 1 Hasta 7 Con Paso 1 Hacer
    Escribir Archivo dias[i], ";", ventas[i]
FinPara
Cerrar Archivo
```

Si no se cierra el archivo, parte de lo escrito puede perderse. Para recuperarlo después se abre en modo lectura.

El sistema operativo administra los archivos

El programa no toca el disco directamente: le pide al sistema operativo (Windows, por ejemplo) que abra, escriba o cierre. El SO lleva el registro de dónde está cada archivo, cuánto ocupa y quién puede usarlo. A esa organización se la llama sistema de archivos.

Operación	Qué hace	Cuándo se usa
Crear / Abrir	prepara el archivo	al empezar a trabajar
Escribir	graba datos	al guardar
Leer	recupera datos	al consultar
Cerrar	asegura y libera	al terminar
Renombrar/Copiar/Mover/Eliminar	administra	para ordenar

Otras operaciones de administración

Además de leer y escribir, el SO permite renombrar, copiar, mover y eliminar archivos. Son las operaciones que se hacen a diario en el Explorador de archivos para mantener ordenada la información del copetín.

Concepto clave

El trabajo con archivos sigue el ciclo abrir → escribir/leer → cerrar. El sistema operativo administra los archivos: registra su ubicación, tamaño y permisos.

El ciclo completo: abrir, trabajar, cerrar

Todo trabajo con archivos respeta el mismo protocolo de tres tiempos: abrir (pedirle el archivo al sistema operativo), trabajar (leer o escribir) y cerrar (devolverlo y confirmar que todo quedó grabado). Saltearse el cierre es la causa clásica de archivos corruptos o datos perdidos: lo que no se cerró puede no haberse grabado.

La figura resume el protocolo y los tres modos de apertura: qué pasa con lo que el archivo ya tenía en cada caso.



Modo	Instrucción	Qué hace	Contenido anterior
Lectura	`Leer Archivo`	se lee tal como está	**no cambia**
Escritura	`Escribir Archivo`	se empieza en blanco	**se pierde**
Agregar	`Escribir Archivo`	se escribe al final	**se conserva**

Figura 16.1 — Abrir, trabajar y cerrar: los tres modos de apertura y qué pasa con el contenido anterior.

Ejemplo — pseudocódigo conceptual: guardar y recuperar el cierre (no ejecutar directamente en PSeInt)

```
// Guardar el cierre del sábado
Abrir "cierres.txt" Para Agregar
Escribir Archivo "Sábado;1100000"
Cerrar Archivo
// Recuperar y sumar todos los cierres de la semana
total <- 0
Abrir "ventas_semana.txt" Para Lectura
Mientras No FinDeArchivo Hacer
    Leer Archivo linea
    total <- total + MontoDe(linea)
FinMientras
Cerrar Archivo
Escribir "Total de la semana: ", total   // 4.200.000
```

// MontoDe(linea) representa conceptualmente extraer el monto de la línea; no es una función incorporada de PSeInt.

Ejemplo resuelto paso a paso — la suma que atraviesa el archivo

La lectura es un ciclo Mientras con una condición especial: seguir hasta el fin de archivo. Trazá las primeras vueltas y el cierre:

Vuelta	Línea leída	Monto extraído	total acumulado
1	Lunes;420000	420.000	420.000
2	Martes;500000	500.000	920.000
3	Miércoles;450000	450.000	1.370.000
…	(jueves a sábado)	…	3.900.000
7	Domingo;300000	300.000	4.200.000
8	fin de archivo	corta el ciclo	4.200.000

Fijate el paralelismo con lo que ya sabés: el archivo se recorre igual que un vector, con un ciclo y un acumulador. La diferencia es la condición de corte — no hay un tamaño fijo conocido, hay una marca de fin de archivo que el sistema operativo administra.

Quién hace cada operación

Operación	La pide el programa…	…y la ejecuta
Crear / abrir / cerrar	con una instrucción	el sistema operativo
Leer / escribir	línea por línea o dato por dato	el sistema operativo, vía buffer
Renombrar / copiar / mover	o el usuario desde el Explorador	el sistema operativo
Eliminar	con confirmación, ojalá	el sistema operativo (Papelera mediante)

Agregar no es sobrescribir

El modo de apertura decide el destino de lo que ya estaba en el archivo. Abrir para escritura arranca de cero — borra el contenido anterior —, mientras que abrir para agregar escribe a continuación de lo existente. Confundirlos es el accidente más caro del trabajo con archivos:

Modo de apertura	Qué pasa con lo anterior	Uso en el copetín
Lectura	intacto: solo se consulta	sumar los cierres de la semana
Escritura	SE BORRA y se escribe de nuevo	regenerar un informe completo
Agregar	intacto: lo nuevo va al final	anotar el cierre de cada día

El cierre diario del copetín usa Agregar: cada día suma su línea a cierres.txt sin tocar las anteriores. Si por error se abriera en Escritura, el primer cierre del mes borraría todo el historial — un clásico que la prueba con un archivo de ensayo detecta antes de que duela.

FinDeArchivo: la condición que administra el sistema

La marca de fin de archivo no la escribe tu programa ni figura en el contenido: la administra el sistema operativo, que sabe exactamente dónde termina cada archivo. La función FinDeArchivo pregunta por esa marca — y por eso el ciclo de lectura es siempre un Mientras No FinDeArchivo: seguir mientras quede algo por leer.

El patrón es idéntico al del centinela de la Unidad 1, con una diferencia elegante: acá el «centinela» lo pone el sistema, no el usuario. Reconocer estos parecidos es aprender a programar de verdad: las estructuras se repiten, cambian los disfraces.

Actividades de aplicación

Ejercitá:

1. Ordená las operaciones para grabar datos en un archivo: cerrar, abrir, escribir.
2. Indicá en qué modo se abre un archivo para grabar datos.
3. Indicá qué operación asegura que lo escrito quede guardado.
4. Nombrá dos operaciones de administración que ofrece el sistema operativo.
5. ¿Quién administra la ubicación y el tamaño de los archivos?

Resolvé (caso Copetín Karumbé):

6. Escribí el pseudocódigo conceptual que abre stock.txt en escritura, graba una línea por producto (código;stock) a partir de los vectores codigos[7] y stock[7], y cierra el archivo.
7. Escribí el pseudocódigo conceptual que abre stock.txt en lectura y muestra todas las líneas guardadas, sin saber de antemano cuántas son.
8. Explicá qué puede pasar si el programa graba datos pero nunca cierra el archivo.

Pensá y decidí (justificá tu respuesta):

9. V o F: se puede leer un archivo sin abrirlo primero. Justificá.
10. Elegí: para conservar lo escrito, al final hay que (cerrar / renombrar) el archivo. Justificá.
11. V o F: el programa escribe en el disco sin intervención del sistema operativo. Justificá.

Desafío:

12. Escribí el ciclo completo del copetín con la notación de la unidad: crear el archivo del día, escribir las ventas por producto, cerrarlo, y luego volver a abrirlo en lectura para mostrar lo guardado. Indicá el orden de las operaciones.

Práctica 16 — Escribir, leer y agregar: operaciones con archivos

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Expresar las operaciones con archivos en la notación conceptual del libro, hacer su prueba de escritorio y realizar las operaciones de administración en el Explorador.

Entorno de trabajo: papel y lápiz y Explorador de archivos de Windows y Bloc de notas.

Lo que necesitás saber

La secuencia es siempre abrir (con un modo) → leer o escribir → cerrar. Escritura empieza en blanco; Agregar escribe al final; Lectura no modifica.

La notación del libro es conceptual (PSeInt no trabaja con archivos): Abrir … Para …, Escribir Archivo, Leer Archivo, FinDeArchivo, Cerrar Archivo.

Usá el archivo articulos.txt de la Práctica 15.

Actividad 1 — Grabar el stock en papel

Objetivo: *Escribir en la notación del libro el procedimiento que graba codigos[8] y stock[8] en el archivo stock.txt.*

Modelo — Esqueleto para completar

```
Abrir "stock.txt" Para …
Para i <- 1 Hasta 8 Con Paso 1 Hacer
    Escribir Archivo …
FinPara
Cerrar Archivo
```

Pasos:

1. Elegí el modo de apertura y justificá por qué no es Agregar.
2. Completá la línea que graba código y stock separados por punto y coma.
3. Escribí cómo quedan las tres primeras líneas del archivo.

Punto de control: Tu procedimiento abre una sola vez, cierra una sola vez y graba exactamente ocho líneas.

Actividad 2 — Prueba de escritorio de una lectura

Objetivo: *Seguir paso a paso la lectura de stock.txt para sumar el stock total.*

Modelo — Procedimiento a seguir

```
total <- 0
Abrir "stock.txt" Para Lectura
Mientras No FinDeArchivo Hacer
    Leer Archivo linea
    total <- total + StockDe(linea)
FinMientras
Cerrar Archivo
// StockDe(linea) representa extraer el número que está después del punto y coma
```

Pasos:

1. Armá la tabla de la prueba de escritorio: vuelta, línea leída, stock extraído, total.
2. Recorré las ocho líneas y agregá la fila en la que FinDeArchivo corta el ciclo.
3. Contrastá el total con la suma de la columna Stock de la tabla de datos.

Punto de control: Tu tabla tiene ocho vueltas con lectura más la fila de corte, y el total final es igual a la suma de la columna Stock.

Actividad 3 — Administrar archivos en el Explorador

Objetivo: *Realizar las operaciones de administración de archivos y registrar qué hizo el sistema operativo en cada una.*

Modelo — Operaciones

Copiar articulos.txt y renombrar la copia como articulos_respaldo.txt.

Crear una subcarpeta Respaldos y mover la copia allí.

Eliminar la copia y restaurarla desde la Papelera de reciclaje.

Pasos:

1. Realizá las operaciones en el orden indicado.
2. Después de cada una, anotá dónde está el archivo y qué cambió en sus propiedades (fecha de modificación, ubicación).
3. Abrí la copia restaurada y verificá su contenido.

Punto de control: Al terminar, el original sigue intacto en tu carpeta y la copia restaurada tiene las mismas ocho líneas.

Desafío final (para quienes terminan antes)

Escribí en la notación del libro el procedimiento que agrega al final de stock.txt el artículo LIB-09 (Escuadra, stock 10) sin borrar lo anterior. ¿Qué pasaría si usaras el modo Escritura?

Clase 17 — Seguridad y protección de archivos; organización, acceso y buffers

Capacidad	Identifica los mecanismos de seguridad y protección aplicados a los sistemas de archivos
Tema	Seguridad y protección de archivos; organización, acceso y buffers
Indicadores de logro	• Enumera mecanismos de seguridad de archivos (permisos, atributo de solo lectura, control de acceso y respaldo). • Distingue el acceso secuencial del acceso directo. • Explica para qué sirve un buffer en la lectura y escritura de archivos.

Proteger la información

Los datos del copetín valen: si se borran o los modifica alguien por error, se pierde el trabajo de meses. Por eso los sistemas de archivos ofrecen mecanismos de seguridad y protección.

Permisos, solo lectura, control de acceso y respaldo

Los permisos definen quién puede leer, escribir o ejecutar un archivo. El atributo de solo lectura puede ayudar a evitar sobrescrituras accidentales en aplicaciones que lo respetan, pero no sustituye los permisos de acceso ni debe considerarse una protección suficiente contra la eliminación. Para restringir el acceso se utilizan permisos y, cuando corresponde, cifrado u otros mecanismos de autenticación. El respaldo (backup) permite recuperar la información si el original se daña o se pierde.

Ejemplo — proteger el archivo de ventas

El copetín protege su archivo de cierres así:

1. Marca ventas_semana.txt como Solo lectura para reducir sobrescrituras accidentales en aplicaciones que respetan el atributo; esto no reemplaza los permisos.
2. Guarda una copia de respaldo en un pendrive cada viernes.
3. Configura permisos de la carpeta Ventas para que solo usuarios autorizados puedan modificarla.

Si el disco falla, el respaldo del pendrive permite recuperar todo.

Organización y acceso

El acceso secuencial lee los datos en orden, del primero al último; es simple pero lento para llegar al final. El acceso directo salta a un dato por su posición, sin recorrer los anteriores; es más rápido para buscar un registro puntual. La organización del archivo determina qué acceso conviene.

Mecanismo / acceso	Para qué sirve
Permisos	definir quién lee o escribe
Solo lectura	reducir sobrescrituras accidentales; no sustituye permisos
Control de acceso / cifrado	restringir el acceso según permisos y herramientas disponibles
Respaldo (backup)	recuperar si el original se daña
Acceso secuencial	leer en orden, del primero al último
Acceso directo	saltar a un registro por su posición
Buffer	acelerar lectura/escritura con memoria intermedia

El buffer

Un buffer es una zona de memoria intermedia donde se juntan los datos antes de escribirlos en el disco (o después de leerlos). Como el disco es lento, el buffer agrupa varias operaciones y las hace de una vez: acelera el trabajo. Al cerrar el archivo, el buffer se vacía y todo queda guardado.

Seguí en la figura el camino de las siete líneas del archivo: el programa lee de a una desde la RAM, pero el disco solo se toca dos veces.

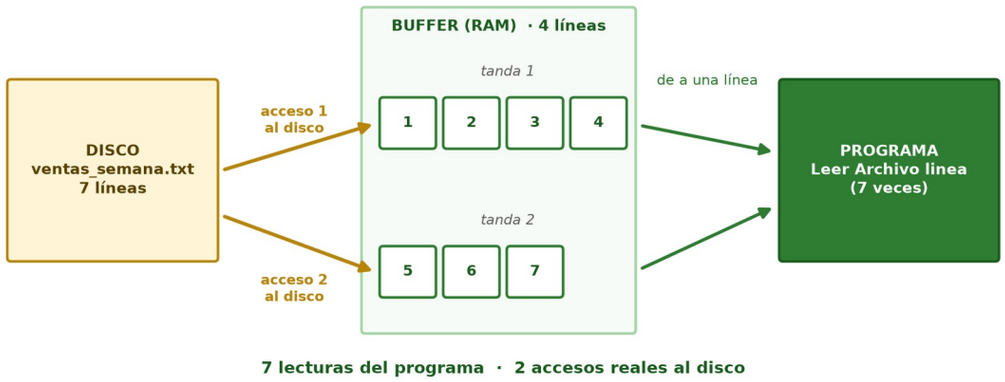


Figura 17.1 — El buffer en acción: 7 lecturas del programa, 2 accesos al disco.

Concepto clave

La seguridad de archivos combina permisos de acceso, atributos como Solo lectura, cifrado o autenticación cuando corresponda y respaldos. El atributo Solo lectura no reemplaza los permisos. El acceso puede ser secuencial (en orden) o directo (por posición). El buffer es una memoria intermedia que acelera lectura y escritura.

Aplicación profesional — datos sensibles

Las oficinas que manejan datos personales respaldan su información con copias periódicas y restringen quién puede abrir cada archivo. Cuando los datos son sensibles, además de los permisos se cifra la copia de respaldo.

La regla 3-2-1 del respaldo

Los datos del copetín valen más que la computadora que los guarda. La regla profesional del respaldo se llama 3-2-1: tres copias de los datos, en dos tipos de soporte distintos, con una copia fuera del local. Un disco que se rompe, un virus o un robo dejan de ser una catástrofe cuando la regla se cumple.

Número	Qué exige	En el copetín
3 copias	el original y dos respaldos	la PC de la caja + pendrive + nube
2 soportes	tecnologías distintas de almacenamiento	disco interno y pendrive (o nube)
1 afuera	una copia fuera del local	la carpeta en la nube del dueño

Ejemplo resuelto paso a paso — el buffer en acción

El archivo de la semana tiene 7 líneas. Leerlas de a una directo del disco costaría 7 accesos — y el disco es miles de veces más lento que la memoria. Con un buffer de 4 líneas, el sistema operativo trae los datos por tandas:

Acceso al disco	Qué trae al buffer	El programa después lee de la RAM
1.º	Lunes, Martes, Miércoles, Jueves	cuatro lecturas rapidísimas
2.º	Viernes, Sábado, Domingo	tres lecturas más y fin de archivo

Resultado: 7 lecturas del programa, pero solo 2 accesos reales al disco. El buffer es invisible para tu algoritmo — vos escribís «Leer Archivo linea» igual que siempre — pero explica por qué cerrar el archivo importa tanto: al cerrar, el sistema vacía el buffer de escritura y recién ahí los datos quedan grabados de verdad.

Concepto clave

El buffer es una zona de memoria intermedia entre el programa y el disco: agrupa lecturas y escrituras para reducir los accesos al soporte lento. Cerrar el archivo fuerza el vaciado del buffer — por eso un corte de luz antes del cierre puede perder los últimos datos.

Permisos: quién puede hacer qué

Permiso	Habilita a…	Uso típico en el copetín
Lectura	ver el contenido sin modificarlo	el ayudante consulta los precios
Escritura	modificar o agregar datos	el cajero registra las ventas
Solo lectura (atributo)	reducir sobrescrituras accidentales en aplicaciones que respetan el atributo	cierres finalizados; siempre acompañado de permisos y respaldo
Control de acceso / cifrado	restringir el acceso completo	la planilla de sueldos del dueño

Amenazas y defensas: el mapa completo

La seguridad de los archivos se piensa por pares: cada amenaza concreta tiene su defensa concreta. El mapa del copetín:

Amenaza	Defensa principal	Defensa de respaldo
Borrado accidental	Papelera de reciclaje	la copia 3-2-1
Falla del disco	respaldo en otro soporte	la copia en la nube
Virus o programa malicioso	antivirus y no ejecutar adjuntos dudosos	respaldo fuera del equipo
Corte de luz a mitad de escritura	cerrar los archivos apenas se termina	UPS (batería de respaldo)
Acceso indebido	permisos de acceso y cifrado cuando corresponda	copia cifrada fuera del local

Fijate que la copia de seguridad aparece como respaldo de casi todo: ninguna defensa es perfecta, pero un buen respaldo convierte cualquier desastre en una molestia de una hora. La pregunta de auditoría que todo negocio debería poder responder: si esta computadora desapareciera ahora mismo, ¿qué perdemos?

Ejemplo cotidiano — el UPS de la caja

Los cortes y las bajadas de tensión, sobre todo con las tormentas de verano, son un riesgo para cualquier archivo abierto. Por eso los comercios que dependen de su sistema protegen la computadora de la caja con un UPS: da tiempo a cerrar los archivos y apagar bien.

El respaldo también se prueba

Un respaldo que nunca se probó es una promesa, no una defensa. La prueba es simple y se hace en cinco minutos por mes: tomar la copia, restaurar un archivo en otra carpeta y verificar que se abre y que los datos están completos. Recién ahí el respaldo existe de verdad.

Las historias de terror de la informática comercial casi siempre empiezan igual: «teníamos respaldo, pero cuando lo necesitamos…» — el pendrive estaba dañado, la copia era de hace un año, el archivo estaba vacío. La rutina profesional junta las dos mitades: respaldar seguido y probar lo respaldado. Ninguna de las dos sirve sola.

Actividades de aplicación

Ejercitá:

1. Indicá qué atributo puede ayudar a evitar que un archivo se sobrescriba por accidente.
2. Indicá qué mecanismo permite recuperar datos si el original se daña.
3. Distinguí en una línea acceso secuencial de acceso directo.
4. Indicá para qué sirve un buffer.
5. Nombrá dos mecanismos de seguridad de archivos.

Resolvé (caso Copetín Karumbé):

6. Proponé tres medidas concretas para proteger el archivo de ventas del copetín y explicá qué evita cada una.
7. El copetín tiene un archivo con 500 cierres diarios y necesita leer solo el número 480. Indicá qué tipo de acceso conviene y por qué.
8. Explicá con el ejemplo del copetín por qué conviene cerrar el archivo después de escribir (relacionalo con el buffer).

Pensá y decidí (justificá tu respuesta):

9. V o F: el respaldo sirve para recuperar información si el original se pierde. Justificá.
10. Elegí: para llegar rápido a un registro puntual conviene el acceso (secuencial / directo). Justificá.
11. V o F: el buffer hace más lenta la escritura en disco. Justificá.

Desafío:

12. Diseñá un plan de protección para los archivos del copetín: qué permisos, qué archivos marcar como solo lectura, cada cuánto respaldar y dónde. Justificá cada decisión en una línea.

Práctica 17 — Proteger y respaldar los archivos de la librería

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Aplicar mecanismos de protección de archivos (solo lectura, respaldo) y razonar sobre acceso secuencial, acceso directo y buffer.

Entorno de trabajo: Explorador de archivos de Windows y Bloc de notas.

Lo que necesitás saber

El atributo Solo lectura reduce sobrescrituras accidentales en aplicaciones que lo respetan; no reemplaza los permisos ni el respaldo.

Un buffer junta datos en memoria para acceder al disco menos veces; cerrar el archivo vacía el buffer.

Trabajá con articulos.txt y stock.txt de las prácticas anteriores. Tené a mano un pendrive o una carpeta de la nube del colegio, si está disponible.

Actividad 1 — El atributo Solo lectura

Objetivo: *Marcar un archivo como Solo lectura y observar qué pasa al intentar guardar cambios.*

Modelo — Registro del experimento

Qué hiciste · Qué mostró la computadora · Qué conclusión sacás

Pasos:

1. En Propiedades de articulos.txt, marcá Solo lectura y aceptá.
2. Abrilo con el Bloc de notas, cambiá un precio e intentá Guardar.
3. Anotá qué pasó, cerrá sin guardar y quitá el atributo.

Punto de control: El archivo original no cambió: su contenido y su fecha de modificación son los mismos que antes del experimento.

Actividad 2 — Un plan de respaldo

Objetivo: *Diseñar y ejecutar un respaldo de los archivos de la librería.*

Modelo — El plan responde

Qué archivos se respaldan.
Cada cuánto.
Dónde (un soporte distinto del original).
Cómo se nombra cada copia.
Quién lo hace.

Pasos:

1. Escribí el plan en cinco líneas.
2. Hacé el primer respaldo: copiá los archivos a otro soporte o carpeta con el nombre que definiste (por ejemplo, con la fecha).
3. Simulá una pérdida: renombrá el original y recuperalo desde el respaldo.

Punto de control: Recuperaste el archivo desde la copia y su contenido es idéntico al original.

Actividad 3 — Acceso y buffer en números

Objetivo: *Calcular accesos al disco y comparar acceso secuencial y directo con casos de la librería.*

Modelo — Casos

(a) Un archivo de 10 líneas se lee con un buffer de 4 líneas: ¿cuántos accesos al disco?
(b) El mismo archivo sin buffer: ¿cuántos accesos?
(c) Un archivo de 300 ventas: para consultar la venta 250, ¿cuántos registros se leen con acceso secuencial y cuántos con acceso directo?

Pasos:

1. Resolvé cada caso con un dibujo de tandas o de registros.
2. Escribí una conclusión: ¿cuándo conviene cada tipo de acceso?
3. Relacioná el caso (a) con la importancia de cerrar el archivo.

Punto de control: En (a) cada línea quedó en exactamente una tanda y la cantidad de tandas es la cantidad de accesos.

Desafío final (para quienes terminan antes)

Averiguá en tu colegio o en tu casa cómo se respaldan los archivos importantes (fotos, trabajos). Compará con tu plan: ¿qué le falta a cada uno?

Clase 18 — El sistema de archivos en la estructura visible del sistema operativo

Capacidad	Reconoce la importancia del sistema de archivos dentro de la estructura visible de un sistema operativo.
Tema	El sistema de archivos en la estructura visible del sistema operativo
Indicadores de logro	• Identifica los elementos de la estructura visible: unidades, carpetas, subcarpetas y rutas. • Organiza archivos en una estructura de carpetas coherente. • Reconoce la importancia de una buena organización de archivos.

La cara visible del sistema de archivos

El sistema de archivos que administra el SO tiene una cara visible: la que se ve en el Explorador de archivos de Windows. Ahí aparecen las unidades (C:, D:, el pendrive), las carpetas y los archivos, organizados como un árbol.

Carpetas, subcarpetas y rutas

Las carpetas agrupan archivos relacionados y pueden contener subcarpetas, formando un árbol de directorios. La ruta es la dirección completa de un archivo: indica la unidad, las carpetas y el nombre, por ejemplo C:\Karumbe\Ventas\2026\ventas_semana.txt.

Organizar bien para encontrar rápido

Una buena organización ahorra tiempo y evita pérdidas. El copetín guarda sus ventas en una carpeta Ventas con una subcarpeta por año, y dentro, una por mes. Así cualquier cierre se encuentra en segundos.

Lo que el sistema operativo sabe de cada archivo

Además del nombre y la ruta, el sistema operativo guarda para cada archivo un conjunto de datos sobre el archivo, que se ven con clic derecho → Propiedades en el Explorador. No forman parte del contenido: son la ficha del archivo, y sirven para buscar, ordenar y proteger.

Propiedad	Qué indica	Ejemplo con ventas_semana.txt
Tipo	el formato, deducido de la extensión	Documento de texto (.txt)
Ubicación	la carpeta donde vive	C:\Karumbe\Ventas\2026\julio
Tamaño	cuántos bytes ocupa el contenido	unos 100 bytes: siete líneas cortas
Creado / Modificado	cuándo nació y cuándo cambió por última vez	útil para saber cuál es la versión más nueva
Atributos	marcas como Solo lectura u Oculto	Solo lectura una vez cerrada la semana

La fecha de modificación es una propiedad muy útil en el trabajo diario: ordenar una carpeta por fecha muestra arriba lo último que se tocó, y comparar fechas responde la pregunta de siempre: ¿cuál de estas dos copias es la buena? Por eso un respaldo bien hecho conserva las fechas originales de los archivos.

Buscar en el árbol

Cuando la estructura crece, recorrer carpeta por carpeta se vuelve lento. El cuadro de búsqueda del Explorador busca dentro de la carpeta abierta y de todas sus subcarpetas, y admite el comodín asterisco: *.txt encuentra todos los archivos de texto, y ventas_* todos los que empiezan con «ventas_». Es el mismo comodín que vas a usar en los filtros de la Unidad 4.

La búsqueda funciona mejor cuanto mejor son los nombres. Un archivo llamado doc1.txt solo se encuentra si alguien recuerda dónde lo dejó; uno llamado cierre_caja_2026-07-13.txt se encuentra escribiendo «cierre» o «2026-07». Organizar bien y nombrar bien son dos caras del mismo trabajo.

Elemento	Qué es	Ejemplo
Unidad	el disco o pendrive	C:
Carpeta	agrupa archivos	Ventas
Subcarpeta	carpeta dentro de otra	2026
Ruta	dirección completa	C:\Karumbe\Ventas\...
Archivo	el dato guardado	ventas_semana.txt

Por qué importa

Sin una estructura clara, los archivos se duplican, se pierden o se sobrescriben. La organización del sistema de archivos es parte del trabajo profesional: un sistema contable o de facturación depende de que cada archivo esté en su lugar.

Concepto clave

La estructura visible del sistema de archivos se ve en el Explorador: unidades, carpetas y subcarpetas forman un árbol, y la ruta es la dirección completa de un archivo. Una buena organización evita pérdidas y ahorra tiempo.

Aplicación profesional — el archivo de un estudio contable

En un estudio contable, los archivos de cada cliente se guardan en carpetas por año y por tipo de trámite. Sin esa organización, encontrar un documento presentado hace tres años llevaría horas.

El árbol de carpetas del copetín

Un sistema de gestión ordenado empieza por una estructura de carpetas pensada: una raíz para el negocio, un nivel por tema y subcarpetas por año y mes. Así, encontrar el archivo de una semana concreta no depende de la memoria de nadie — la ruta lo cuenta todo.

La Figura 18.1 lo muestra en forma gráfica: el árbol de carpetas del copetín: de la raíz C:\Karumbe a la ruta completa del archivo de ventas de julio de 2026.

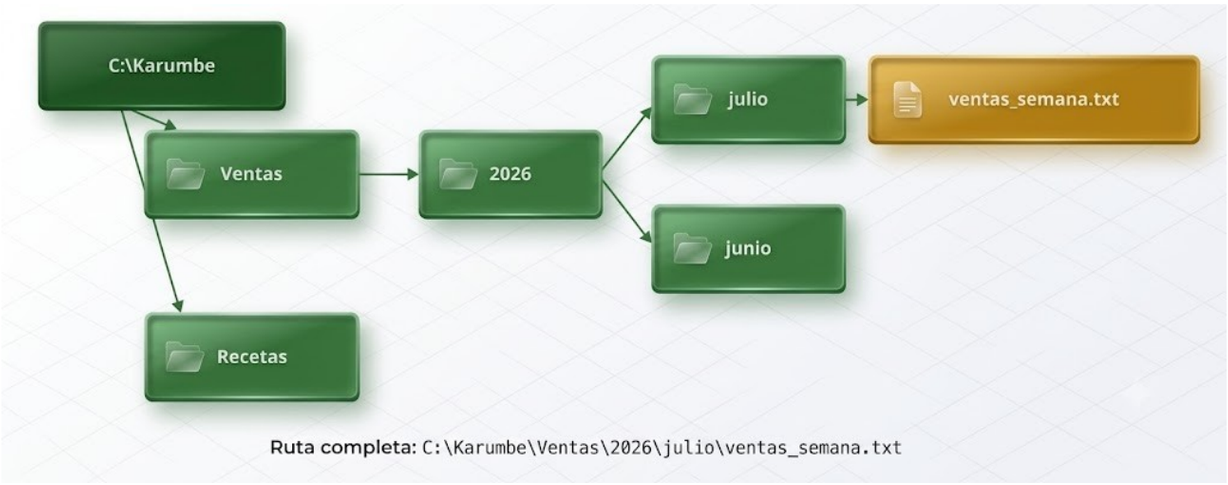


Figura 18.1 — El árbol de carpetas del copetín: de la raíz C:\Karumbe a la ruta completa del archivo de ventas de julio de 2026.

Ejemplo resuelto paso a paso — leer una ruta completa

La ruta C:\Karumbe\Ventas\2026\julio\ventas_semana.txt se lee de izquierda a derecha, nivel por nivel. Desarmala:

Tramo	Qué es	Qué indica
C:	la unidad	el disco donde vive todo
\Karumbe	carpeta raíz del negocio	todo lo del copetín cuelga de acá
\Ventas	carpeta temática	separa ventas de recetas, sueldos, etc.
\2026\julio	subcarpetas de tiempo	el año y el mes del registro
ventas_semana.txt	el archivo	nombre + extensión .txt (texto)

La ruta es también la dirección que usan tus algoritmos: la instrucción de abrir archivo necesita saber exactamente dónde está. Una carpeta renombrada o un archivo movido rompen la ruta — y el programa que la usaba deja de encontrarlo.

Convenciones de nombres que salvan tiempo

Práctica	Ejemplo malo	Ejemplo bueno
Nombres descriptivos	doc1.txt	cierre_caja_2026-07-13.txt
Fechas al estilo año-mes-día	13-7-26.txt	2026-07-13 (ordena solo)
Sin espacios raros ni símbolos	ventas $$ FINAL(2).txt	ventas_semana.txt
Una versión, no diez copias	final_FINAL_ahora_sí.txt	una carpeta \historico para lo viejo

Rutas absolutas y relativas

La ruta completa desde la unidad — C:\Karumbe\Ventas\2026\julio\ventas_semana.txt — se llama absoluta: funciona desde cualquier lugar. La relativa, en cambio, se cuenta desde la carpeta donde uno está parado: si ya estás en C:\Karumbe\Ventas, alcanza con 2026\julio\ventas_semana.txt.

Tipo	Ejemplo	Cuándo conviene
Absoluta	C:\Karumbe\Ventas\2026\julio\ventas_semana.txt	instrucciones inequívocas, accesos directos
Relativa	2026\julio\ventas_semana.txt	programas y carpetas que se mudan juntos

Los sistemas bien armados prefieren rutas relativas hacia sus propios datos: así, mover la carpeta Karumbe completa a otra máquina no rompe nada. Con rutas absolutas, cada mudanza obliga a corregir el programa — el tipo de detalle que separa un sistema profesional de uno frágil.

Actividades de aplicación

Ejercitá:

1. Indicá qué representa C: en la estructura visible.
2. Escribí la ruta de un archivo cierre.txt dentro de C:\Karumbe\Ventas.
3. Distinguí carpeta de subcarpeta con un ejemplo.
4. Nombrá la herramienta de Windows que muestra la estructura de archivos.
5. Indicá un problema que causa la mala organización de archivos.

Resolvé (caso Copetín Karumbé):

6. Diseñá el árbol de carpetas que usaría el copetín para guardar las compras de todo el año 2026, organizado por mes.
7. Escribí la ruta completa del archivo compras_semana.txt de marzo dentro de esa estructura.
8. Explicá dos ventajas concretas de organizar bien los archivos del copetín.

Pensá y decidí (justificá tu respuesta):

9. V o F: una carpeta puede contener subcarpetas. Justificá.
10. Elegí: la dirección completa de un archivo es su (ruta / extensión). Justificá.
11. V o F: la organización de archivos no afecta el trabajo profesional. Justificá.

Desafío:

12. Diseñá la estructura de carpetas completa del copetín para 2026 (ventas, compras, respaldos), con al menos dos niveles de subcarpetas, y escribí la ruta de un archivo de ejemplo dentro de ella.

Práctica 18 — El árbol de carpetas de la librería

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Crear y recorrer una estructura de carpetas, escribir rutas absolutas y relativas y buscar archivos con comodines.
Entorno de trabajo: Explorador de archivos de Windows y Bloc de notas.

Lo que necesitás saber

La ruta absoluta empieza en la unidad (C:\…); la relativa se cuenta desde la carpeta en la que uno está.

En la búsqueda del Explorador, el asterisco reemplaza cualquier cantidad de caracteres: *.txt encuentra todos los archivos de texto.

Trabajá dentro de tu carpeta de Documentos. Si en el colegio no podés crear carpetas, hacé las actividades en papel y verificalas en la primera clase con equipo disponible.

Actividad 1 — Construir el árbol

Objetivo: *Crear la estructura de carpetas de la librería para 2026.*

Modelo — Estructura a crear

Arandu
- Ventas → 2026 → marzo, abril
- Compras → 2026 → marzo
- Respaldos

Pasos:

1. Dibujá el árbol en tu carpeta antes de crearlo.
2. Creá las carpetas en el Explorador.
3. Copiá articulos.txt en Ventas\2026\marzo y stock.txt en Compras\2026\marzo.

Punto de control: El panel de navegación del Explorador muestra el mismo árbol que dibujaste, con cada archivo en su carpeta.

Actividad 2 — Rutas absolutas y relativas

Objetivo: *Escribir las rutas de los archivos y comprobarlas en la barra de direcciones.*

Modelo — Tabla para completar

Archivo · Ruta absoluta · Ruta relativa desde Arandu

Pasos:

1. Escribí la ruta absoluta y la relativa de los dos archivos copiados.
2. Pegá cada ruta absoluta en la barra de direcciones del Explorador (sin el nombre del archivo) y verificá que abra la carpeta correcta.
3. Escribí cuántos niveles hay entre la unidad y cada archivo.

Punto de control: Cada ruta absoluta abre la carpeta esperada al pegarla en la barra de direcciones.

Actividad 3 — Buscar con comodines

Objetivo: *Usar el cuadro de búsqueda con comodines y anticipar los resultados.*

Modelo — Búsquedas

*.txt dentro de Arandu
stock* dentro de Arandu
*.txt dentro de Arandu\Compras

Pasos:

1. Antes de cada búsqueda, anotá cuántos archivos esperás encontrar y cuáles.
2. Hacé las tres búsquedas.
3. Compará con lo anotado.

Punto de control: Cada búsqueda encontró exactamente los archivos que anticipaste; si no, encontraste por qué en el árbol.

Desafío final (para quienes terminan antes)

Proponé una regla de nombres para los archivos de ventas diarias (por ejemplo, con la fecha en formato año-mes-día) y explicá por qué ese formato ordena bien los archivos por nombre.

Evaluación de la Unidad 3

Sistema de archivos (Clases 15 a 18)

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Datos de la evaluación — la fotocopiadora Ñandutí

El archivo trabajos.txt tiene una línea por trabajo con el formato fecha;cliente;copias. Sus cinco líneas registran 120, 45, 300, 80 y 55 copias.

Parte A — Diagnóstico rápido (marcá la opción correcta):

1. La memoria que se borra al apagar la máquina es:
   a) el pendrive
   b) el disco
   c) la RAM (memoria principal)
   d) la memoria secundaria

2. La operación que asegura que lo escrito quede guardado en el archivo es:
   a) cerrar
   b) abrir
   c) leer
   d) renombrar

3. La dirección completa de un archivo (unidad + carpetas + nombre) es su:
   a) atributo
   b) extensión
   c) ruta
   d) tamaño

Parte B — Resolución y aplicación:

4. Compará la memoria principal y la secundaria en velocidad y permanencia, y decí dónde vive trabajos.txt.

5. Ordená y nombrá las operaciones para agregar un trabajo nuevo a trabajos.txt y luego mostrar todo el archivo.

6. Escribí en la notación del libro el procedimiento que agrega al final la línea "2026-10-05;Colegio San José;250" y justificá el modo de apertura.

7. Hacé la prueba de escritorio de un recorrido que suma las copias de trabajos.txt con Mientras No FinDeArchivo: indicá el total parcial después de cada lectura y el total final.

8. Si el sistema lee trabajos.txt con un buffer de 2 líneas, ¿cuántos accesos al disco hace? Y para consultar el trabajo 900 de un archivo de 1.000 trabajos, ¿qué acceso conviene?

9. Diseñá el árbol de carpetas de la fotocopiadora para guardar los trabajos de 2026 por mes y escribí la ruta del archivo de octubre.

UNIDAD 4 — Consultas a bases de datos

En esta unidad vas a consultar una base de datos en Microsoft Access: filtrar, seleccionar con criterios y parámetros, calcular totales y campos, y resumir en una referencia cruzada para tomar decisiones.

Clase 19 — Bases de datos y filtros: mostrar solo lo que cumple una condición

Capacidad	Identifica las características principales de los filtros y consultas.
Tema	Bases de datos y filtros: mostrar solo lo que cumple una condición
Indicadores de logro	• Distingue los conceptos de base de datos, tabla, registro y campo. • Explica qué hace un filtro y en qué se diferencia de borrar datos. • Aplica un filtro para localizar un grupo de registros que cumplen una condición.

De los archivos a las bases de datos

Un archivo de texto guarda datos, pero buscarlos es incómodo. Una base de datos los organiza en tablas, con filas y columnas, y permite consultarlos con precisión. El copetín lleva su lista de productos en una tabla.

Tabla, registro y campo

En una tabla, cada fila es un registro (un producto completo) y cada columna es un campo (una característica: código, nombre, precio, categoría, stock). La tabla Productos del copetín tiene un registro por producto y cinco campos.

Ejemplo — filtrar los productos que hay que reponer

Tabla Productos, filtro: stock < 10

Resultado (solo los registros que cumplen):

```
    MIX-01  Sándwich mixto   stock 8
    SOP-01  Sopa paraguaya   stock 6
```

Los demás productos siguen en la tabla: el filtro no los borra, solo los oculta.

La base de datos del sistema del copetín no tiene una sola tabla: el diagrama entidad-relación muestra sus dos tablas y el campo que las conecta.

La Figura 19.1 lo muestra en forma gráfica: diagrama entidad-relación de la base del copetín: Productos (1) y Ventas (N), conectadas por el campo Código.

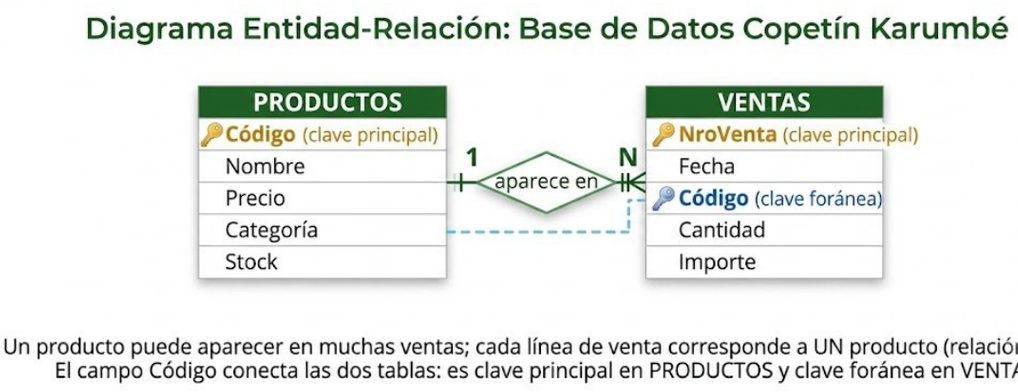


Figura 19.1 — Diagrama entidad-relación de la base del copetín: Productos (1) y Ventas (N), conectadas por el campo Código.

Qué es un filtro

Un filtro muestra solo los registros que cumplen una condición y oculta el resto, sin borrar nada: al quitar el filtro, vuelven a verse todos. Sirve para localizar rápidamente un grupo de registros, por ejemplo los productos con poco stock.

Código	Nombre	Precio	Categoría	Stock
CHI-01	Chipa	3000	salado	40
EMP-01	Empanada	5000	salado	30
MIX-01	Sándwich mixto	12000	salado	8
COQ-01	Coquito	500	dulce	200
GAS-01	Gaseosa	8000	bebida	25
COC-01	Cocido	6000	bebida	12
SOP-01	Sopa paraguaya	10000	salado	6

Filtrar en el gestor de base de datos

El gestor (el programa que administra la base) aplica el filtro con una condición sobre un campo: stock < 10 deja a la vista solo los productos que hay que reponer. La condición usa los mismos operadores relacionales de las estructuras de control.

> **Concepto clave**
> En una base de datos, cada fila es un registro y cada columna un campo. Un filtro muestra solo los registros que cumplen una condición, sin borrar los demás.

En el laboratorio vas a trabajar con Microsoft Access, el gestor de bases de datos de escritorio incluido en algunas ediciones de Microsoft Office. Las rutas de menú de este libro toman como referencia Access 2016 en castellano, versión cuyo soporte finalizó en octubre de 2025. Para las clases de 2026 se recomienda Access para Microsoft 365 o Access 2024, con licencia y soporte vigentes: los conceptos son los mismos, pero conviene comprobar las rutas de menú y el separador de las expresiones (punto y coma o coma) según la configuración regional del equipo.

El esquema muestra el recorrido completo del filtro: la tabla entra con sus 7 registros y solo salen a la vista los que cumplen la condición.

La Figura 19.2 lo muestra en forma gráfica: el filtro stock < 10 sobre la tabla Productos: solo el Sándwich mixto y la Sopa paraguaya quedan a la vista; el resto queda oculto, no borrado.



Figura 19.2 — El filtro stock < 10 sobre la tabla Productos: solo el Sándwich mixto y la Sopa paraguaya quedan a la vista; el resto queda oculto, no borrado.

Tres filtros de gestión diaria

La misma tabla Productos responde preguntas distintas según el filtro que se le aplique. Tres filtros que el copetín usa todas las semanas:

Pregunta del negocio	Filtro (criterio)	Registros que salen	Cantidad
¿Qué hay que reponer?	stock < 10	Sándwich mixto, Sopa paraguaya	2
¿Qué bebidas vendemos?	categoria = "bebida"	Gaseosa, Cocido	2
¿Cuáles son los productos caros?	precio >= 8000	Sándwich mixto, Gaseosa, Sopa	3

Ejemplo resuelto paso a paso — el filtro de reposición, registro por registro

El filtro no es magia: el gestor evalúa la condición sobre cada registro, igual que un Si dentro de un ciclo. Seguí la evaluación de stock < 10 producto por producto:

Registro	stock	¿stock < 10?	¿Se muestra?
Chipa	40	Falso	no
Empanada	30	Falso	no
Sándwich mixto	8	Verdadero	sí
Coquito	200	Falso	no
Gaseosa	25	Falso	no
Cocido	12	Falso	no
Sopa paraguaya	6	Verdadero	sí

Siete evaluaciones, dos verdaderos: la lista de reposición del lunes tiene dos renglones. Fijate que la lógica es exactamente la de la Unidad 1 — una condición que da Verdadero o Falso — aplicada ahora por el gestor de base de datos a cada registro de la tabla.

Concepto clave

Un filtro es una condición evaluada registro por registro: los que dan Verdadero se muestran, los que dan Falso se ocultan (sin borrarse). Todo lo que aprendiste sobre condiciones y operadores en la Unidad 1 se reutiliza acá tal cual.

Filtros combinados: dos condiciones a la vez

Los operadores lógicos de la Unidad 1 también trabajan en los filtros: el criterio puede exigir dos cosas (Y) o conformarse con una (O). Tres combinaciones sobre la tabla de siempre:

Filtro combinado	Registros que salen	Cantidad
categoria = "bebida" Y precio >= 8000	Gaseosa	1
categoria = "salado" O categoria = "dulce"	Chipa, Empanada, Mixto, Coquito, Sopa	5
stock < 10 Y precio >= 8000	Sándwich mixto, Sopa paraguaya	2

La lectura del primero: de las dos bebidas, solo la gaseosa llega a los 8.000. La del segundo: salados y dulces suman 4 + 1 = 5 registros. Y el tercero encuentra a los productos caros que además escasean — la lista de reposición urgente. Mismos operadores, misma lógica: solo cambió quién evalúa la condición.

Ordenar el resultado del filtro

El filtro decide QUÉ registros se ven; el orden decide EN QUÉ SECUENCIA. El gestor permite combinar ambos: filtrar los salados y ordenarlos por precio descendente muestra Sándwich mixto (12.000), Sopa (10.000), Empanada (5.000) y Chipa (3.000) — la carta del copetín de la más cara a la más barata.

¿Te suena? Es la ordenación de la Clase 11 aplicada por el gestor: por detrás, la base de datos ejecuta un método de ordenación sobre los registros filtrados. La diferencia es de comodidad, no de concepto — vos ya sabés lo que el gestor hace por vos, y eso te vuelve un usuario que entiende sus herramientas en lugar de solo apretarles los botones.

El filtro que no encuentra nada

Aplicá stock > 300 a la tabla Productos: ningún registro cumple — el stock más alto es el del coquito, con 200 — y el resultado queda vacío. Eso NO es un error: es una respuesta, y de las valiosas. «No hay productos con más de 300 unidades» es información que el dueño puede necesitar.

El error sería confundir el resultado vacío con una falla del gestor y «arreglarlo» aflojando la condición hasta que aparezca algo. Los informes se diseñan para responder la pregunta del negocio, no para devolver filas sí o sí. Un técnico que entiende esto revisa primero la pregunta — ¿el criterio expresa lo que se quería saber? — y solo después sospecha de los datos.

Actividades de aplicación

Ejercitá:

1. Indicá cuántos campos tiene la tabla Productos.
2. Indicá cuántos registros tiene la tabla Productos.
3. Escribí la condición del filtro «productos con stock menor a 15».
4. Indicá qué productos quedan visibles con el filtro stock < 15 y cuántos son.
5. V o F: un filtro borra los registros que no cumplen la condición.

Resolvé (caso Copetín Karumbé):

6. Aplicá a la tabla Productos el filtro categoria = "salado" e indicá qué registros quedan visibles.
7. El copetín quiere ver los productos que cuestan más de 5.000. Escribí la condición del filtro e indicá el resultado.
8. Explicá la diferencia entre filtrar los productos sin stock y borrarlos de la tabla.

Pensá y decidí (justificá tu respuesta):

9. V o F: al quitar un filtro se pierden los registros ocultados. Justificá.

10. Elegí: en la tabla Productos, "precio" es un (campo / registro). Justificá.
11. V o F: un filtro puede usar los operadores relacionales (>, <, =). Justificá.

Desafío:

12. Diseñá tres filtros útiles para el copetín (uno por cada campo distinto), distintos de los de la clase, escribí su condición y anticipá cuántos registros mostraría cada uno con la tabla dada.

Práctica 19 — La tabla Artículos en Access: diseño, carga y filtros

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Crear una tabla en Access con sus campos, tipos de datos y clave principal, cargar registros y aplicar filtros anticipando el resultado.

Entorno de trabajo: Microsoft Access.

Lo que necesitás saber

Cada campo tiene un tipo de dato: Texto corto para códigos y nombres, Número para cantidades y precios. Si el precio se guarda como texto, los filtros numéricos no funcionan bien.

Un filtro oculta los registros que no cumplen el criterio; no los borra. Al quitar el filtro, vuelven todos.

Abrí Access, elegí Base de datos en blanco y guardala como Arandu.accdb en tu carpeta. Las rutas de menú corresponden a Access 2016 en castellano, sin soporte desde octubre de 2025; en Access para Microsoft 365 o Access 2024 comprobá las rutas y el separador de las expresiones. Si no hay laboratorio, resolvé las actividades en papel con la tabla de datos de la práctica.

Actividad 1 — Diseñar la tabla

Objetivo: *Crear la tabla Articulos en vista Diseño con sus cinco campos, tipos y clave principal.*

Modelo — Diseño de la tabla Articulos

codigo · Texto corto · clave principal
nombre · Texto corto
precio · Número (Entero largo)
categoria · Texto corto
stock · Número (Entero largo)

Pasos:

1. Crear → Diseño de tabla. Escribí los cinco nombres de campo y elegí el tipo de dato de cada uno.
2. Seleccioná codigo y marcá Clave principal (el indicador de clave aparece a la izquierda del campo).
3. Guardá la tabla con el nombre Articulos y cerrá la vista Diseño.

Punto de control: En el panel de navegación aparece la tabla Articulos y, al reabrirla en Diseño, precio y stock figuran como Número y codigo tiene el indicador de clave.

Actividad 2 — Cargar los ocho artículos

Objetivo: *Cargar los registros en la vista Hoja de datos y comprobar la clave principal.*

Modelo — Datos

Los ocho artículos de la tabla de datos, al final de la práctica.

Pasos:

1. Abrí Articulos en vista Hoja de datos y cargá los ocho registros.
2. Intentá cargar un noveno registro con el código LIB-03 repetido y anotá qué pasa. Después borrá esa fila incompleta (Esc).
3. Contá los registros en la barra inferior de la hoja.

Punto de control: La barra de registros indica 8 y Access no aceptó el código repetido.

Actividad 3 — Filtros con anticipación

Objetivo: *Aplicar tres filtros, anticipando antes cuántos registros va a mostrar cada uno.*

Modelo — Filtros

stock < 20

categoria = "escritura"

precio >= 8000

Pasos:

1. Para cada filtro, anotá en tu carpeta cuántos registros esperás y cuáles.
2. Aplicá cada filtro desde Inicio → Ordenar y filtrar (Filtro por formulario o Filtros de número / de texto en el encabezado de la columna) y alterná con Alternar filtro.
3. Quitá el último filtro y contá los registros.

Punto de control: Cada filtro mostró los registros que anticipaste y, al quitar el filtro, la barra vuelve a indicar 8.

Desafío final (para quienes terminan antes)

Combiná dos criterios en un mismo filtro: artículos de escritura con stock menor a 60. Anticipá el resultado antes de aplicarlo.

Datos para cargar

Artículos de la Librería Arandu

Código	Nombre	Precio	Categoría	Stock
LIB-01	Cuaderno	8000	papelería	35
LIB-02	Bolígrafo	3000	escritura	120
LIB-03	Lápiz	2000	escritura	90
LIB-04	Regla	4000	geometría	18
LIB-05	Carpeta	15000	papelería	12
LIB-06	Resaltador	6000	escritura	44
LIB-07	Goma	1500	escritura	59
LIB-08	Compás	12000	geometría	7

Clase 20 — Consultas: selección, paramétrica, cálculo de totales y campos calculados

Capacidad	Aplica técnicas y procedimientos en la formulación de filtros y consultas a una base de datos, utilizando el gerenciador.
Tema	Consultas: selección, paramétrica, cálculo de totales y campos calculados
Indicadores de logro	• Formula una consulta de selección eligiendo campos y un criterio. • Distingue una consulta paramétrica de una de selección fija. • Calcula totales (conteo y suma) y define un campo calculado.

De filtrar a consultar

Un filtro oculta filas en la misma tabla. Una consulta es más potente: elige qué campos mostrar, con qué criterio y en qué orden, y puede combinar tablas o calcular. La consulta guarda la "pregunta" para repetirla cuando haga falta.

Consulta de selección y paramétrica

La consulta de selección muestra los registros que cumplen un criterio fijo (por ejemplo, categoria = "salado"). La consulta paramétrica no fija el criterio: se lo pide al usuario al ejecutarla (¿qué categoría?), así una misma consulta sirve para cualquier categoría. Una consulta es una pregunta guardada. En el esquema, seguí el recorrido: de la tabla origen, pasando por los campos, el criterio y el orden elegidos, hasta el resultado.

La Figura 20.1 lo muestra en forma gráfica: el recorrido de una consulta de selección: de la tabla Productos al resultado con los 4 productos salados, ordenados por precio.



Figura 20.1 — El recorrido de una consulta de selección: de la tabla Productos al resultado con los 4 productos salados, ordenados por precio.

Cálculo de totales

Las consultas de totales resumen: cuentan registros (¿cuántos productos hay por categoría?) o suman valores (¿cuánto stock hay en total?). En el copetín: 4 productos salados, 1 dulce y 2 bebidas; y un stock total de 321 unidades.

Tipo de consulta	Qué hace	Ejemplo en el copetín
Selección	muestra registros por criterio fijo	categoria = "salado"
Paramétrica	pide el criterio al ejecutar	¿qué categoría?
Totales (conteo)	cuenta registros	productos por categoría
Totales (suma)	suma un campo	stock total = 321
Campo calculado	nueva columna operada	valorStock = precio × stock

Campos calculados

Un campo calculado es una columna nueva que se obtiene operando otros campos. Por ejemplo, valorStock = precio × stock estima cuánto vale el inventario de cada producto. Para la Chipa: 3.000 × 40 = 120.000.

Concepto clave

Una consulta de selección usa un criterio fijo; una paramétrica lo pide al ejecutarse. Las consultas de totales cuentan o suman, y un campo calculado crea una columna nueva a partir de otros campos.

En Microsoft Access, un campo calculado se define escribiendo la fórmula en el diseño de la consulta; así un comercio obtiene el valor de su inventario sin agregar los datos a mano.

Ejemplo resuelto paso a paso — el valor del inventario completo

El campo calculado valorStock = precio × stock, aplicado a los siete productos, responde una pregunta que el dueño se hace todos los meses: ¿cuánta plata hay parada en el estante? Calculalo producto por producto y verificá el total:

Producto	Precio	Stock	valorStock
Chipa	3.000	40	120.000
Empanada	5.000	30	150.000
Sándwich mixto	12.000	8	96.000
Coquito	500	200	100.000
Gaseosa	8.000	25	200.000
Cocido	6.000	12	72.000
Sopa paraguaya	10.000	6	60.000
TOTAL	—	321	798.000

El inventario completo vale G. 798.000. Y una consulta de totales agrupada por categoría permite el doble control: salado 426.000, dulce 100.000 y bebida 272.000 — que sumados dan otra vez 798.000. Cuando dos caminos distintos llegan al mismo número, el informe es confiable.

La consulta paramétrica en acción

La consulta paramétrica es la misma consulta de selección con un espacio en blanco que se completa al ejecutar. Seguí una ejecución completa:

Momento	Qué pasa
1. Se ejecuta la consulta	aparece el cuadro: «¿Qué categoría querés ver?»
2. El usuario responde	escribe: bebida
3. El gestor filtra	aplica categoria = "bebida" a los 7 registros
4. Resultado	Gaseosa y Cocido: 2 registros

La gracia comercial: una sola consulta guardada sirve para todas las categorías, hoy y siempre. Sin la paramétrica, el copetín necesitaría una consulta por categoría — y otra nueva cada vez que aparezca una categoría más.

En Access, el criterio de una consulta paramétrica se escribe entre corchetes — [¿Qué categoría querés ver?] — y el gestor lo convierte en una pregunta al usuario cada vez que la consulta se ejecuta.

Las funciones de agrupamiento, una por una

El cálculo de totales del gestor ofrece varias funciones además de contar y sumar. Aplicadas a la tabla Productos, cada una responde su propia pregunta:

Función	Pregunta que responde	Sobre el campo	Resultado
Contar	¿cuántos productos hay?	cualquiera	7
Suma	¿cuánto stock total hay?	stock	321
Máx	¿cuál es el precio más alto?	precio	12.000
Mín	¿cuál es el precio más bajo?	precio	500

Combinadas con el agrupamiento por categoría, estas funciones arman en segundos informes que a mano llevarían una tarde: cuántos productos hay por categoría, el stock de cada rubro, el precio máximo de cada grupo. La consulta se diseña una vez y queda lista para ejecutarse cada semana con los datos frescos.

Agrupar: lo que el gestor hace antes de totalizar

Toda consulta de totales tiene un paso invisible previo: agrupar. El gestor junta los registros que comparten el valor del campo elegido — las tres categorías forman tres grupos — y recién entonces aplica la función a cada grupo por separado:

Grupo (categoría)	Registros que lo integran	Contar
salado	Chipa, Empanada, Sándwich mixto, Sopa paraguaya	4
dulce	Coquito	1
bebida	Gaseosa, Cocido	2

Los tres conteos suman 7 — el total de control de siempre. Entender el agrupamiento como paso propio te prepara para el informe central de la clase siguiente: la referencia cruzada, que no es más que agrupar por DOS campos a la vez.

Actividades de aplicación

Ejercitá:

1. Escribí el criterio de una consulta de selección de productos de categoría "bebida".
2. Indicá qué distingue una consulta paramétrica de una de selección fija.
3. Contá cuántos productos cuestan 5.000 o menos.
4. Calculá el stock total de los productos salados (suma del campo stock con el criterio categoria = "salado").
5. Si el Cocido subiera a 7.000, ¿cuál sería su valorStock?

Resolvé (caso Copetín Karumbé):

6. Formulá una consulta de selección que muestre nombre y precio de los productos que cuestan menos de 6.000, ordenados de menor a mayor precio.
7. Formulá una consulta de totales que cuente, por categoría, los productos con stock menor a 30, e indicá el resultado.
8. La Empanada recibe 20 unidades más. Recalculá su valorStock y el total de valorStock de la tabla.

Pensá y decidí (justificá tu respuesta):

9. V o F: una consulta paramétrica fija el criterio de antemano. Justificá.
10. Elegí: para saber cuántos productos hay por categoría se usa una consulta de (selección / totales). Justificá.
11. V o F: un campo calculado guarda un dato escrito a mano. Justificá.

Desafío:

12. Formulá cuatro consultas para el copetín —una de selección, una paramétrica, una de totales y una con campo calculado— distintas de las de la clase, e indicá el resultado de cada una con la tabla dada.

Práctica 20 — Consultas de la librería: selección, parámetro, totales y campo calculado

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Diseñar, ejecutar y guardar consultas de selección, paramétricas, de totales y con campo calculado, comparando cada resultado con lo anticipado.
Entorno de trabajo: Microsoft Access.

Lo que necesitás saber

Una consulta se arma en la cuadrícula de diseño: campos, orden, criterios. Se guarda con un nombre y se puede volver a ejecutar.

Un parámetro se escribe entre corchetes en la fila Criterios: Access lo pregunta al ejecutar. Un campo calculado se escribe como Nombre: expresión.

Usá la base Arandu.accdb con la tabla Articulos de la Práctica 19. Las rutas de menú corresponden a Access 2016 en castellano, sin soporte desde octubre de 2025; en Access para Microsoft 365 o Access 2024 comprobá las rutas y el separador de las expresiones. Si no hay laboratorio, resolvé las actividades en papel con la tabla de datos de la práctica.

Actividad 1 — Selección con criterio y orden

Objetivo: *Mostrar nombre, precio y stock de los artículos de escritura, ordenados por precio de mayor a menor.*

Modelo — Cuadrícula de diseño

Campos: nombre, precio, stock, categoria.
Criterios: "escritura" en categoria (desmarcá Mostrar en esa columna).
Orden: Descendente en precio.

Pasos:

1. Crear → Diseño de consulta; agregá la tabla Articulos.
2. Anotá el resultado esperado (cantidad y orden) antes de ejecutar.
3. Ejecutá con Diseño → Ejecutar y guardá la consulta como Escritura_por_precio.

Punto de control: El resultado tiene la cantidad y el orden que anotaste, y la columna categoria no se muestra.

Actividad 2 — Una consulta paramétrica

Objetivo: *Pedir la categoría al ejecutar la consulta.*

Modelo — Criterio con parámetro

En la fila Criterios de categoria: [¿Qué categoría?]

Pasos:

1. Creá la consulta con nombre, categoria y stock y el parámetro en categoria.
2. Ejecutala tres veces: papelería, geometría y una categoría que no existe (por ejemplo, arte).
3. Guardala como Por_categoria.

Punto de control: Las dos categorías existentes muestran solo sus artículos y la inexistente devuelve una hoja vacía, sin error.

Actividad 3 — Totales por categoría y valor del stock

Objetivo: *Agrupar por categoría para contar artículos y sumar stock, y calcular el valor del stock de cada artículo.*

Modelo — Dos consultas

Totales: categoria (Agrupar por) · codigo (Cuenta) · stock (Suma).
Campo calculado: ValorStock: [precio]*[stock], con nombre y los dos campos.

Pasos:

1. Para la de totales, activá Diseño → Totales y elegí la función de cada columna.
2. Para la otra, escribí el campo calculado en una columna vacía y agregá una fila de totales en la hoja de datos (Inicio → Totales) para sumar ValorStock.
3. Guardá las dos consultas.

Punto de control: La suma de las cuentas por categoría da 8, la suma de los stocks por categoría coincide con el stock total de la tabla, y el ValorStock de cada artículo es su precio por su stock.

Desafío final (para quienes terminan antes)

Creá una consulta paramétrica con el criterio <=[Precio máximo] en precio y ejecutala con 6000. ¿Qué artículos se pueden comprar con ese presupuesto por unidad?

Clase 21 — Consultas de referencias cruzadas y buenas prácticas

Capacidad	Reconoce la importancia de utilizar correctamente los filtros y consultas a una base de datos.
Tema	Consultas de referencias cruzadas y buenas prácticas
Indicadores de logro	• Explica qué resume una consulta de referencias cruzadas. • Interpreta una tabla de referencias cruzadas de datos del copetín. • Aplica buenas prácticas al formular filtros y consultas.

Resumir en dos dimensiones

Una consulta de referencias cruzadas resume datos en una grilla: agrupa por un campo en las filas y por otro en las columnas, y en el cruce coloca un total. Es como la matriz de la Unidad 2, pero generada por el gestor a partir de la base.

Un ejemplo del copetín

Para ver cuántos productos hay por categoría y por rango de precio, una consulta de referencias cruzadas pone las categorías en las filas, los rangos en las columnas y, en cada cruce, la cantidad. De un vistazo se entiende la composición del menú.

Buenas prácticas al consultar

Una consulta mal formulada engaña. Conviene: escribir criterios claros y precisos; recordar que filtrar no borra (nunca eliminar datos para "ver mejor"); revisar que el resultado tenga sentido (si una consulta de productos salados devuelve un coquito, algo está mal); y nombrar las consultas para reutilizarlas.

Buena práctica	Por qué
Criterios claros y precisos	evita resultados ambiguos
Filtrar, no borrar	no se pierden datos
Revisar el resultado	detecta errores de criterio
Nombrar y guardar la consulta	permite reutilizarla

Por qué importa consultar bien

Las decisiones del copetín (qué reponer, qué producto conviene, cuánto vale el stock) salen de las consultas. Una consulta correcta da información confiable; una mal hecha lleva a decisiones equivocadas.

Concepto clave

Una consulta de referencias cruzadas resume datos en filas y columnas con un total en el cruce. Consultar bien exige criterios claros, no borrar datos al filtrar y revisar que el resultado tenga sentido.

Aplicación profesional — informes de gestión

Los informes que preparan los comercios para analizar sus ventas por sucursal y por mes son, técnicamente, consultas de referencias cruzadas hechas en el gestor de base de datos.

Ejemplo resuelto paso a paso — armar la grilla celda por celda

La consulta de referencias cruzadas del copetín cruza la categoría (filas) con la disponibilidad de stock (columnas: «poco» si stock < 10, «suficiente» si no). Armá la grilla como lo hace el gestor: clasificando cada registro en una celda y contando.

Producto	Categoría (fila)	stock	Columna
Chipa	salado	40	suficiente
Empanada	salado	30	suficiente
Sándwich mixto	salado	8	poco
Coquito	dulce	200	suficiente
Gaseosa	bebida	25	suficiente
Cocido	bebida	12	suficiente
Sopa paraguaya	salado	6	poco

La Figura 21.1 lo muestra en forma gráfica: la grilla terminada: cada producto cae en una sola celda y los totales cierran en 7, la cantidad de registros de la tabla.



Figura 21.1 — La grilla terminada: cada producto cae en una sola celda y los totales cierran en 7, la cantidad de registros de la tabla.

La grilla final cuenta 2 salados con poco stock, 2 con suficiente, 1 dulce y 2 bebidas con suficiente. El control de calidad del informe es el total general: 2 + 2 + 1 + 2 = 7, exactamente los registros de la tabla. Si la grilla sumara 6 u 8, algún registro quedó sin clasificar o se contó dos veces.

Otra cruz con los mismos datos: categoría por rango de precio

Cambiar la pregunta de las columnas produce un informe nuevo sin tocar la tabla. Cruzando categoría con el rango de precio (hasta G. 5.000 / más de G. 5.000):

categoría \ precio	hasta 5.000	más de 5.000	TOTAL
salado	2	2	4
dulce	1	0	1
bebida	0	2	2
TOTAL	3	4	7

La lectura comercial es inmediata: el copetín tiene 3 productos económicos y 4 de precio medio-alto, y todas sus bebidas están en el rango caro. Dos grillas, la misma tabla, decisiones distintas — esa es la potencia de las referencias cruzadas.

Del informe a la decisión

Ningún informe vale por sí mismo: vale por la decisión que habilita. La grilla de stock le dice al dueño qué comprar el lunes; la de precios, dónde le falta variedad económica. Un buen técnico en informática no entrega «datos»: entrega la consulta correcta para la decisión que el negocio tiene que tomar.

La referencia cruzada en Access, paso a paso

En Access, la consulta de referencias cruzadas se arma de dos maneras: con el asistente (Crear → Asistente para consultas → Asistente para consultas de referencias cruzadas) o desde la vista Diseño de una consulta, cambiando el tipo con Tipo de consulta → Tabla de referencias cruzadas. El asistente es el camino más seguro la primera vez, y tiene una condición: trabaja sobre una sola tabla o consulta de origen.

1. Preparar el origen. Si la columna de la grilla no existe en la tabla —como «poco» o «suficiente», que se deduce del stock—, primero se crea una consulta que la calcule con una expresión, por ejemplo Disponibilidad: SiInm([stock]<10;"poco";"suficiente"). En la vista SQL la misma función aparece como IIf.
2. Elegir los encabezados de fila: el campo cuyos valores van a ser las filas (categoría).
3. Elegir el encabezado de columna: el campo cuyos valores van a ser las columnas (Disponibilidad).

4. Elegir el valor del cruce y la función: qué se cuenta o se suma en cada celda (Cuenta de nombre).

5. Ejecutar y controlar: la grilla tiene que sumar los registros del origen. Si no cierra, el error está en el origen o en categorías que se solapan.

Concepto clave — el origen manda

La referencia cruzada solo resume lo que su origen le entrega. Si la disponibilidad se calcula mal en la consulta de origen, la grilla saldrá prolija… y equivocada. Por eso el control se hace en dos lugares: el origen (¿cada registro tiene su valor correcto?) y la grilla (¿cierra en el total?).

Errores típicos al armar una cruz

Las referencias cruzadas fallan casi siempre por el diseño de las categorías, no por el gestor. La lista de control antes de dar por bueno el informe:

Error de diseño	Síntoma	Corrección
Columnas que se solapan	un registro cae en dos celdas y el total da de más	rangos excluyentes: < 10 y >= 10, sin zona gris
Columnas que no cubren todo	registros que no caen en ninguna celda	una categoría «resto» que atrape lo demás
Olvidar el total de control	errores invisibles	la grilla debe sumar los registros de la tabla: 7

La disciplina del total de control te acompaña desde la Unidad 2: la matriz cerraba en 243 por filas y por columnas, y la grilla cierra en 7 por donde se la sume. Todo informe serio trae incorporada su propia verificación — esa es la marca de un técnico que no entrega números sin red.

Actividades de aplicación

Ejercitá:

1. Indicá qué coloca una consulta de referencias cruzadas en el cruce de fila y columna.

2. En una grilla categoría × stock con las columnas «25 o más» y «menos de 25», ¿cuántos productos salados caen en cada columna?

3. En esa misma grilla, ¿cuántos productos hay en total en la columna «menos de 25»?

4. Nombrá dos buenas prácticas al consultar una base de datos.

5. V o F: para "ver mejor" conviene borrar los registros que molestan.

Resolvé (caso Copetín Karumbé):

6. Interpretá la grilla de referencias cruzadas categoría × precio de la clase: ¿qué categoría no tiene ninguna opción de hasta 5.000 y qué decisión podría tomar el copetín con ese dato?

7. Proponé una consulta de referencias cruzadas distinta de las de la clase (elegí qué va en filas y qué en columnas) y explicá qué mostraría.

8. Señalá dos errores posibles al formular una consulta y cómo los detectarías revisando el resultado.

Pensá y decidí (justificá tu respuesta):

9. V o F: una consulta de referencias cruzadas resume datos en filas y columnas. Justificá.

10. Elegí: si una consulta de productos salados devuelve un dulce, el problema está en (el criterio / el orden de las columnas). Justificá.

11. V o F: nombrar y guardar una consulta permite reutilizarla. Justificá.

Desafío:

12. Diseñá una consulta de referencias cruzadas que cruce categoría (filas) con tres tramos de stock (columnas: «menos de 10», «de 10 a 29», «30 o más») y completá la grilla con los datos de la tabla Productos, con totales.

Práctica 21 — La referencia cruzada de la librería y el informe de reposición

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Preparar el origen con un campo calculado condicional, construir una consulta de referencias cruzadas con el asistente y controlarla contra la tabla.
Entorno de trabajo: Microsoft Access.

Lo que necesitás saber

El asistente de referencias cruzadas trabaja sobre una sola tabla o consulta: si una columna no existe, se calcula antes en una consulta de origen.

En la cuadrícula de Access en castellano, la función condicional se escribe SiInm(condición;valor_si;valor_no); en la vista SQL aparece como IIf.

Usá Arandu.accdb. Las rutas de menú corresponden a Access 2016 en castellano, sin soporte desde octubre de 2025; en Access para Microsoft 365 o Access 2024 comprobá las rutas y el separador de las expresiones. Si no hay laboratorio, resolvé las actividades en papel con la tabla de datos de la práctica.

Actividad 1 — La consulta de origen

Objetivo: *Crear una consulta que agregue a cada artículo su nivel de stock: «reponer» si el stock es menor que 20 y «ok» si no.*

Modelo — Campo calculado

Nivel: SiInm([stock]<20;"reponer";"ok")

Pasos:

1. Creá una consulta con nombre, categoria, stock y el campo calculado Nivel.
2. Antes de ejecutar, marcá en la tabla de datos qué artículos deberían decir «reponer».
3. Ejecutá, revisá la vista SQL para ver cómo aparece la expresión y guardá la consulta como Origen_nivel.

Punto de control: Cada artículo tiene el nivel que marcaste en la tabla de datos. Si Access rechaza la expresión, revisá el separador (punto y coma) y las comillas.

Actividad 2 — La referencia cruzada con el asistente

Objetivo: *Construir la grilla categoría × nivel con el asistente.*

Modelo — Opciones del asistente

Crear → Asistente para consultas → Asistente para consultas de referencias cruzadas.
Origen: la consulta Origen_nivel.
Encabezado de fila: categoria · encabezado de columna: Nivel · valor: nombre con la función de conteo.

Pasos:

1. Antes de usar el asistente, dibujá en papel la grilla con sus celdas completas.
2. Recorré el asistente con las opciones del modelo y guardá la consulta como Cruce_nivel.
3. Compará la grilla de Access con la de papel.

Punto de control: Las celdas de Access coinciden con tu grilla de papel y la suma de todas las celdas da la cantidad de artículos de la tabla.

Actividad 3 — Del dato a la decisión

Objetivo: *Usar la grilla para redactar un informe breve de reposición.*

Modelo — El informe responde

¿Qué categorías necesitan reposición y cuántos artículos?
¿Qué categoría no tiene ningún artículo para reponer?
¿Qué artículo pedirías primero y por qué (mirá también el stock)?

Pasos:

1. Respondé las tres preguntas con datos de la grilla y de la consulta de origen.
2. Cambiá en la tabla el stock de la Regla a 25, volvé a ejecutar Cruce_nivel y anotá qué celda cambió.
3. Devolvé el stock de la Regla a 18.

Punto de control: Después del cambio, la grilla sigue sumando 8 y solo cambiaron las dos celdas de la fila de la categoría de la Regla.

Desafío final (para quienes terminan antes)

Cambiá la grilla para que en lugar de contar artículos sume el stock. ¿Qué información nueva da y qué información se pierde?

Evaluación de la Unidad 4

Consultas a bases de datos (Clases 19 a 21)

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Datos de la evaluación — tabla Insumos de la heladería Yvoty

INS-01 Leche 9.000 lácteo 40 · INS-02 Crema 18.000 lácteo 12 · INS-03 Azúcar 7.000 seco 25 · INS-04 Cacao 30.000 seco 6 · INS-05 Frutilla 22.000 fruta 9 · INS-06 Limón 8.000 fruta 30

Campos: codigo, nombre, precio, categoria, stock.

Parte A — Diagnóstico rápido (marcá la opción correcta):

1. En una tabla, cada columna (código, precio, stock) es un:
   a) registro
   b) campo
   c) filtro
   d) total

2. La consulta que pide el criterio al usuario al ejecutarse es la:
   a) de selección
   b) de totales
   c) paramétrica
   d) de referencias cruzadas

3. Un filtro, respecto de los registros que no cumplen la condición:
   a) los borra
   b) los copia
   c) los ordena
   d) los oculta sin borrarlos

Parte B — Resolución y aplicación:

4. Indicá cuántos registros y cuántos campos tiene la tabla Insumos y qué tipo de dato le corresponde a precio.

5. Aplicá el filtro stock < 15 e indicá qué insumos quedan visibles.

6. Formulá una consulta de selección que muestre nombre y precio de los insumos de categoría "seco", de mayor a menor precio.

7. Formulá una consulta de totales que cuente los insumos y sume el stock por categoría, e indicá el resultado.

8. Escribí el campo calculado ValorStock y calculalo para la Crema y el Limón.

9. Completá una consulta de referencias cruzadas categoría × nivel («reponer» si stock < 15, «ok» si no) y escribí el campo calculado que la prepara.

Evaluación integradora de la 2.ª etapa

Unidades 1 a 4 · Cierre de la 2.ª etapa (noviembre)

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Datos de la evaluación — tabla Insumos de la heladería Yvoty (carga de noviembre)

HEL-11 Yogur 11.000 lácteo 28 · HEL-12 Queso 16.000 lácteo 8 · HEL-13 Harina 6.000 seco 35 · HEL-14 Galleta 14.000 seco 11 · HEL-15 Mango 12.000 fruta 16 · HEL-16 Naranja 5.000 fruta 22

Campos: codigo, nombre, precio, categoria, stock.

Parte A — Diagnóstico rápido (marcá la opción correcta):

1. Para recorrer una matriz completa se necesitan:
   a) un filtro
   b) dos ciclos anidados
   c) un ciclo
   d) ningún ciclo

2. Un archivo se guarda en la:
   a) RAM
   b) consulta
   c) memoria secundaria
   d) variable

3. Un filtro sobre una tabla:
   a) muestra solo los registros que cumplen la condición
   b) ordena la tabla
   c) borra registros
   d) crea un campo

Parte B — Resolución y aplicación:

4. Escribí un algoritmo con Para que cargue el vector stock[6] con los stocks de Insumos, calcule el stock total y cuente cuántos insumos tienen menos de 15 unidades.

5. Ordená de mayor a menor el vector de precios con el método de selección e indicá qué insumo queda primero.

6. Definí una función ValorStock(precio, stock) y calculá el valor del stock de Harina.

7. Escribí en la notación del libro el procedimiento que guarda los seis insumos en insumos.txt, una línea por insumo con el formato código;stock.

8. Aplicá a Insumos el filtro stock < 15 y una consulta de totales que sume el stock por categoría; indicá ambos resultados y verificá que el total coincida con el del ejercicio 4.

9. Escribí el campo calculado ValorStock y calculalo para el Queso; después explicá qué decisión podría tomar la heladería con ese dato y con su stock.

Proyecto Final Integrador — El sistema de gestión de un emprendimiento

Durante todo el año trabajaste sobre un emprendimiento paraguayo: el Copetín Karumbé. Ahora, en equipo, vas a construir el sistema de gestión de un emprendimiento —el copetín u otro que elija tu grupo— y presentarlo en la Feria de Informática de fin de año.

El proyecto reúne todo lo que aprendiste en Algorítmica (algoritmos, estructuras de datos, archivos y bases de datos) y se conecta con las demás materias prácticas de Informática: cada una aporta su parte para un mismo producto.

Competencia integradora

Diseñar y presentar, en equipo, un sistema de gestión sencillo para un emprendimiento, aplicando algoritmos, una base de datos y las herramientas de las demás asignaturas prácticas.

Lo que aportás desde Algorítmica

- Un algoritmo de caja en PSeInt que lea los productos y cantidades de una venta, calcule el total, aplique el descuento del emprendimiento cuando corresponda y muestre el ticket, con condicionales, ciclos y al menos un módulo.
- Una base de datos en Access con la tabla de productos del emprendimiento y sus consultas.
- Un informe de resultados: el ranking de los productos más vendidos (ordenación) y el total y el promedio de una semana de ventas (vector y acumulador).

Este es el flujograma del algoritmo de caja que tu equipo va a programar en PSeInt. Fijate cómo reúne todo lo del año: un ciclo de carga, un módulo, una decisión y el ticket final.

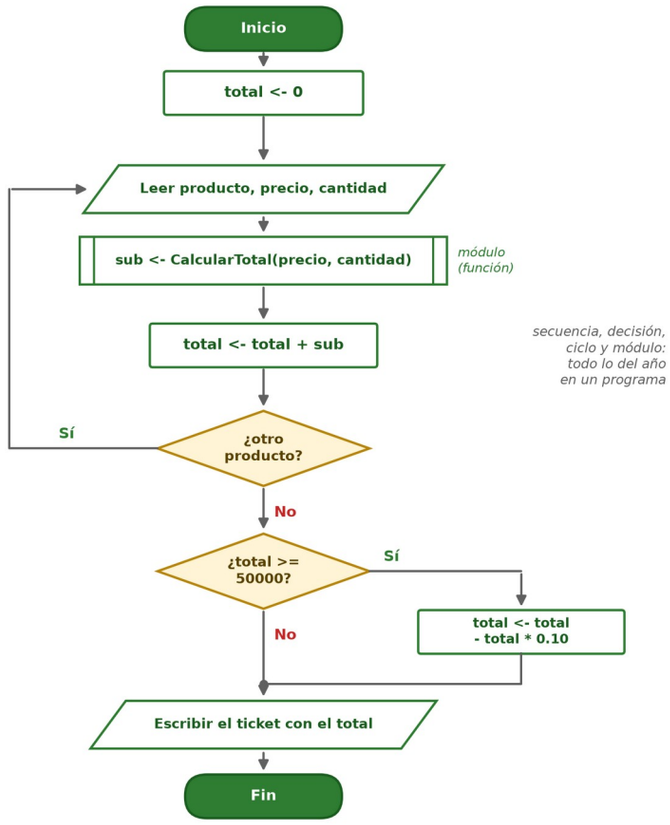


Figura PFI.1 — Flujograma del algoritmo de caja del proyecto: ciclo de carga, módulo CalcularTotal, descuento del 10 % cuando el total llega a 50.000 y ticket final.

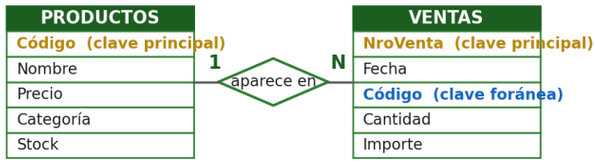
Requisitos mínimos de la solución

Componente	Qué debe tener
Algoritmo de caja	Programa en PSeInt que corre sin errores: carga de varios productos con un ciclo, al menos un módulo (función o procedimiento), decisión del descuento y ticket final. Probado con al menos tres casos anotados antes de ejecutar.
Estructuras de datos	Un vector con las ventas de una semana (total y promedio) y un ranking de productos ordenado con burbuja o selección.
Base de datos	Tabla de productos en Access con tipos de datos y clave principal, y datos propios del emprendimiento (no los del libro).
Consultas	Al menos cuatro: un filtro o consulta de productos a reponer, una de selección con criterio, una de totales por categoría y una de referencias cruzadas.
Presentación	Afiche o presentación del emprendimiento y guion de una demostración de cinco minutos.

Extensión recomendada: agregá una tabla Ventas relacionada con Productos y guardá el ticket de cada venta en un archivo de texto organizado en carpetas. No son requisitos mínimos para aprobar el proyecto, pero acercan la solución a un sistema de gestión real.

Y esta es la base de datos del proyecto en Access, en su versión ampliada: la tabla Productos y, como extensión recomendada, una tabla Ventas conectada por el campo Código.

Diagrama entidad-relación de la base del Copetín Karumbé



Un producto puede aparecer en muchas ventas; cada línea de venta corresponde a UN producto (relación 1 a N). El campo Código conecta las dos tablas: es clave principal en PRODUCTOS y clave foránea en VENTAS.

Figura PFI.2 — Las dos tablas de la versión ampliada del proyecto: Productos (1) y Ventas (N), conectadas por el campo Código.

Etapas del proyecto

Taller	Qué hacés
Taller 1 — Definición	Elegís el emprendimiento en equipo, listás sus productos y precios, decidís qué va a resolver el sistema y repartís los roles.
Taller 2 — Construcción	Programás y probás el algoritmo de caja en PSeInt y creás la base de datos en Access con sus consultas.
Taller 3 — Integración y ensayo	Sumás los aportes de las otras materias, verificás cada resultado por otro camino y ensayás la demostración para la feria.

Trabajo interdisciplinario: qué aporta cada materia

Materia	Aporte a la solución digital
Algorítmica	El algoritmo de caja, las estructuras de datos y la base de datos con sus consultas.
Matemática Aplicada a la Informática	Los cálculos y la estadística: porcentajes, promedios y los gráficos de las ventas.
Gabinete de Software	El informe, la planilla con gráficos y la presentación para el stand.
Gabinete de Laboratorio / Hardware	La preparación del equipo del stand, las conexiones y el resguardo de los archivos del proyecto.
Dibujo Técnico	El diagrama de flujo del sistema, el logo del emprendimiento y el plano del stand.

Qué entrega el equipo

- El sistema funcionando: el algoritmo en PSeInt (.psc) y la base de datos en Access (.accdb) con sus consultas guardadas.
- La carpeta del proyecto con los casos de prueba del algoritmo, el informe de resultados y el reparto de roles.
- Un afiche o una presentación que explique el emprendimiento y qué hace el sistema, y la demostración en vivo en el stand.

Pautas para el equipo

- Trabajen en equipos de 3 o 4 integrantes, con roles claros (algoritmo, base de datos, informe, presentación).
- Usen el mismo emprendimiento en todas las materias.
- Anoten el resultado esperado de cada prueba antes de ejecutar y verifiquen cada consulta por otro camino.
- Sean claros y honestos en la presentación: qué resuelve el sistema y qué quedó pendiente.

Algorítmica · Algoritmos, archivos y consultas

2.° Curso · Bachillerato Técnico en Servicios · Especialidad Informática

En este libro

- 21 clases desarrolladas con ejemplos resueltos y diagramas
- 21 prácticas paso a paso en PSeInt, en el Explorador de archivos y en Microsoft Access
- Estructuras de control, vectores, matrices, módulos, archivos y consultas
- Prueba diagnóstica, 4 evaluaciones de unidad y 2 de etapa
- Proyecto Final Integrador para la Feria de Informática

Para el docente

- Solucionario docente con todas las respuestas
- Plan anual de 36 encuentros (4 horas cátedra semanales)
- 36 planes de clase listos para usar

Enfoque «Aprender Haciendo»

Caso integrador: Copetín Karumbé

República del Paraguay · 2026